



Vivekanand Education Society's

Institute of Technology

An Autonomous Institute Affiliated to University of Mumbai

Hashu Advani Memorial Complex, Collector Colony, Chembur East, Mumbai - 400074.

Department of Information Technology

CERTIFICATE

This is to certify that **Mohit Patil** of **D20A** semester **VIII**, have successfully completed necessary experiments in the **ITC801: Blockchain & DLT Lab** under my supervision in **VES Institute of Technology** during the academic year **2025-2026**.

Subject Teacher

Name: Mrs. Kajal Joseph

Signature:

Head of Department

Name: Dr. Mrs. Shalu Chopra

Signature:

Lab Teacher

Name: Kajal Joseph

Signature:



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY**
Department of Information Technology
Academic Year: 2025-26

Lab Code	Lab Name	Credit
ITC801	Blockchain and DLT Lab	1

Programme Outcomes: The graduate will be able to:

PO1	Engineering Knowledge: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.
PO2	Problem Analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development. (WK1 to WK4)
PO3	Design/Development of Solutions: Design creative solutions for complex engineering problems and design / develop systems /components / processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required. (WK5)
PO4	PO4: Conduct Investigations of Complex Problems: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions. (WK8).
PO5	Engineering Tool Usage: Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems. (WK2 and WK6)
PO6	The Engineer and The World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment. (WK1, WK5, and WK7).
PO7	Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws. (WK9)
PO8	Individual and Collaborative Team work: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.
PO9	Communication: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences
PO10	Project Management and Finance: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY**
Department of Information Technology
Academic Year: 2025-26

PO11	Life-Long Learning: Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change.
PSO 1	Computing & Emerging Technology: Apply core computing and modern technologies, such as AI, Cloud Computing, IoT, cybersecurity, and data-driven tools, to build efficient and innovative solutions.
PSO 2	Professionalism & Innovation : Demonstrate ethics, teamwork, and an entrepreneurial mindset to deliver sustainable solutions for society.

Course Outcomes: Student will be able	
ITC801.1	Describe the basic concept of Blockchain and Distributed Ledger Technology.
ITC801.2	Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions
ITC801.3	Implement smart contracts in Ethereum using different development frameworks.
ITC801.4	Develop applications in permissioned Hyperledger Fabric network.
ITC801.5	Interpret different Crypto assets and Crypto currencies
ITC801.6	The use of Blockchain with AI, IoT and Cyber Security using case studies.

ITC801 Blockchain and DLT Lab													
CO	PO											PSO	
	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PSO 1	PSO 2
ITC801.1	2	3	3	3	3	3	3	3	3	2	2	3	3
ITC801.2	2	3	3	3	3	3	3	3	3	2	2	3	3
ITC801.3	2	3	3	3	3	3	3	3	3	2	2	3	3
ITC801.4	2	3	3	3	3	3	3	3	3	2	2	3	3
ITC801.5	2	3	3	3	3	3	3	3	3	2	2	3	3
ITC801.6	2	3	3	3	3	3	3	3	3	2	2	3	3
Avg PO AL	2	3	3	3	3	3	3	3	3	2	2	3	3



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY**
Department of Information Technology
Academic Year: 2025-26

ITC801 Blockchain and DLT Lab												
CO	Experiment											
	Exp 1	Exp 2	Exp 3	Exp 4	Exp 5	Exp 6	Exp 7	Exp 8	Exp 9	Exp 10	Exp 11	Exp 12
ITC801.1	3	3	2	-	2	2	2	-	-	3	-	3
ITC801.2	2	3	3	-	2	2	3	2	-	3	-	3
ITC801.3	-	-	2	3	3	3	3	3	2	-	-	3
ITC801.4	-	-	-	-	-	-	-	-	-	-	3	3
ITC801.5	2	3	3	-	-	2	2	2	-	3	-	3
ITC801.6	-	-	-	-	-	-	-	-	-	2	-	3

INDEX

Sr. No.	List of Experiments
1	Write a Python program to understand SHA and Cryptography in Blockchain, Merkle root tree hash
2	Create a Blockchain using Python
3	Create a Cryptocurrency using Python and perform mining in the Blockchain created.
4	Hands-on Solidity Programming Assignments for creating Smart Contracts
5	Deploying a Voting/Ballot Smart Contract
6	Creating Smart Contracts and performing transactions using Solidity and Remix IDE
7	Implement the embedding wallet (Metamask) and transaction using Solidity
8	To implement the Blockchain platform Ganache and deploy a simple Solidity smart contract using Metamask and Remix IDE.
9	To implement a Private Ethereum Blockchain using Geth.
10	Explore the Cryptocurrency Landscape
11	Implementation of Hyperledger Fabric Network and Asset Management using Chaincode
12	Mini Project
12	Assignment-1



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY**
Department of Information Technology
Academic Year: 2025-26

13	Assignment-2
14	Assignment-3
15	Mock Viva

Subject teacher



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment No. 01

Aim: Write a Python program to understand SHA and Cryptography in Blockchain, Merkle root tree hash

Roll No.	04
Name	Mohit Patil
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1: Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2: Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO5: Interpret different Crypto assets and Crypto currencies

Blockchain Experiment 1

AIM: Cryptography in Blockchain, Merkle root Tree Hash

THEORY:

Q1: Cryptographic Hash functions in Blockchain

Cryptographic hash functions are mathematical algorithms that convert input data of any size into a fixed-length string of characters, known as a hash value. These functions are fast, deterministic, and one-way, meaning that the original input cannot be retrieved from the hash.

Working of Cryptographic Hash Functions

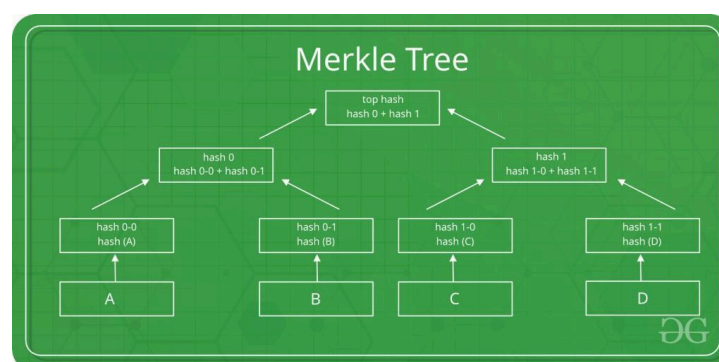
- **Input Processing**
The function accepts input data of any length, such as text, files, or data streams, and processes it using mathematical operations.
- **Fixed-Size Output Generation**
Regardless of input size, the output hash has a fixed length, usually represented as a hexadecimal string.
- **Deterministic Operation**
The same input always produces the same hash, enabling reliable data verification.
- **Avalanche Effect**
A minor change in input, even a single bit, results in a completely different hash output.
- **One-Way Computation**
It is computationally infeasible to reverse the hash to obtain the original input.
- **Collision Resistance**
The probability of two different inputs generating the same hash value is extremely low.

Popular Cryptographic Hash Algorithms

- MD5 (Message Digest Algorithm 5)
- SHA-1 (Secure Hash Algorithm 1)
- SHA-2 Family (SHA-256, SHA-512)
- SHA-3 (Keccak)
- BLAKE2 and BLAKE3

Q2: What is a Merkle Tree?

A Merkle Tree, also known as a Hash Tree, is a tree-based data structure in which each leaf node stores the hash of a data block, and each non-leaf node stores the hash of its child nodes. The final hash at the top of the tree is called the Merkle Root.



Structure of a Merkle Tree

- **Leaf Nodes:**
Contain hash values of individual transactions or data blocks.
- **Internal Nodes:**
Store hashes generated by combining and hashing child nodes.
- **Merkel Root:**
The topmost node representing the combined hash of all underlying data.

Q3: What is a Cryptographic Puzzle and explain the Golden Nonce Cryptographic Puzzle

A cryptographic puzzle is a problem that miners must solve to:

- Add a new block to the blockchain
- Prove computational work (Proof-of-Work)

The puzzle requires finding a hash that satisfies a difficulty condition.

Golden Nonce

- A nonce is a random number added to block data
- Miners repeatedly change the nonce and recompute the hash
- The Golden Nonce is the specific nonce value that produces a hash meeting the required difficulty (e.g., hash starts with a certain number of zeros)

Q4: How does a Merkle Tree work?

Step-by-Step Working

- a. Each transaction is hashed using a cryptographic hash function
- b. Hashes are paired and combined
- c. Combined hashes are hashed again
- d. If the number of hashes is odd, the last hash is duplicated
- e. This process continues until one hash remains
- f. The final hash is the Merkle Root

Q5: Benefits of Merkle Tree

- Reduces the amount of data required for verification
- Improves data integrity and security
- Enables fast synchronization between nodes
- Suitable for large-scale systems

Q6: Use cases of Merkle Tree

- **Blockchain Technology**
Used to store and verify transactions in a block.
- **Distributed Systems**
Ensures consistency of data across nodes.
- **Version Control Systems**
Used in systems like Git for tracking file changes.

- **Database Verification**
Detects unauthorized data modification.

TASKS PERFORMED:

Task 1: Hash Generation using SHA-256: Developed a Python program to compute a SHA-256 hash for any given input string using the hashlib library.

```
import hashlib

def create_hash(string):
    # Create a hash object using SHA-256 algorithm
    hash_object = hashlib.sha256()
    # Convert the string to bytes and update the hash
    # object
    hash_object.update(string.encode('utf-8'))
    # Get the hexadecimal representation of the hash
    hash_string = hash_object.hexdigest()
    # Return the hash string
    return hash_string

# Example usage
input_string = input("Enter a string: ")
hash_result = create_hash(input_string)
print("Hash:", hash_result)
```

Output:

```
... Enter a string: Mohit
Enter the nonce: 1
Hash Code: c6c672adb6cd1e798e2d03306ea3fe2f&cd2b35eda
29a02c&8a64dd6660bc58a6
```

Task 2: Target Hash Generation with Nonce: Created a program to generate a hash code by concatenating a user input string and a nonce value to simulate the mining process.

```
import hashlib

# Get user input
input_string = input("Enter a string: ")
nonce = input("Enter the nonce: ")

# Concatenate the string and nonce
hash_string = input_string + nonce

# Calculate the hash using SHA-256
```

```
hash_object = hashlib.sha256(hash_string.encode('utf-8'))
hash_code = hash_object.hexdigest()
```

```
# Print the hash code
print("Hash Code:", hash_code)
```

Output:

```
... Enter a string: Mohit
Enter the nonce: 1
Hash Code: c6c672adb6cd1e798e2d03306ea3fe2f&cd2b35eda
```

Task 3: Proof-of-Work Puzzle Solving: Implemented a program to find the nonce that, when combined with a given input string, produces a hash starting with a specified number of leading zeros.

```
import hashlib
def find_nonce(input_string, num_zeros):
    nonce = 0
    hash_prefix = '0' * num_zeros

    while True:
        # Concatenate the string and nonce
        hash_string = input_string + str(nonce)
        # Calculate the hash using SHA-256
        hash_object = hashlib.sha256(hash_string.encode('utf-8'))
        hash_code = hash_object.hexdigest()

        # Check if the hash code has the required number of leading zeros
        if hash_code.startswith(hash_prefix):
            print("Hash:", hash_code)
            return nonce

        nonce += 1

# Get user input
input_string = "Mohit"
num_zeros = 1

# Find the expected nonce
expected_nonce = find_nonce(input_string, num_zeros)

# Print the expected nonce
print("Input String:", input_string)
```

```
print("Leading Zeros:", num_zeros)
print("Expected Nonce:", expected_nonce)
```

```
... Input String: Mohit
    Leading Zeros: 1
    Expected Nonce: 3
    Hash: 0a74ef1bca
    5574860bcd141665468433f64C3c231fabfeded22df8bfd2d8a7f196e5ab39
```

Task 4: Merkle Tree Construction: Built a Merkle Tree from a list of transactions by recursively hashing pairs of transaction hashes, doubling up last nodes if needed, and generated the Merkle Root hash for blockchain transaction integrity.

```
import hashlib
def build_merkle_tree(transactions):
    if len(transactions) == 0:
        return None
    # Hash each transaction initially
    hashed_transactions = [
        hashlib.sha256(t.encode('utf-8')).hexdigest()
        for t in transactions
    ]
    print("Initial Hashed Transactions:", hashed_transactions)
    if len(hashed_transactions) == 1:
        return hashed_transactions[0]
    # Recursive construction of the Merkle Tree
    current_level = hashed_transactions
    level = 1
    while len(current_level) > 1:
        print(f"\nLevel {level} Hashes:")
        # If odd number of nodes, duplicate last
        if len(current_level) % 2 != 0:
            current_level.append(current_level[-1])
        new_level = []
        for i in range(0, len(current_level), 2):
            combined = current_level[i] + current_level[i + 1]
            hash_combined = hashlib.sha256(
                combined.encode('utf-8')
            ).hexdigest()
            new_level.append(hash_combined)
        print(
            f"    Combining {current_level[i][:6]}... and "
```

```

        f"{current_level[i+1][:6]}... to get
{hash_combined[:6]}..."
    )
    current_level = new_level
    level += 1
    return current_level[0]

# Example usage
transactions = [
    "Anuprita -> Sneha : 800",
    "Sneha -> Shreya : 540",
    "Shreya -> Ria : 100",
    "Eesha -> Anuprita : 500",
    "Ria -> Sneha : 680"
]

print("\nMerkle Tree Generation")
merkle_root = build_merkle_tree(transactions)

print("\nMerkle Root:", merkle_root)

```

```

'''
Merkle Tree Generation
Initial Hashed Transactions: ['4d5a82f66fccc5af28031a82a5ceb39dc3fe23097c56c8dff7720841a02aef1f', 'fdb9eafb0c7f5403
Level 1 Hashes:
    Combining 4d5a82... and fdb9e... to get 9cd41e...
    Combining 43dd57... and 2ed5c8... to get 9f37c0...
    Combining 4313e3... and 4313e3... to get bc2172...
Level 2 Hashes:
    Combining 9cd41e... and 9f37c0... to get 6fa7f1...
    Combining bc2172... and bc2172... to get ba86e9...
Level 3 Hashes:
    Combining 6fa7f1... and ba86e9... to get c5db5f...
Merkle Root: c5db5fb4092cf71773baf667101b0158168f44d9a99579c5385353c80ef38aff
'''

```

CONCLUSION:

In this experiment, cryptographic hash functions and Merkle Tree construction were successfully implemented using Python. SHA-256 hashing ensured data integrity, while nonce-based Proof-of-Work and Merkle Root generation demonstrated secure and tamper-resistant transaction verification in blockchain systems.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY**
Department of Information Technology
Academic Year: 2025-26

ITC801: Blockchain & DLT Lab
Experiment 02

Aim: Create a Blockchain using Python

Roll No.	04
Name	Mohit Patil
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1: Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2: Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO5: Interpret different Crypto assets and Crypto currencies

Blockchain Experiment 2

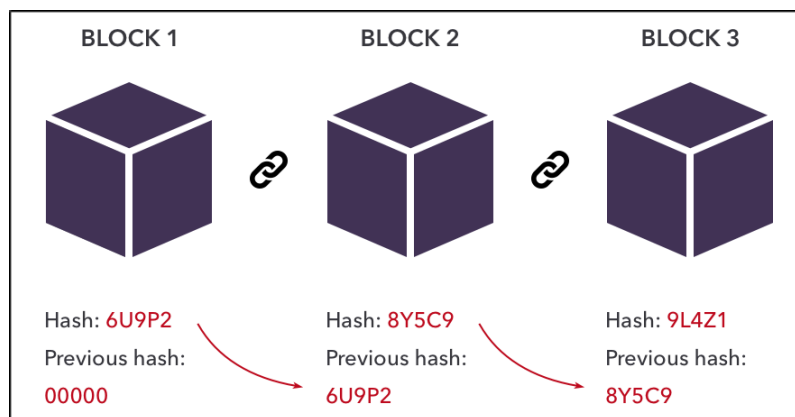
AIM: Create a Blockchain using Python

THEORY:

Q1: What is a Blockchain?

A Blockchain is a distributed and decentralized digital ledger that records transactions across multiple computers in a secure and transparent manner. Each record is stored in a block, and these blocks are linked together using cryptographic hash values, forming a chain.

Once data is added to a block and confirmed, it cannot be altered or deleted, which makes blockchain highly secure and tamper-resistant. Blockchain technology removes the need for a central authority and allows participants in the network to trust the system through cryptographic techniques and consensus mechanisms.



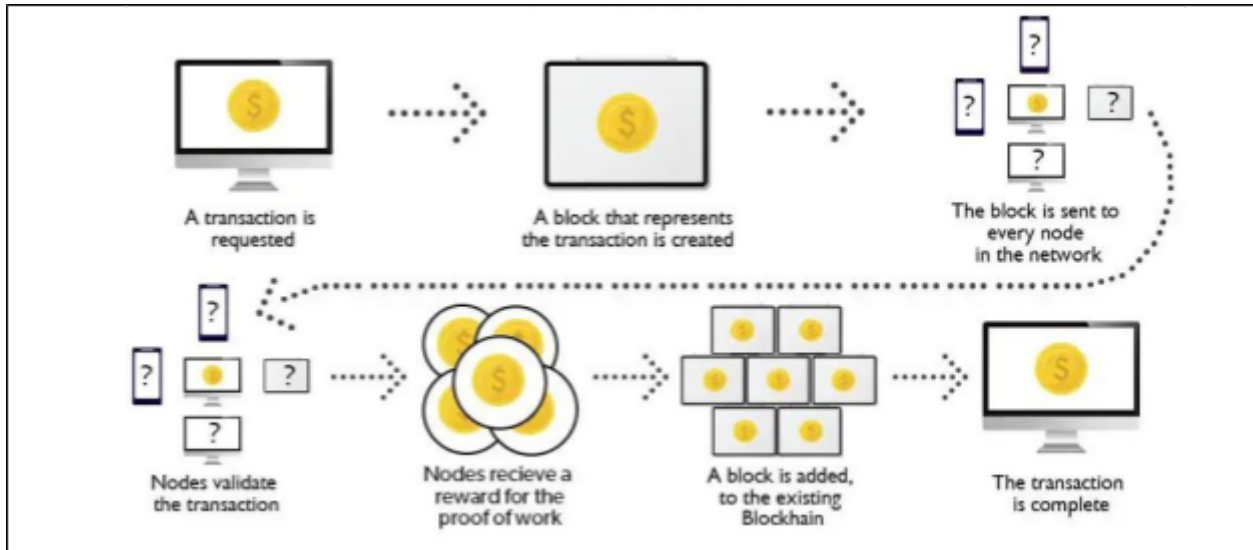
Q2: Process of Mining

Mining is the process of adding new blocks to the blockchain. It involves validating transactions and solving a cryptographic puzzle using computational power.

Steps in the Mining Process:

- Transactions are collected and grouped into a block.
- Miners calculate the hash of the block data along with a nonce.
- The nonce is repeatedly changed until the hash meets the network's difficulty condition (e.g., leading zeros).
- The first miner to find the valid hash broadcasts the block to the network.
- Other nodes verify the solution.
- Once verified, the block is added to the blockchain.

Mining ensures network security, prevents double spending, and maintains consensus among participants.



Q3: How to check the validity of blocks in a Blockchain

The validity of blocks in a blockchain is checked to ensure that the data has not been tampered with and that all blocks follow the network rules.

Block Validation Process:

- **Previous Hash Verification:**
Each block stores the hash of the previous block. If this hash does not match the actual hash of the previous block, the chain is invalid.
- **Proof-of-Work Verification:**
The block's proof value is checked to confirm that it satisfies the required difficulty condition.
- **Hash Integrity Check:**
The block data is re-hashed to ensure the hash value remains unchanged.
- **Chain Continuity Check:**
Blocks are validated sequentially from the genesis block to the latest block.

If all these conditions are satisfied, the blockchain is considered valid and secure.

TASKS PERFORMED:

Task: Implementation of a Simple Blockchain with Proof-of-Work and Flask API

Importing libraries

```
import datetime          # To generate timestamps for
blocks import hashlib    # To generate SHA-256 hashes
import json              # To convert block data into JSON
format from flask import Flask, jsonify # To create REST API
endpoints
```

Part 1 - Building the Blockchain

```
class Blockchain:
```

```
    def __init__(self):
```

```

# List to store the blockchain
self.chain = []

# Create the genesis block
self.create_block(proof=1, previous_hash='0')

def create_block(self, proof, previous_hash):
    """
    Create a new block and add it to the blockchain
    """
    block = {
        'index': len(self.chain) + 1,
        'timestamp': str(datetime.datetime.now()),
        'proof': proof,
        'previous_hash': previous_hash
    }

    self.chain.append(block)
    return block

def get_previous_block(self):
    """
    Return the last block in the chain
    """
    return self.chain[-1]

def proof_of_work(self, previous_proof):
    """
    Proof of Work Algorithm:
    Find a number such that the hash of
    (new_proof^2 - previous_proof^2)
    starts with '0000'
    """
    new_proof = 1
    check_proof = False

    while not check_proof:
        hash_operation = hashlib.sha256(
            str(new_proof**2 - previous_proof**2).encode()
        ).hexdigest()

```

```

        if hash_operation[:4] == '0000':
            check_proof = True
        else:
            new_proof += 1

    return new_proof

def hash(self, block):
    """
    Create a SHA-256 hash of a block
    """
    encoded_block = json.dumps(block, sort_keys=True).encode()
    return hashlib.sha256(encoded_block).hexdigest()

def is_chain_valid(self, chain):
    """
    Check whether the blockchain is valid
    """
    previous_block = chain[0]
    block_index = 1

    while block_index < len(chain):
        block = chain[block_index]

        # Check previous hash
        if block['previous_hash'] != self.hash(previous_block):
            return False

        # Check proof of work
        previous_proof = previous_block['proof']
        proof = block['proof']

        hash_operation = hashlib.sha256(
            str(proof**2 - previous_proof**2).encode()
        ).hexdigest()

        if hash_operation[:4] != '0000':
            return False

```

```

        previous_block = block
        block_index += 1

    return True

# Part 2 - Mining the Blockchain (Flask API)
# Create Flask app
app = Flask(__name__)

# Create Blockchain object
blockchain = Blockchain()

# API Endpoint - Mine a new block
@app.route('/mine_block', methods=['GET'])
def mine_block():
    previous_block = blockchain.get_previous_block()
    previous_proof = previous_block['proof']

    proof = blockchain.proof_of_work(previous_proof)
    previous_hash = blockchain.hash(previous_block)

    block = blockchain.create_block(proof, previous_hash)

    response = {
        'message': 'Congratulations Mohit, you just mined a block!',
        'index': block['index'],
        'timestamp': block['timestamp'],
        'proof': block['proof'],
        'previous_hash': block['previous_hash']
    }

    return jsonify(response), 200

# API Endpoint - Get full blockchain
@app.route('/get_chain', methods=['GET'])
def get_chain():
    response = {
        'chain': blockchain.chain,

```

```

        'length': len(blockchain.chain)
    }
    return jsonify(response), 200

# API Endpoint - Check blockchain validity
@app.route('/is_valid', methods=['GET'])
def is_valid():
    valid = blockchain.is_chain_valid(blockchain.chain)

    if valid:
        response = {'message': 'Blockchain is valid, Mohit.'}
    else:
        response = {'message': 'Blockchain is NOT valid, Mohit.'}

    return jsonify(response), 200

# Run the Flask Application
if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)

```

```

{
  "index": 2,
  "message": "Congratulations Mohit, you just mined a block!",
  "previous_hash": "0ebe338dfafa5872f2c36be444f98c12a34db34c84o8c6663076",
  "proof": 533,
  "timestamp": "202-01-30 23:13:56.889069"
}

```



```
{
  "chain": [
    {
      "index": 1,
      "previous_hash": "0",
      "proof": 1,
      "timestamp": "2026-01-30 23:13:36.861288"
    },
    {
      "index": 2,
      "previous_hash": "0ebe338dfafa5872f2c36be444f98c12a34db61c46464a842c9e2948c6863076",
      "proof": 533,
      "timestamp": "2026-01-30 23:13:56.889069"
    }
  ],
  "length": 2
}
```



```
{
  "message": "Blockchain is valid, Mohit."
}
```

CONCLUSION:

In this experiment, a simple blockchain was successfully implemented using Python and Flask. The system allows block mining, viewing the complete blockchain, and checking its validity. Proof-of-Work was used to make block creation secure, while cryptographic hashing ensured that the data stored in blocks remains unchanged. This implementation helps in understanding the basic concepts of blockchain such as immutability, consensus, and decentralization, and provides a good foundation for learning advanced blockchain applications.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment 03

Aim: Create a Cryptocurrency using Python and perform mining in the Blockchain created.

Roll No.	04
Name	Mohit Patil
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1: Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2: Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO3: Implement smart contracts in Ethereum using different development frameworks. CO5: Interpret different Crypto assets and Crypto currencies

BLOCKCHAIN EXPERIMENT 3

Aim: Create a Cryptocurrency using Python and perform mining in the Blockchain created.

Guidelines

Lab Objectives: To implement public and private Blockchain.

Lab Outcomes (LO): Demonstrate the concept of Blockchain in real-world Applications (LO4)

Task to be performed :

1. Download the code from folder, Lab_3
2. Install requests in the virtual environment created in the Lab 2. (Follow the instructions)
3. Run the files - hadcoin_node_5001.py, hadcoin_node_5002.py, hadcoin_node_5003.py in 3 different terminals.
4. Open Postman, from each node - invoke connect_node() and pass the peers as POST requests.
5. Perform the following functions
 - Add Transactions - invoke add_transactions() as a POST request.
 - mining - mine_block(),
 - fetch the chain - get_chain(),
 - replace the longest chain - replace_chain()
6. Modify the code such that transactions are removed after they are added to the block. Tools & Libraries used :
 - Install Flask : pip install Flask
 - Download Postman from <https://www.postman.com/>
 - Python Libraries : datetime, jsonify, hashlib, uuid4, urlparse, request
 - Install requests : pip install

requests==2.18.4 Instructions : (Prepare for viva for the following topics)

1. Challenges in P2P networks
2. How transactions are performed on the network?
3. Explain the role of mem pools
4. Write briefly about the libraries and the tools used during implementation.

Outcome :

1. Understood the challenges in P2P networks, how transactions are performed and how a miner mines a block to be added in a blockchain.
2. Implemented a Cryptocurrency in Python using Flask, Postman and Python libraries such as datetime, jsonify, hashlib, uuid4, urlparse, request.
3. Successfully mined the blocks among a P2P network with 3 nodes.
4. Performed transactions via the network.
5. Successfully updated the block across the network
6. Prepare a document with Aim, Tasks performed, Program, Output and Conclusion.
7. Submit the hardcopy by the 2nd week of August 2023 (As per the instructions, submit a hard copy of the same).

Theory:

1. Blockchain Overview

Blockchain is a **distributed and decentralized ledger** that stores information in a series of linked blocks.

Each block contains:

- Transaction data
- Timestamp
- Previous block's hash
- Its own unique hash (digital fingerprint)

Once data is recorded in a blockchain, it becomes **immutable** because altering one block would require recalculating all subsequent blocks.

2. Mining

Mining is the process of:

1. Collecting pending transactions into a block.
2. Performing a computational puzzle (Proof-of-Work) to find a valid hash.
3. Adding the new block to the blockchain. Broadcasting it to all connected peers.

Miners are rewarded with cryptocurrency for successfully mining a block.

3. Multi-Node Blockchain Network

In this lab, we simulate **three independent blockchain nodes** (5001, 5002, 5003).

Each node:

- Runs on a separate port.
- Maintains its own copy of the blockchain.
- Can connect with peers to share and validate blocks.

4. Consensus Mechanism

We use the **Longest Chain Rule**:

- If multiple versions of the chain exist, the **longest valid chain** is chosen.
- This ensures all nodes agree on a single transaction history.

5. Transactions & Mining Reward

Each transaction has:

- Sender
- Receiver
- Amount

When mining a block:

- Pending transactions are added to the block.
- A **reward transaction** is added automatically to pay the miner.

6. Chain Replacement

When `/replace_chain` is called:

1. Node requests chains from peers.
2. If it finds a longer and valid chain, it replaces its own.
3. This keeps the blockchain consistent across all nodes.

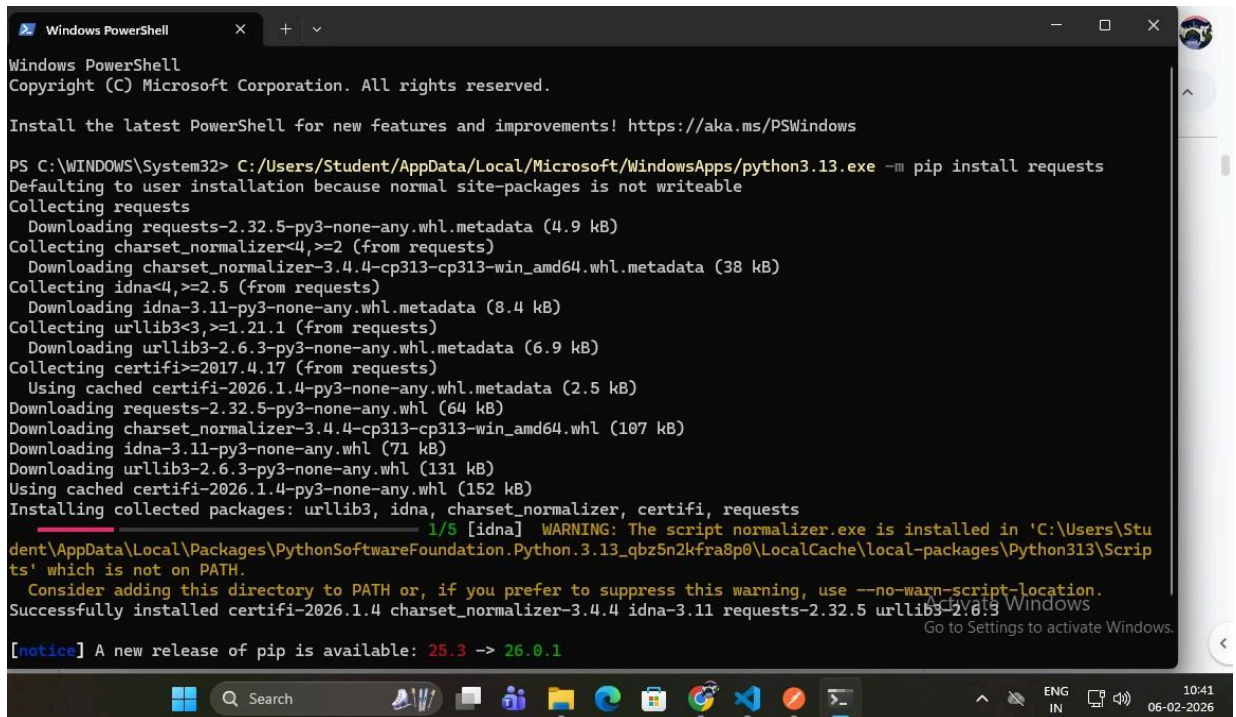
Tools & Libraries Used

- **Python 3.x**
- **Flask** – Web framework for API endpoints
`pip install Flask`
- **Requests** – For HTTP communication between nodes
`pip install requests==2.18.4`
- **Postman** – For testing API requests
- Python Standard Libraries:
 - `datetime`
 - `jsonify`

- hashlib
- uuid4
- urlparse
- request

Procedure and Output:

1. Download `hadcoin_node_5001.py`,
`hadcoin_node_5002.py`, `hadcoin_node_5003.py`.
2. Install required packages.



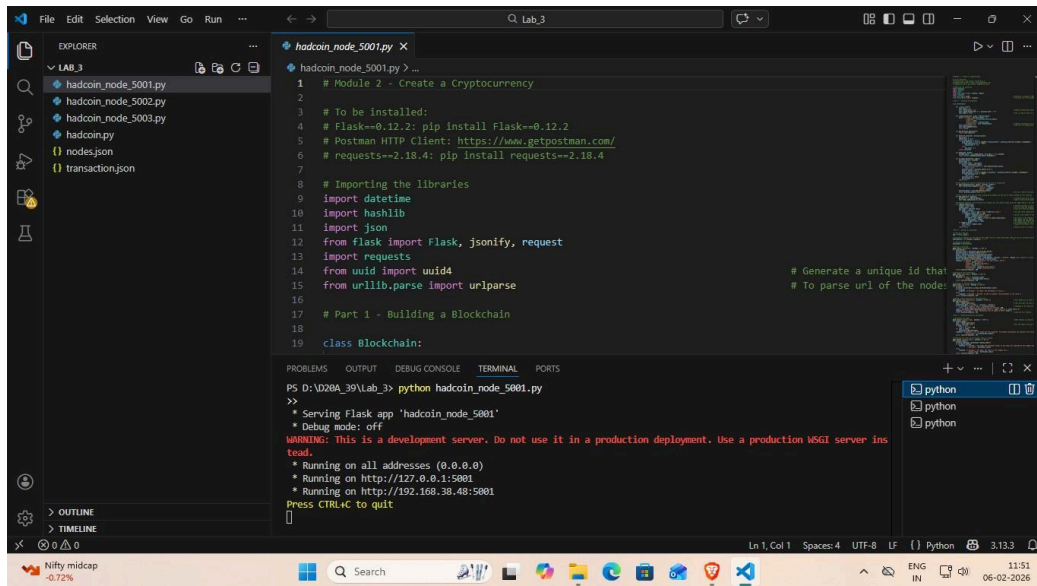
```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\WINDOWS\System32> C:/Users/Student/AppData/Local/Microsoft/WindowsApps/python3.13.exe -m pip install requests
Defaulting to user installation because normal site-packages is not writeable
Collecting requests
  Downloading requests-2.32.5-py3-none-any.whl.metadata (4.9 kB)
Collecting charset_normalizer<4,>=2 (from requests)
  Downloading charset_normalizer-3.4.4-cp313-cp313-win_amd64.whl.metadata (38 kB)
Collecting idna<4,>=2.5 (from requests)
  Downloading idna-3.11-py3-none-any.whl.metadata (8.4 kB)
Collecting urllib3<3,>=1.21.1 (from requests)
  Downloading urllib3-2.6.3-py3-none-any.whl.metadata (6.9 kB)
Collecting certifi>=2017.4.17 (from requests)
  Using cached certifi-2026.1.4-py3-none-any.whl.metadata (2.5 kB)
Downloading requests-2.32.5-py3-none-any.whl (64 kB)
Downloading charset_normalizer-3.4.4-cp313-cp313-win_amd64.whl (107 kB)
Downloading idna-3.11-py3-none-any.whl (71 kB)
Downloading urllib3-2.6.3-py3-none-any.whl (131 kB)
Using cached certifi-2026.1.4-py3-none-any.whl (152 kB)
Installing collected packages: urllib3, idna, charset_normalizer, certifi, requests
  1/5 [idna] WARNING: The script normalizer.exe is installed in 'C:\Users\Student\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.13_qbz5n2kfra8p0\LocalCache\local-packages\Python313\Scripts' which is not on PATH.
  Consider adding this directory to PATH or, if you prefer to suppress this warning, use --no-warn-script-location.
Successfully installed certifi-2026.1.4 charset_normalizer-3.4.4 idna-3.11 requests-2.32.5 urllib3-2.6.3

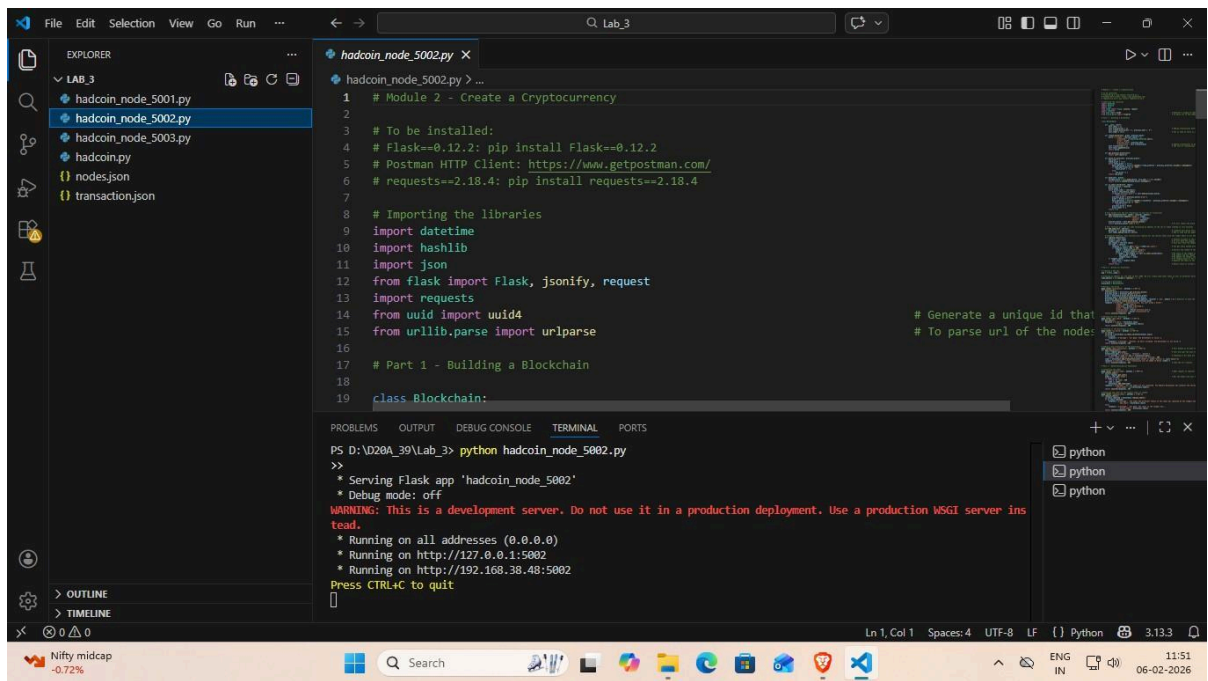
[notice] A new release of pip is available: 25.3 -> 26.0.1
```

3. Run each node in separate terminals.



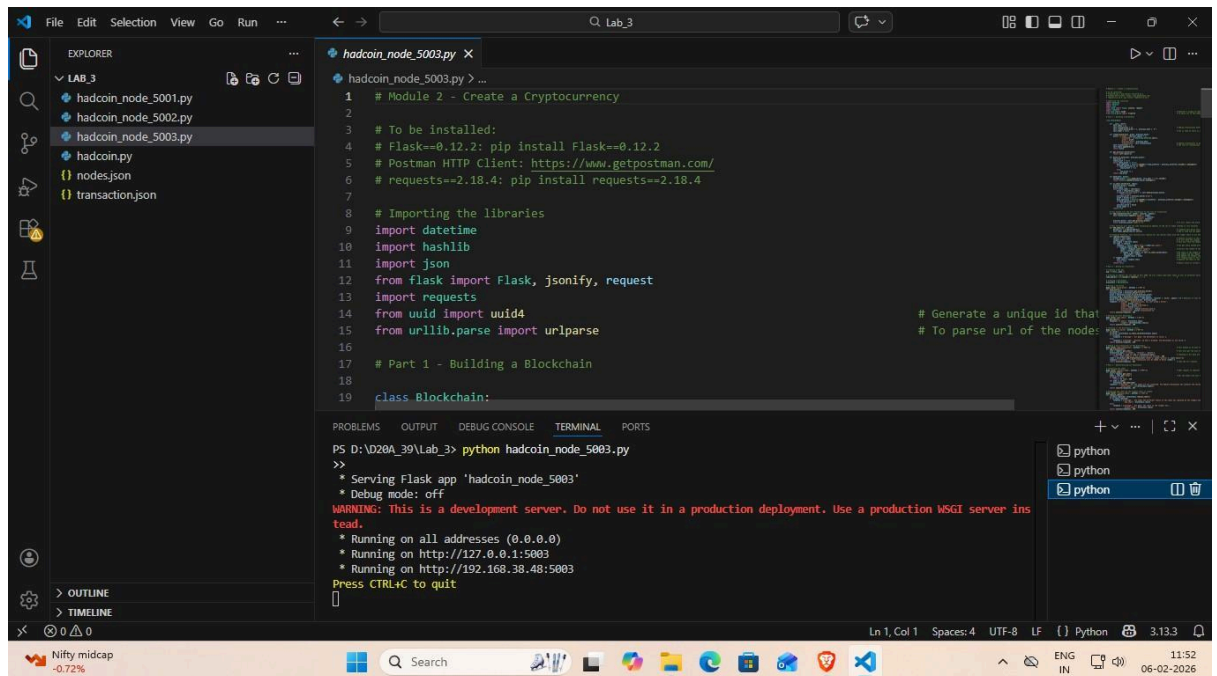
The screenshot shows the Visual Studio Code editor with the file explorer on the left displaying a project named 'LAB_3'. The file explorer contains the following files: `hadcoin_node_5001.py`, `hadcoin_node_5002.py`, `hadcoin_node_5003.py`, `hadcoin.py`, `nodes.json`, and `transaction.json`. The main editor window displays the code for `hadcoin_node_5001.py`. The code is a Python script for a Flask application that serves as a blockchain node. It includes comments for installation instructions, library imports, and a `Blockchain` class definition. The terminal at the bottom shows the command `python hadcoin_node_5001.py` being executed, resulting in the following output:

```
PS D:\D20A_39\Lab_3> python hadcoin_node_5001.py
>>
* Serving Flask app 'hadcoin_node_5001'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5001
* Running on http://192.168.38.48:5001
Press CTRL+C to quit
```



The screenshot shows the Visual Studio Code editor with the file explorer on the left displaying the same project 'LAB_3'. The main editor window displays the code for `hadcoin_node_5002.py`. The code is a Python script for a Flask application that serves as a blockchain node. It includes comments for installation instructions, library imports, and a `Blockchain` class definition. The terminal at the bottom shows the command `python hadcoin_node_5002.py` being executed, resulting in the following output:

```
PS D:\D20A_39\Lab_3> python hadcoin_node_5002.py
>>
* Serving Flask app 'hadcoin_node_5002'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5002
* Running on http://192.168.38.48:5002
Press CTRL+C to quit
```



4. Connect nodes using Postman – POST `/connect_node`.

```
{  
  
  "nodes":  
  
  [  
  
    "http://127.0.0.1:5002",  
  
    "http://127.0.0.1:5002"  
  
  ]  
  
}
```

5.

The screenshot shows the Postman application interface. The workspace is named "HIMESH PATHAI's Workspace". The active collection is "My Collection". The selected request is a POST request to "http://127.0.0.1:5001/connect_node". The request body is raw JSON, containing a "nodes" array with two elements: "http://127.0.0.1:5002" and "http://127.0.0.1:5003". The response is a 201 CREATED status, with a response time of 10 ms and a body size of 325 B. The response body is JSON, containing a "message" field with the text "All the nodes are now connected. The Hadooin Blockchain now contains the following nodes:" and a "total_nodes" array with two elements: "127.0.0.1:5003" and "127.0.0.1:5002".

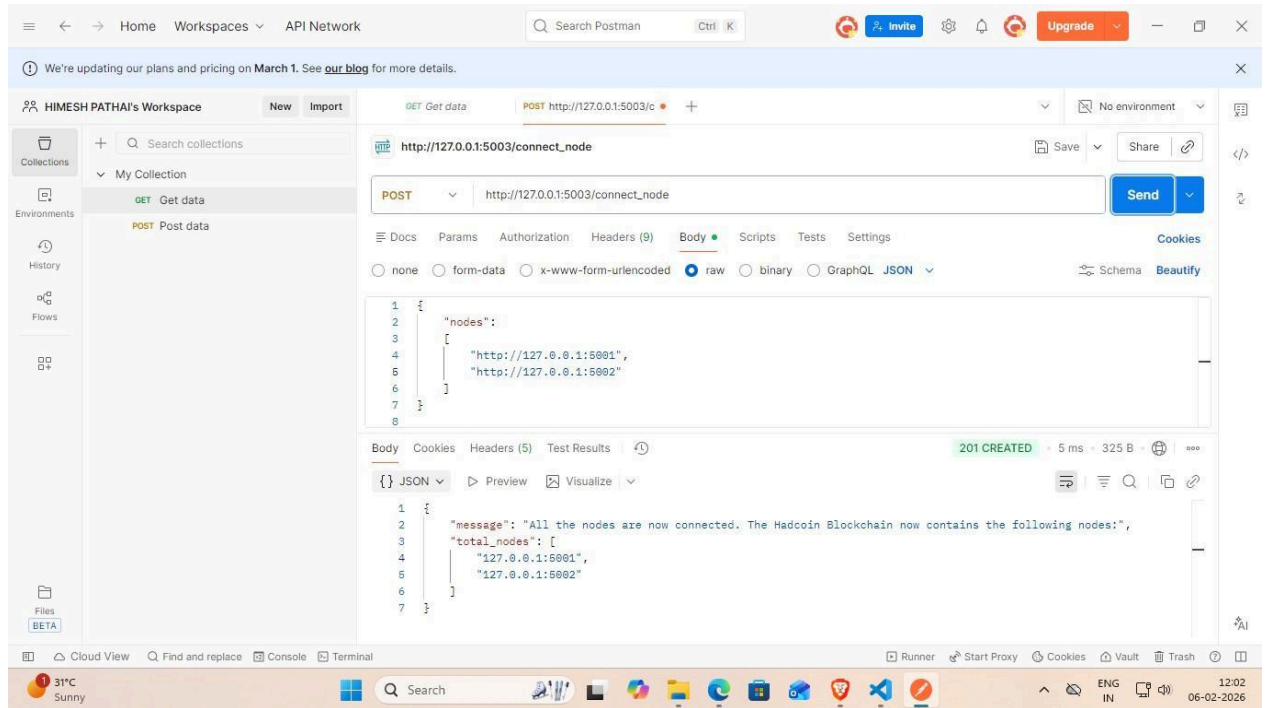
```
1 {
2   "nodes":
3   [
4     "http://127.0.0.1:5002",
5     "http://127.0.0.1:5003"
6   ]
7 }
```

```
1 {
2   "message": "All the nodes are now connected. The Hadooin Blockchain now contains the following nodes:",
3   "total_nodes": [
4     "127.0.0.1:5003",
5     "127.0.0.1:5002"
6   ]
7 }
```

The screenshot shows the Postman application interface. The workspace is named "HIMESH PATHAI's Workspace". The active collection is "My Collection". The selected request is a POST request to "http://127.0.0.1:5002/connect_node". The request body is raw JSON, containing a "nodes" array with two elements: "http://127.0.0.1:5001" and "http://127.0.0.1:5003". The response is a 201 CREATED status, with a response time of 17 ms and a body size of 325 B. The response body is JSON, containing a "message" field with the text "All the nodes are now connected. The Hadooin Blockchain now contains the following nodes:" and a "total_nodes" array with two elements: "127.0.0.1:5001" and "127.0.0.1:5003".

```
1 {
2   "nodes":
3   [
4     "http://127.0.0.1:5001",
5     "http://127.0.0.1:5003"
6   ]
7 }
```

```
1 {
2   "message": "All the nodes are now connected. The Hadooin Blockchain now contains the following nodes:",
3   "total_nodes": [
4     "127.0.0.1:5001",
5     "127.0.0.1:5003"
6   ]
7 }
```

6. Add transactions – POST
7. Add 3 transactions in 5001
8. Condition 2 — Each node must have **different chain lengths**

Example:

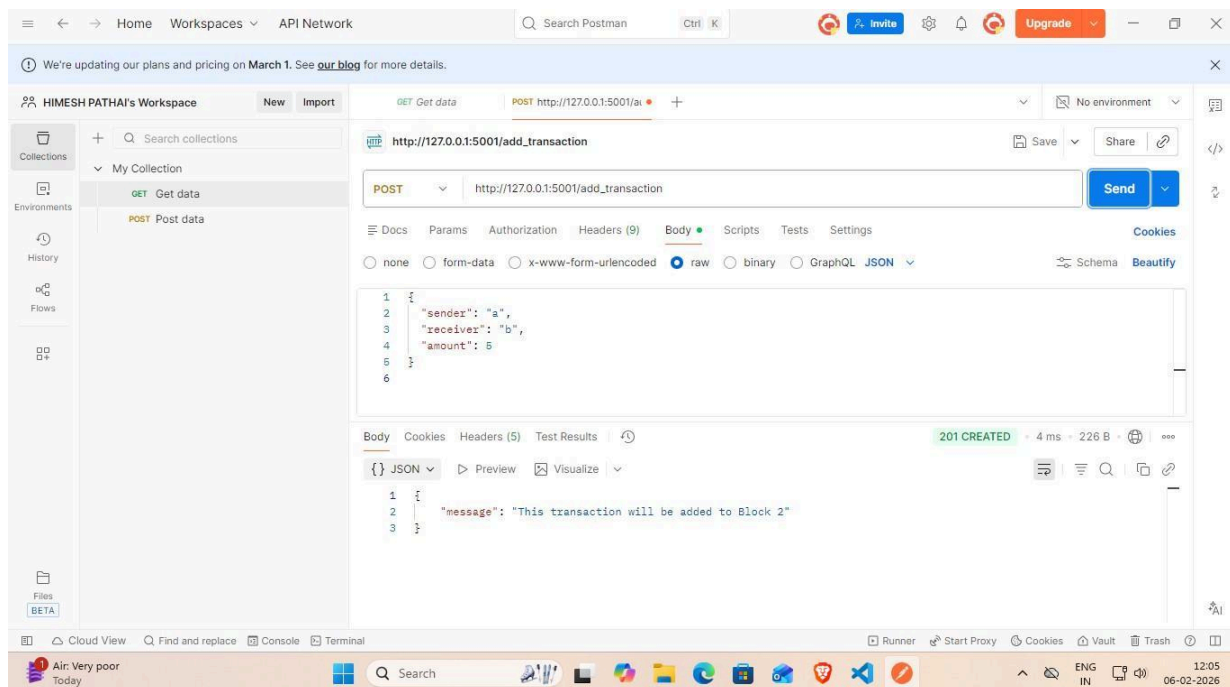
If you mined 1 block on 5001, but did not mine on 5002 or 5003 →

- 5001 chain length = 2
- 5002 chain length = 1
- 5003 chain length = 1

STEP 2 — Perform all transactions **ONLY** from one node (e.g., 5001)

POST → `http://127.0.0.1:5001/add_transaction`

```
{  
  
  "sender": "a",  
  
  "receiver": "b",  
  
  "amount": 5  
}
```



Then 5001 → GET /mine_block

The screenshot displays the Postman API client interface. The top navigation bar includes 'Home', 'Workspaces', and 'API Network'. A search bar and a 'Search Postman' button are present. A notification banner at the top states: 'We're updating our plans and pricing on March 1. See [our blog](#) for more details.'

The main workspace shows a collection named 'HIMESH PATHAI's Workspace' with two items: 'GET Get data' and 'POST Post data'. The 'GET Get data' item is selected, and its details are shown in the right pane. The request is a GET method to the URL 'http://127.0.0.1:5001/mine_block'. The 'Body' tab is active, showing a JSON payload:

```
1 {
2   "sender": "a",
3 }
```

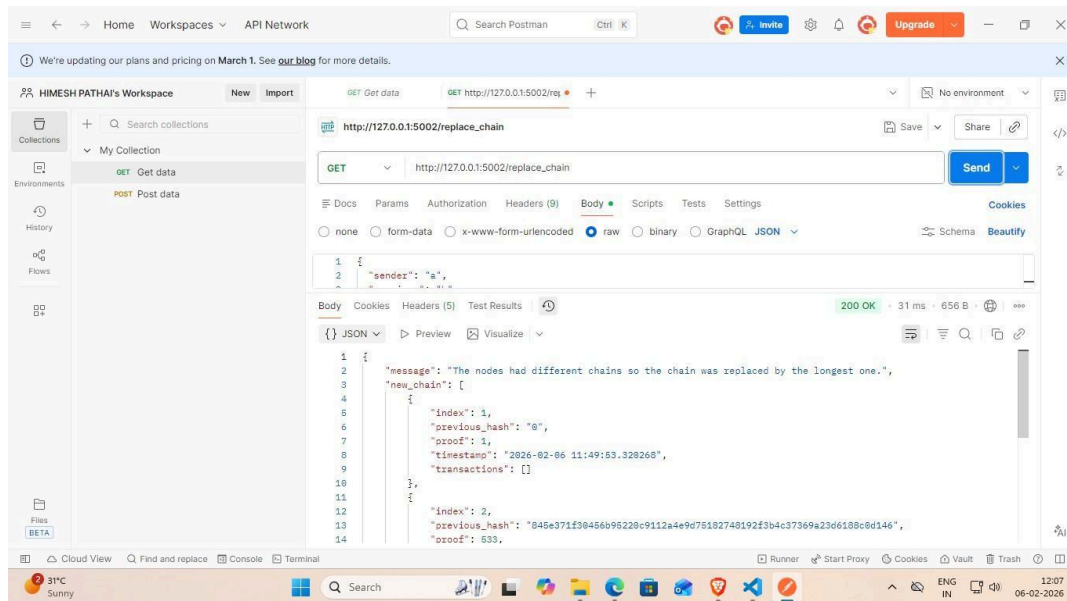
The response is a 200 OK status with a 15 ms response time and 503 B of data. The response body is displayed in the 'Body' tab, showing a JSON object:

```
1 {
2   "index": 2,
3   "message": "Congratulations, you just mined a block!",
4   "previous_hash": "845e371f38456b95228c9112a4e9d75182748192f3b4c37369a23d6188c0d146",
5   "proof": 533,
6   "timestamp": "2026-02-06 12:06:21.927314",
7   "transactions": [
8     {
9       "amount": 5,
10      "receiver": "b",
11      "sender": "a"
12    },
13    {
14      "amount": 1,
```

The bottom status bar shows the system clock as 12:06 on 06-02-2026, with a temperature of 31°C and a 'Sunny' weather icon.

STEP 3 — Now go to 5002 and run:

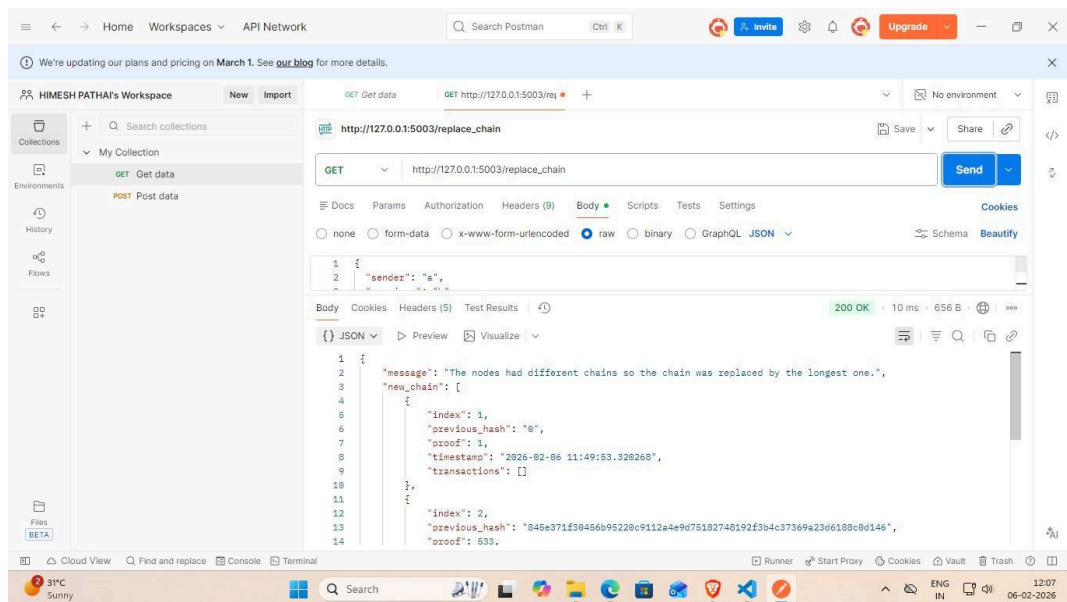
GET → `/replace_chain`



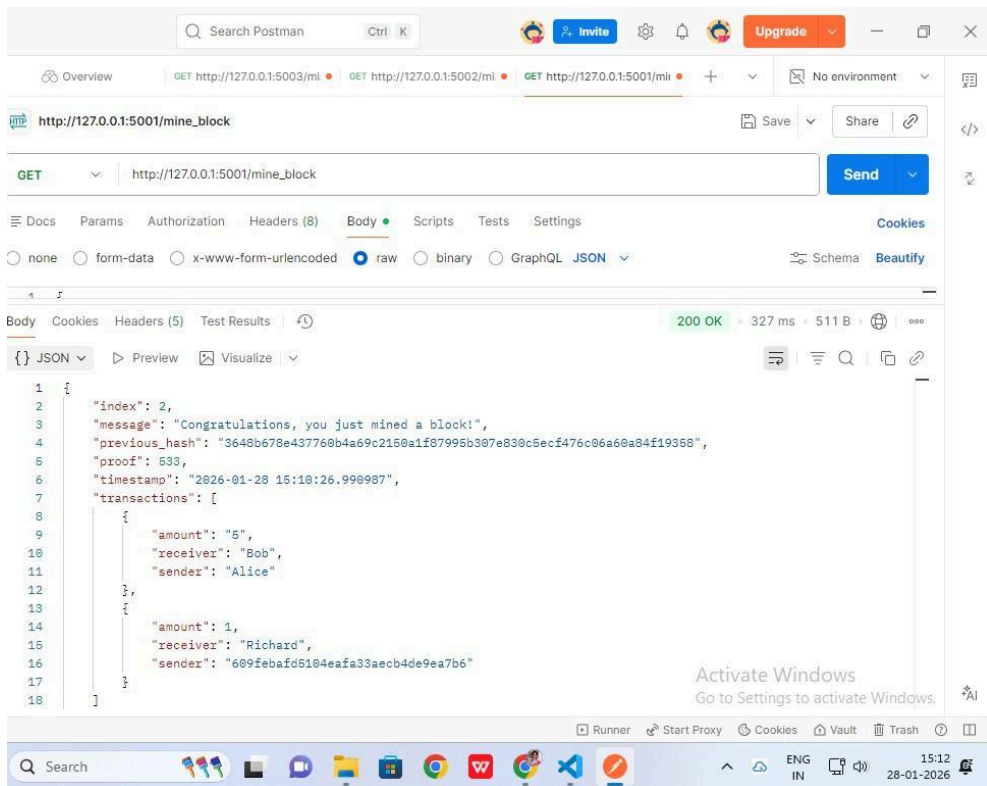
STEP 4 — Repeat on 5003

GET → `/replace_chain`

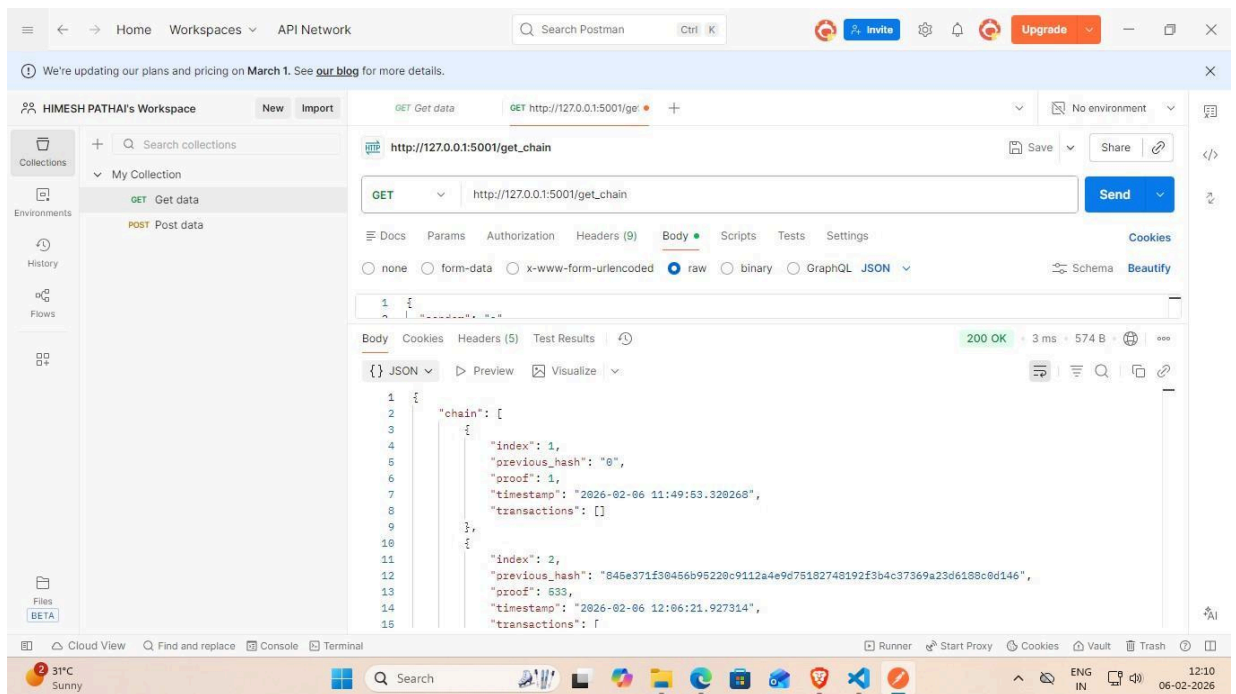
Same result.



9. Mine blocks – GET /mine_block.



10. Fetch blockchain – GET /get_chain.



Home Workspaces API Network Search Postman Ctrl K

We're updating our plans and pricing on March 1. See [our blog](#) for more details.

HIMESH PATHAI's Workspace New Import

GET Get data GET http://127.0.0.1:5002/ge

http://127.0.0.1:5002/get_chain

GET http://127.0.0.1:5002/get_chain

Body raw

```
1 {
2   "sender": "a",
3   "receiver": "b",
4   "amount": 5
}
```

Body Cookies Headers (5) Test Results

200 OK 5 ms 574 B

JSON Preview Visualize

```
1 {
2   "chain": [
3     {
4       "index": 1,
5       "previous_hash": "0",
6       "proof": 1,
7       "timestamp": "2026-02-06 11:49:53.320268",
8       "transactions": []
9     },
10    {
11      "index": 2,
12      "previous_hash": "845e371f30456b95220c9112a4e9d75182748192f3b4c37369a23d6188c8d146",

```

Cloud View Find and replace Console Terminal Runner Start Proxy Cookies Vault Trash

31°C Sunny Search 12:10 06-02-2026

Home Workspaces API Network Search Postman Ctrl K

We're updating our plans and pricing on March 1. See [our blog](#) for more details.

HIMESH PATHAI's Workspace New Import

GET Get data GET http://127.0.0.1:5003/ge

http://127.0.0.1:5003/get_chain

GET http://127.0.0.1:5003/get_chain

Body raw

```
1 {
2   "sender": "a",
3   "receiver": "b",
4   "amount": 5
}
```

Body Cookies Headers (5) Test Results

200 OK 3 ms 574 B

JSON Preview Visualize

```
1 {
2   "chain": [
3     {
4       "index": 1,
5       "previous_hash": "0",
6       "proof": 1,
7       "timestamp": "2026-02-06 11:49:53.320268",
8       "transactions": []
9     },
10    {
11      "index": 2,
12      "previous_hash": "845e371f30456b95220c9112a4e9d75182748192f3b4c37369a23d6188c8d146",

```

Cloud View Find and replace Console Terminal Runner Start Proxy Cookies Vault Trash

31°C Sunny Search 12:11 06-02-2026

Conclusion:

We developed a cryptocurrency using Python and implemented mining in a simulated three-node blockchain network. Each node maintained its own ledger and synchronized with peers using the Longest Chain Rule to ensure consistency. Mining was performed through Proof-of-Work, securely adding transactions and rewarding miners. This demonstrated key blockchain concepts, including decentralized consensus, transaction validation, and network synchronization.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment 04

Aim: Hands-on Solidity Programming Assignments for creating Smart Contracts

Roll No.	04
Name	Mohit Patil
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO3:Implement smart contracts in Ethereum using different development frameworks.

-Blockchain Lab Experiment 4

Mohit Patil
D20A 04

Aim - Hands on Solidity Programming Assignments for creating Smart Contracts.

Theory -

Primitive Data Types, Variables, and Functions (Pure & View)

In Solidity, primitive data types are the basic building blocks of smart contracts. Commonly used data types include:

- **uint** / **int** – Unsigned and signed integers of various sizes (e.g., `uint256`, `int128`).
- **bool** – Represents logical values (`true` or `false`).
- **address** – Stores a 20-byte Ethereum address, typically used for user accounts or contract addresses.
- **bytes** / **string** – Used to store binary data or textual information.

Types of Variables

Solidity supports different categories of variables:

- **State variables** – Stored permanently on the blockchain.
- **Local variables** – Temporary variables created during function execution.
- **Global variables** – Predefined variables such as `msg.sender`, `msg.value`, and `block.timestamp`.

Functions in Solidity

Functions define the logic of a smart contract. Two important function types are:

- **pure functions** – Cannot read or modify blockchain state. They rely only on input parameters and internal calculations.
- **view functions** – Can read state variables but cannot modify them.

Using `pure` and `view` appropriately helps reduce gas costs and ensures function integrity.

Inputs and Outputs of Functions

Solidity functions can accept input parameters and return one or more output values.

- **Inputs** allow users or other contracts to pass data into a function.
- **Outputs** return results after performing computations.

For example, a function may accept an Ether amount and return whether a transaction was successful. Solidity also supports **named return variables**, which enhance code readability and simplify debugging.

Visibility, Modifiers, and Constructors

Function Visibility

Visibility determines who can access a function:

- **public** – Accessible from inside and outside the contract.
- **private** – Accessible only within the same contract.
- **internal** – Accessible within the contract and its derived (child) contracts.
- **external** – Can be called only from outside the contract.

Modifiers

Modifiers are reusable code blocks that alter function behavior. They are commonly used for access control, such as restricting certain functions to the contract owner (e.g., **onlyOwner**).

Constructors

A constructor is a special function that runs only once during contract deployment. It is typically used to initialize important variables, such as assigning the deployer as the contract owner.

Control Flow: if-else and Loops

Solidity's control flow mechanisms are similar to traditional programming languages:

- **if-else statements** enable conditional execution, such as verifying sufficient balance before transferring funds.
- **Loops (for, while, do-while)** allow repeated execution of code, such as iterating through an array.

However, loops must be used cautiously because excessive iterations increase gas consumption, making transactions more expensive.

Data Structures: Arrays, Mappings, Structs, and Enums

Solidity provides several data structures:

- **Arrays** – Store ordered lists of elements. They can be fixed-size or dynamic. Example: an array of user addresses.
- **Mappings** – Store key-value pairs for efficient lookups. Example:
`mapping(address => uint)` for tracking account balances. Unlike arrays, mappings cannot be iterated directly.
- **Structs** – Custom data types that group related properties. Example:
`struct Player { string name; uint score; }`
- **Enums** – Define a set of predefined constants, improving readability. Example:
`enum Status { Pending, Active, Closed }`

Data Locations

Solidity uses three main data locations:

- **storage** – Permanently stored on the blockchain (used for state variables).
- **memory** – Temporary storage available only during function execution.
- **calldata** – Non-modifiable, non-persistent storage used for external function parameters. It is more gas-efficient than **memory**.

Understanding data locations is essential because they directly affect gas costs and contract performance.

Transactions: Ether, Wei, Gas, and Sending Transactions

Ether and Wei

Ether is the primary currency of Ethereum. All values are internally measured in **Wei**, the smallest unit:

1 Ether = 10¹⁸ Wei

Using Wei ensures high precision in financial calculations.

Gas and Gas Price

Every transaction consumes **gas**, which represents the computational effort required to execute it.

- **Gas price** determines how much Ether is paid per unit of gas.
- Higher gas prices encourage miners (validators) to prioritize a transaction.

Sending Transactions

Transactions are used to:

- Transfer Ether
- Interact with smart contracts

Common methods include:

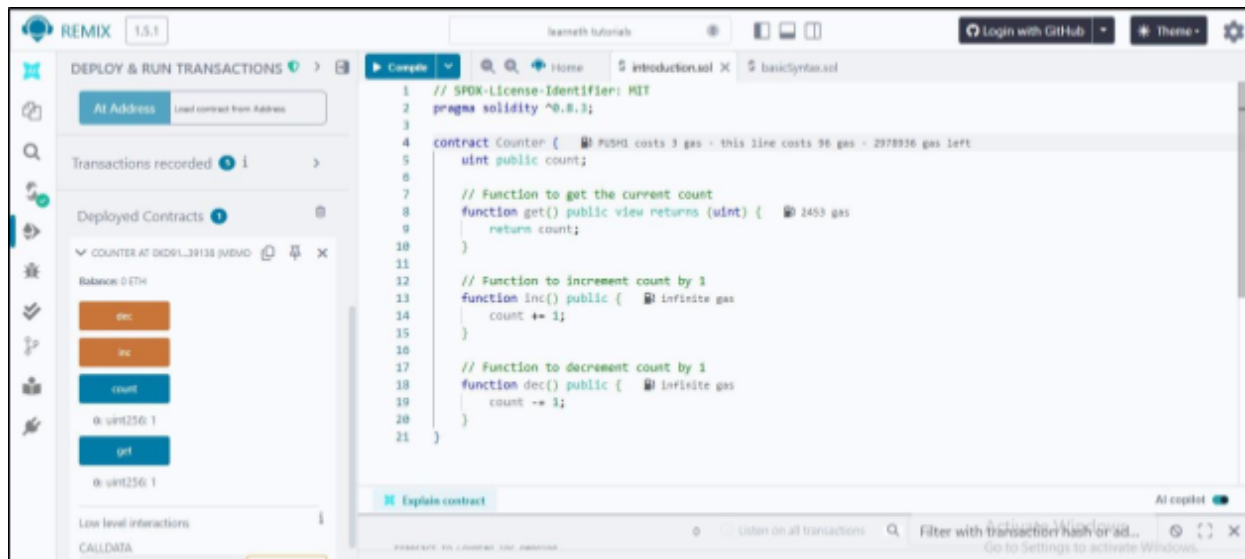
- `transfer()` – Sends Ether with fixed gas.
- `send()` – Similar to transfer but returns a boolean.
- `call()` – More flexible and commonly recommended for sending Ether.

Since every transaction requires gas, writing optimized and efficient smart contracts is crucial.

Code & Output -

Tutorial 1

1. Get counter value



```
from          0x5B38Da6a701c568545dCfcB03Fc8875f56beddC4 ⓘ

to            Counter.get() 0xd9145CCE52D386f254917e481eB44e9943F39138 ⓘ

execution cost 2453 gas (Cost only applies when called by a contract) ⓘ

input         0x6d4...ce63c ⓘ

output        0x0000000000000000000000000000000000000000000000000000000000000001 ⓘ

decoded input  {} ⓘ

decoded output {
               "0": "uint256: 1"
             } ⓘ
```

2. Increment counter value

```
✓ [vm] from: 0x5B3...eddC4 to: Counter.inc() 0xd91...39138 value: 0 wei data: 0x371...303c0 logs: 0 hash: 0xfce...8cdc3 Debug ^

status          1 Transaction mined and execution succeed

transaction hash 0xfcec1001233a0cc437ac7e87c2db11f10da2fe0976a8ef4e98ee5c65228cdc ⓘ

block hash      0xc8733a651aaad2a98067f80a95784141e7e550cd40388c50feefcd1145229228 ⓘ

block number    6 ⓘ

from            0x5B38Da6a701c568545dCfcB03Fc8875f56beddC4 ⓘ

to              Counter.inc() 0xd9145CCE52D386f254917e481eB44e9943F39138 ⓘ

transaction cost 26417 gas ⓘ
```

Activate Windows

3. Decrement counter value

```
✓ [vm] from: 0x5B3...eddC4 to: Counter.dec() 0xd91...39138 value: 0 wei data: 0xb3b...cfa82 logs: 0 hash: 0x21d...52d6d Debug ^

status          1 Transaction mined and execution succeed

transaction hash 0x21daa184e0a457ef2f35508a0094a8fc9c2eac05b53ab95e6d96e1c2c4452d6d ⓘ

block hash      0xca425c3b6aa253d68e49726515232861814af84154eb74afe6c4683415457db8 ⓘ

block number    7 ⓘ

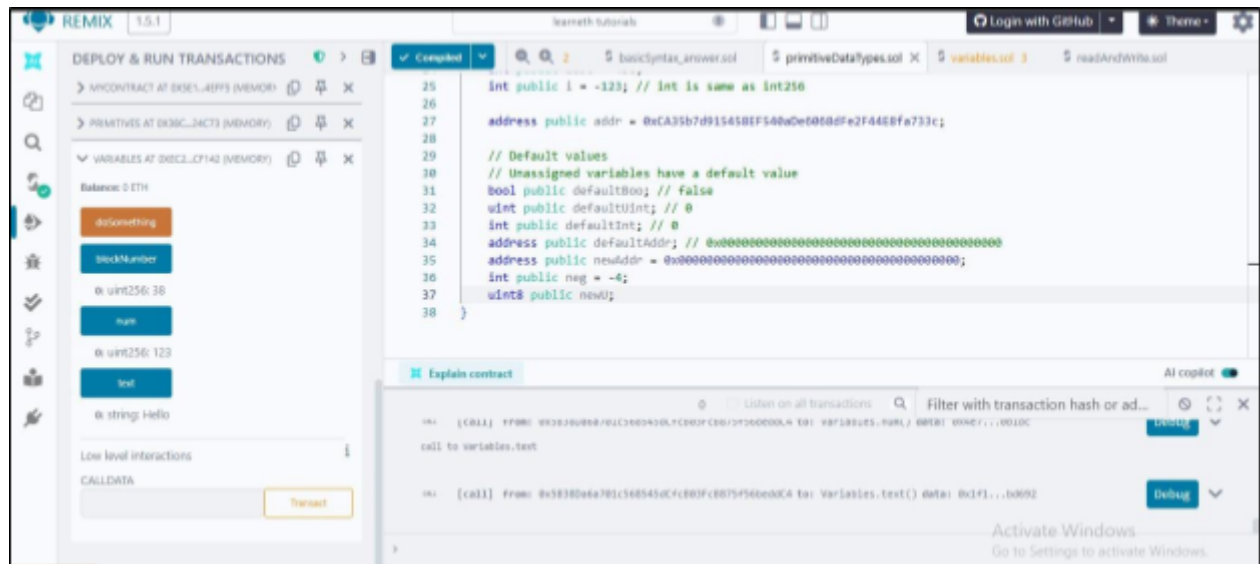
from            0x5B38Da6a701c568545dCfcB03Fc8875f56beddC4 ⓘ

to              Counter.dec() 0xd9145CCE52D386f254917e481eB44e9943F39138 ⓘ

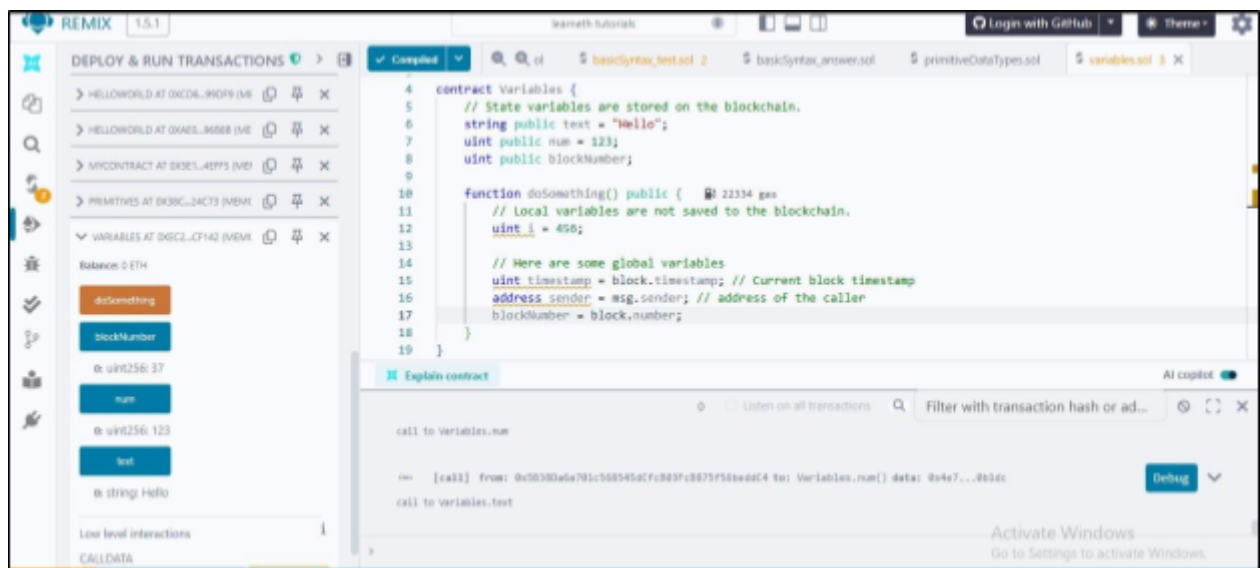
transaction cost 26461 gas ⓘ
```

Activate Windows

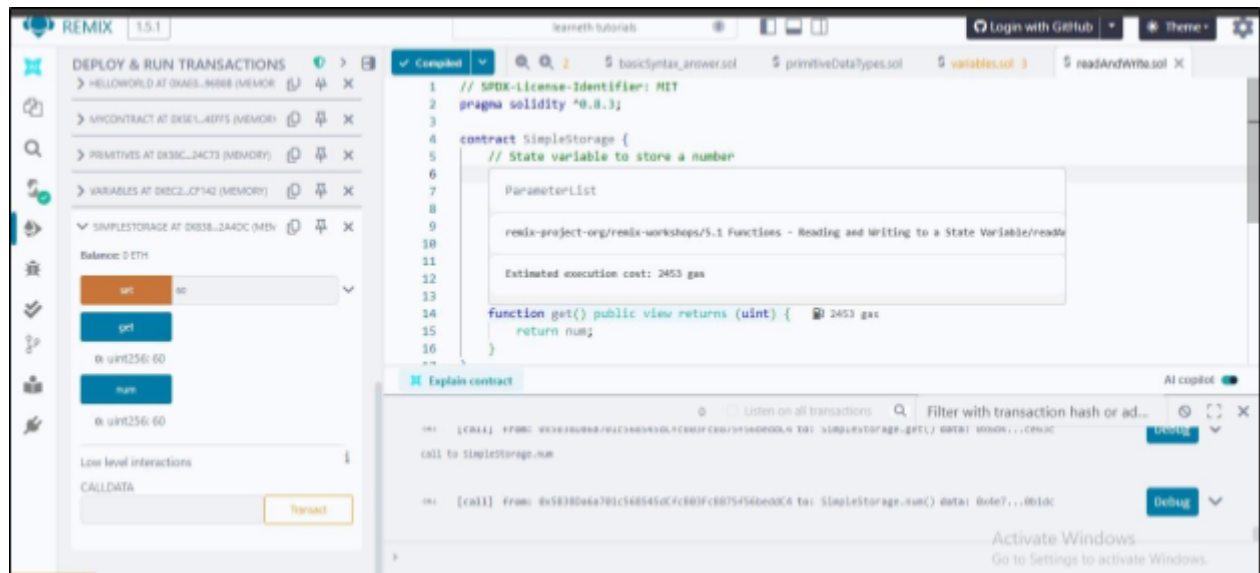
4. Get count value



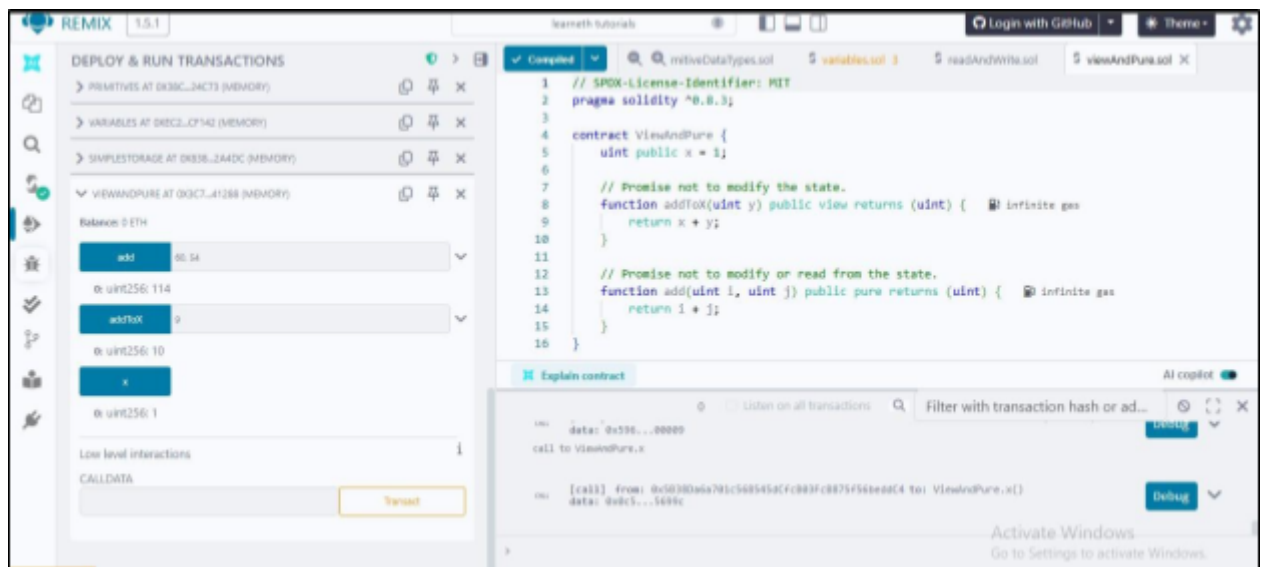
Tutorial 4



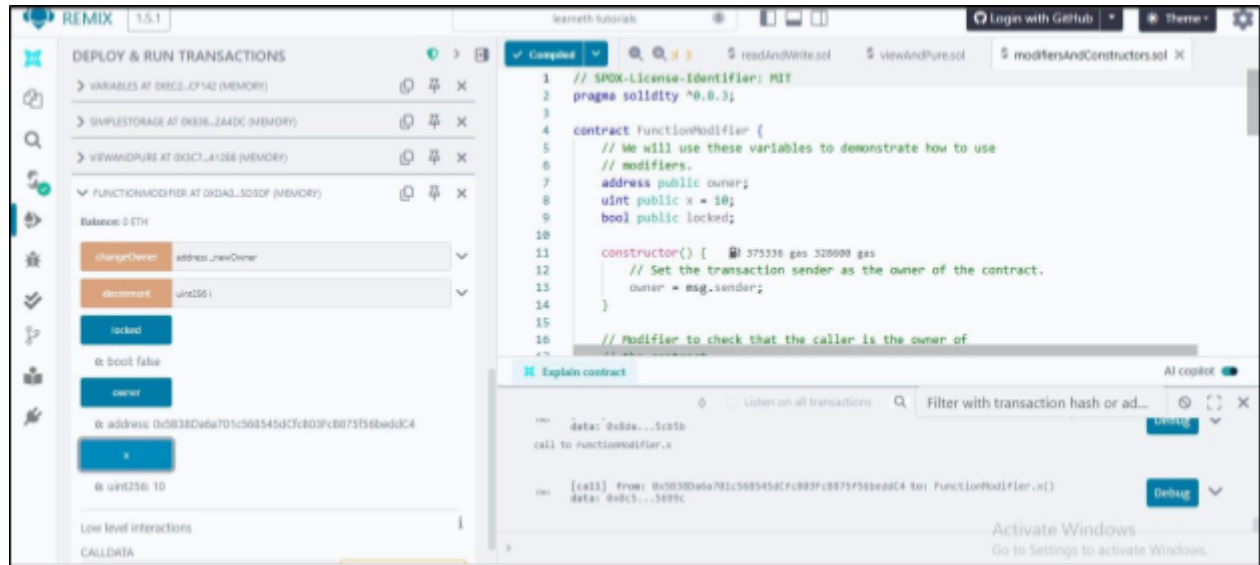
Tutorial 5



Tutorial 6



Tutorial 7



Tutorial 8

The screenshot shows the Remix IDE interface. On the left, the 'LEARNETH' panel displays the '6.4 Functions - Inputs and Outputs' section. It explains how to return multiple values from a function and how to name return values. The main editor shows the Solidity code for 'Function', which includes a function 'returnMany' that returns multiple values. The bottom panel shows the transaction details for the deployment, including the gas used and the transaction hash.

Tutorial 9

REMX1.5.1

LEARNETH

6. Visibility

6. Visibility

The `visibility` specifier is used to control who has access to functions and state variables.

There are four types of visibilities: `external`, `public`, `internal`, and `private`.

They regulate if functions and state variables can be called from inside the contract, from contracts that derive from the contract (child contracts), or from other contracts and transactions.

private

- Can be called from inside the contract

internal

- Can be called from inside the contract
- Can be called from a child contract

public

- Can be called from inside the contract
- Can be called from a child contract
- Can be called from other contracts or transactions

external

- Can be called from other contracts or transactions
- State variables can not be `external`

In this example, we have two contracts, the `Base` contract (line 4) and the `Base` contract (line 35) which inherits the functions and state variables from the `Base` contract.

When you uncomment the `testPrivateFunc` (lines 54-60) you get an error because the child contract doesn't have access to the private function `privateFunc` from the `Base` contract.

If you compile and deploy the two contracts, you will not be able to call the functions `privateFunc` and `testPrivateFunc` directly. You will only be able to call them via `testPrivateFunc` and `testInternalFunc`.

Contracts

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4
5
6 contract Base {
7     // Private function can only be called
8     // - inside this contract
9     // - inside contracts that inherit this contract.
10    function privateFunc() private pure returns (string memory) {
11        return "private function called";
12    }
13
14    function testPrivateFunc() public pure returns (string memory) {
15        return privateFunc();
16    }
17
18    // Internal function can be called
19    // - inside this contract
20    // - inside contracts that inherit this contract
21    function internalFunc() internal pure returns (string memory) {
22        return "internal function called";
23    }
24
25    function testInternalFunc() public pure virtual returns (string memory) {
26        return internalFunc();
27    }
28
29    // Public functions can be called
30    // - inside this contract
31    // - inside contracts that inherit this contract
32    // - by other contracts and accounts

```

Expand contract

0

Load on all transactions

Filter

view

from: 0x503...dd8c4 to: 0x503...dd8c4 value: 0 wei data: 0x00...f823 logs: 0 hash: 0x00...

Tutorial 10

REMX1.5.1

LEARNETH

7.1 Control Flow - If/Else

7.1 Control Flow - If/Else

Solidity supports different control flow statements that determine which parts of the contract will be executed. The conditional `if/else` statement enables contracts to make decisions depending on whether boolean conditions are either `true` or `false`.

Solidity differentiates between three different `if/else` statements: `if`, `if/else`, and `if/else if`.

if

The `if` statement is the most basic statement that allows the contract to perform an action based on a boolean expression.

In this contract, the `foo` function (line 5) the `if` statement (line 6) checks if `x` is smaller than `10`. If the statement is true, the function returns `0`.

else

The `else` statement enables our contract to perform an action if conditions are not met.

In this contract, the `foo` function uses the `else` statement (line 10) to return `1` if none of the other conditions are met.

else if

With the `else if` statement we can combine several conditions.

If the first condition (line 6) of the `foo` function is not met, but the condition of the `else if` statement (line 8) becomes true, the function returns `2`.

Watch a video tutorial on the `if/else` statements.

★ **Assignment**

Create a new function called `ternary` in the `Base` contract.

- That takes in a `uint` as an argument.
- The function returns `uint` if the argument is even, and `uint` if the argument is odd.

Contracts

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4
5
6 contract IFElse {
7     function foo(uint x) public pure returns (uint) {
8         if (x < 10) {
9             return 0;
10        } else if (x < 20) {
11            return 1;
12        } else {
13            return 2;
14        }
15    }
16
17    function ternary(uint x) public pure returns (uint) {
18        // if (x < 10) {
19        //     return 1;
20        // }
21        // return 2;
22
23        // shorthand way to write if / else statement
24        return x < 10 ? 1 : 2;
25    }
26 }

```

Expand contract

0

Load on all transactions

Filter

view

from: 0x503...dd8c4 to: 0x503...dd8c4 value: 0 wei data: 0x00...f823 logs: 0 hash: 0x00...

Tutorial 11

REMX 1.5.1

LEARNETH

7.2 Control Flow - Loops

11 / 15

7.2 Control Flow - Loops

Solidity supports iterative control flow statements that allow contracts to execute code repeatedly.

Solidity differentiates between three types of loops: `for`, `while`, and `do while` loops.

for

Generally, `for` loops (line 7) are great if you know how many times you want to execute a certain block of code. In solidity, you should specify this amount to avoid transactions running out of gas and failing if the amount of iterations is too high.

while

If you don't know how many times you want to execute the code but want to break the loop based on a condition, you can use a `while` loop (line 20). Loops are seldom used in Solidity since transactions might run out of gas and fail if there is no limit to the number of iterations that can occur.

do while

The `do while` loop is a special kind of while loop where you can ensure the code is executed at least once, before checking on the condition.

continue

The `continue` statement is used to skip the remaining code block and start the next iteration of the loop. In this contract, the `continue` statement (line 10) will prevent the second `if` statement (line 12) from being executed.

break

The `break` statement is used to exit a loop. In this contract, the `break` statement (line 14) will cause the `for` loop to be terminated after the sixth iteration.

[Watch a video tutorial on Loop statements](#)

★ **Assignment**

1. Create a public `uint` state variable called `count` in the `Loop` contract.

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4
5
6 contract Loop {
7     function loop() public {
8         // for loop
9         for (uint i = 0; i < 10; i++) {
10             if (i == 3) {
11                 // Skip to next iteration with continue
12                 continue;
13             }
14             if (i == 5) {
15                 // Exit loop with break
16                 break;
17             }
18         }
19
20         // while loop
21         uint j;
22         while (j < 10) {
23             j++;
24         }
25     }
26 }
27

```

Explain contract

Transaction hash: 0x...

Contract address: 0x...

Filter with transaction hash or

View from: 0x500... 0x004 to: Array (constructor) value: 0 wei data: 0x000...7f003 logs: 0 hash:

Tutorial 12

REMX 1.5.1

LEARNETH

8.1 Data Structures - Arrays

11 / 18

8.1 Data Structures - Arrays

empty (line 6)

Initializing arrays

We can initialize the elements of an array all at once (line 7), or initiate new elements one by one (`arr[i] = 1`). If we declare an array, we automatically initialize its elements with the default value 0 (line 9).

Accessing array elements

We access elements inside an array by providing the name of the array and the index in brackets (line 12).

Adding array elements

Using the `push()` member function, we add an element to the end of a dynamic array (line 28).

Removing array elements

Using the `pop()` member function, we delete the last element of a dynamic array (line 31).

We can use the `delete` operator to remove an element with a specific index from an array (line 42). When we remove an element with the `delete` operator all other elements stay the same, which means that the length of the array will stay the same. This will create a gap in our array. If the order of the array is not important, then we can remove the last element of the array to the place of the deleted element (line 40), or use a mapping. A mapping might be a better choice if we plan to remove elements in our data structure.

Array length

Using the `length` member, we can read the number of elements that are stored in an array (line 30).

[Watch a video tutorial on Arrays](#)

★ **Assignment**

1. Initialize a public fixed-sized array called `arr` with the values 0, 1, 2. Make the size as small as possible.
2. Change the `push()` function to return the value of `arr`.

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3 // Praveen Titankar DORA/08
4 contract Array {
5     // Several ways to initialize an array
6     uint[] public arr;
7     uint[] public arr2 = [1, 2, 3];
8     // Fixed sized array, all elements initialize to 0
9     uint[10] public myFixedSizedArr;
10
11     function get(uint i) public view returns (uint) {
12         return arr[i];
13     }
14
15     // Solidity can return the entire array.
16     // But this function should be avoided for
17     // arrays that can grow indefinitely in length.
18     function getArr() public view returns (uint[] memory) {
19         return arr;
20     }
21
22     function push(uint i) public {
23         // Append to array
24         // This will increase the array length by 1.
25         arr.push(i);
26     }
27
28     function pop() public {
29         // Remove last element from array
30         // This will decrease the array length by 1
31         arr.pop();
32     }
33 }
34

```

Explain contract

Transaction hash: 0x...

Contract address: 0x...

Filter with transaction hash or

View from: 0x500... 0x004 to: Array (constructor) value: 0 wei data: 0x000...7f003 logs: 0 hash: 0x000...7f753

Tutorial 13

8.2 Data Structures - Mappings

In Solidity, mappings are a collection of key types and corresponding value type pairs.

The biggest difference between a mapping and an array is that you can't iterate over mappings. If we don't know a key we won't be able to access its value. If we need to know all of our data or iterate over it, we should use an array.

If we want to retrieve a value based on a known key we can use a mapping (e.g. addresses are often used as keys). Looking up values with a mapping is easier and cheaper than iterating over arrays. If arrays become too large, the gas cost of iterating over it could become too high and cause the transaction to fail.

We could also store the keys of a mapping in an array that we can iterate over.

Creating mappings

Mappings are declared with the syntax `mapping(keyType => valueType) variableName`. The key type can be any built-in value type or any contract, but not a reference type. The value type can be of any type.

In this contract, we are creating the public mapping `myMap` (line 6) that associates the key type `uint256` with the value type `uint`.

Accessing values

The syntax for interacting with key-value pairs of mappings is similar to that of arrays. To find the value associated with a specific key, we provide the name of the mapping and the key in brackets (line 11).

In contrast to arrays, we won't get an error if we try to access the value of a key whose value has not been set yet. When we create a mapping, every possible key is mapped to the default value 0.

Setting values

We set a new value for a key by providing the mapping's name and key in brackets and assigning it a new value (line 16).

Removing values

We can use the delete operator to delete a value associated with a key, which will set it to the default value of 0. As

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4
5
6 contract Mapping {
7     // Mapping from address to uint
8     mapping(address => uint) public myMap;
9
10    function get(address _addr) public view returns (uint) {
11        // Mapping always returns a value.
12        // If the value was never set, it will return the default value.
13        return myMap[_addr];
14    }
15
16    function set(address _addr, uint _i) public {
17        // Update the value at this address
18        myMap[_addr] = _i;
19    }
20
21    function remove(address _addr) public {
22        // Reset the value to the default value.
23        delete myMap[_addr];
24    }
25 }
26
27 contract NestedMapping {
28     // Nested mapping (mapping from address to another mapping)
29     mapping(address => mapping(uint => bool)) public nested;
30
31    function get(address _addr, uint _i) public view returns (bool) {
32        // You can get values from a nested mapping

```

Explain contract

Summary of the contract: The contract defines a mapping from address to uint. It includes functions for getting, setting, and removing values. It also includes a nested mapping example.

Tutorial 14

8.3 Data Structures - Structs

In Solidity, we can define custom data types in the form of structs. Structs are a collection of variables that can consist of different data types.

Defining structs

We define a struct using the `struct` keyword and a name (line 5). Inside curly braces, we can define our struct's members, which consist of the variable names and their data types.

Initializing structs

There are different ways to initialize a struct.

Positional parameters: We can provide the name of the struct and the values of its members as parameters in parentheses (line 16).

Key-value mapping: We provide the name of the struct and the keys and values as a mapping inside curly braces (line 19).

Initialize and update a struct: We initialize an empty struct first and then update its member by assigning it a new value (line 23).

Accessing structs

To access a member of a struct we can use the dot operator (line 33).

Updating structs

To update a struct's member we also use the dot operator and assign it a new value (lines 39 and 45).

Assignment

Create a function `update` that takes a `struct` as a parameter and updates a struct member with the given index in the `struct` mapping.

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4
5
6 contract Todos {
7     struct Todo {
8         string text;
9         bool completed;
10    }
11
12    // An array of 'Todo' structs
13    Todo[] public todos;
14
15    function create(string memory _text) public {
16        // 3 ways to initialize a struct
17        // - calling it like a function
18        todos.push(Todo(_text, false));
19
20        // key value mapping
21        todos.push(Todo({text: _text, completed: false}));
22
23        // Initialize an empty struct and then update it
24        Todo memory todo;
25        todo.text = _text;
26        // todo.completed initialized to false
27        todos.push(todo);
28    }
29
30    // Solidity automatically created a getter for 'todos' so
31    // you don't actually need this function.

```

Explain contract

Summary of the contract: The contract defines a struct 'Todo' with 'text' and 'completed' members. It includes functions for creating, initializing, and accessing structs. It also includes a nested mapping example.

Tutorial 15

REMIT 1.5.1

Learneth

Tutorials list

8.4 Data Structures - Enums

15 / 19

8.4 Data Structures - Enums

In Solidity enums are custom data types consisting of a limited set of constant values. We use enums when our variables should only get assigned a value from a predefined set of values.

In this contract, the state variable `status` can get assigned a value from the limited set of provided values of the enum `Status` representing the various states of a shipping status.

Defining enums

We define an enum with the `enum` keyword, followed by the name of the custom type we want to create (line 6). Inside the curly braces, we define all available members of the enum.

Initializing an enum variable

We can initialize a new variable of an enum type by providing the name of the enum, the visibility, and the name of the variable (line 16). Upon its initialization, the variable will be assigned the value of the first member of the enum, in this case, `Pending` (line 7).

Even though enum members are named when you define them, they are stored as unsigned integers, not strings. They are numbered in the order that they were defined, the first member starting at 0. The initial value of `status`, in this case, is 0.

Accessing an enum value

To access the enum value of a variable, we simply need to provide the name of the variable that is storing the value (line 25).

Updating an enum value

We can update the enum value of a variable by assigning it the `enum` representing the enum member (line 30). `Shipped` would be 1 in this example. Another way to update the value is using the dot operator by providing the name of the enum and its member (line 30).

Removing an enum value

We can use the delete operator to delete the enum value of the variable, which means as for arrays and mappings, `delete` the default value is 0.

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4
5
6 contract Enum {
7     // Enum representing shipping status
8     enum Status {
9         Pending,
10        Shipped,
11        Accepted,
12        Rejected,
13        Canceled
14    }
15
16    // Default value is the first element listed in
17    // definition of the type, in this case "Pending"
18    Status public status;
19
20    // Returns uint
21    // Pending - 0
22    // Shipped - 1
23    // Accepted - 2
24    // Rejected - 3
25    // Canceled - 4
26    function get() public view returns (Status) {
27        return status;
28    }
29
30    // Update status by passing uint into input
31    function set(Status _status) public {
32        status = _status;
33    }
34
35    // Explain contract
36
37    [vm] from: 0x503...ad04 to: Array (constructor) value: 0 wei data: 0x00...f003

```

Tutorial 16

REMIT 1.5.1

Learneth

Tutorials list

9. Data Locations

11 / 19

9. Data Locations

The values of variables in Solidity can be stored in different data locations: memory, storage, and calldata.

As we have discussed before, variables of the value type store an independent copy of a value, while variables of the reference type (array, struct, mapping) only store the location (reference) of the value.

If we use a reference type in a function, we have to specify in which data location their values are stored. The price for the execution of the function is influenced by the data location; creating copies from reference types costs gas.

Storage

Values stored in storage are stored permanently on the blockchain and, therefore, are expensive to use.

In this contract, the state variables `arr`, `map`, and `myStructs` (lines 5, 6, and 10) are stored in storage. State variables are always stored in storage.

Memory

Values stored in memory are only stored temporarily and are not on the blockchain. They only exist during the execution of an external function and are discarded afterward. They are cheaper to use than values stored in storage.

In this contract, the local variable `myStruct` (line 19), as well as the parameter `arr` (line 31), are stored in memory. Function parameters need to have the data location memory or calldata.

Calldata

Calldata stores function arguments. Like memory, calldata is only stored temporarily during the execution of an external function. In contrast to values stored in memory, values stored in calldata can not be changed. Calldata is the cheapest data location to use.

In this contract, the parameter `arr` (line 31) has the data location calldata. If we wanted to assign a new value to the first element of the array `arr`, we could do that in the `function` (line 31) but not in the `function` (line 30). This is because `arr` in `function` has the data location memory and function `h` has the data location calldata.

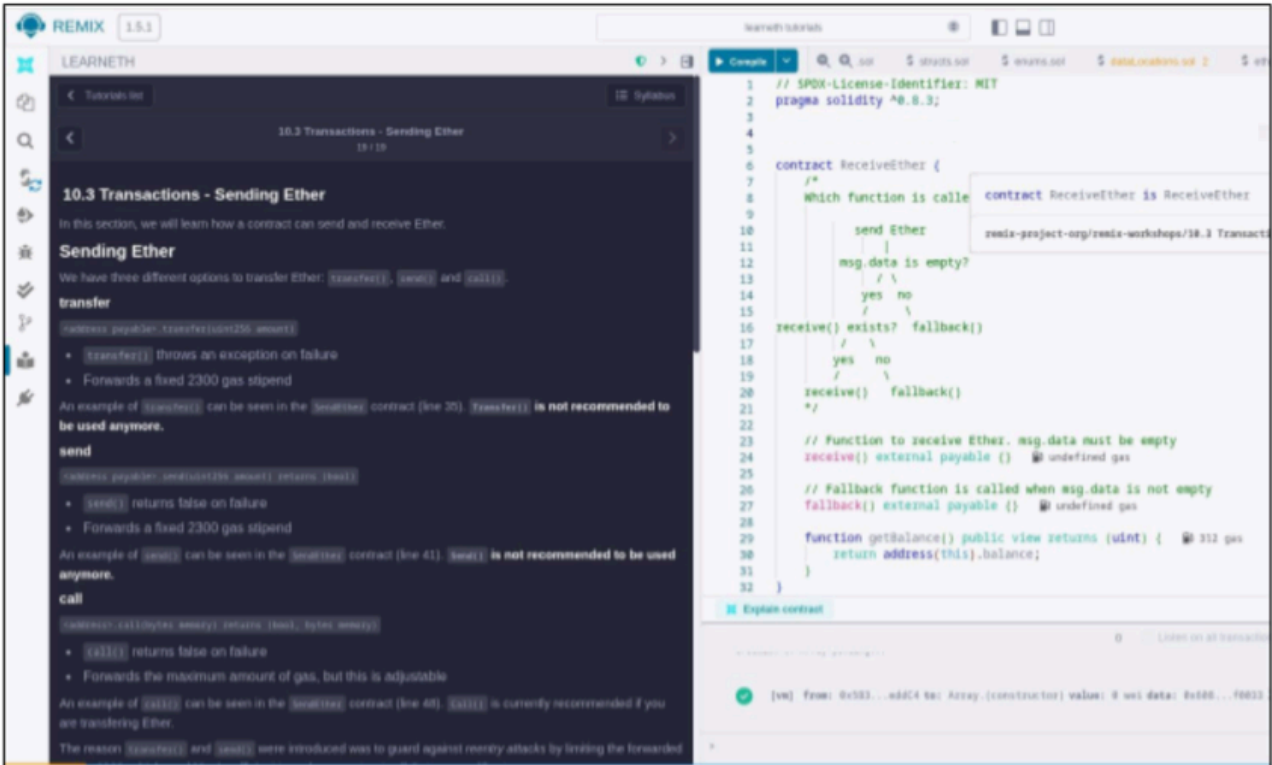
Assignments

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4
5
6 contract DataLocations {
7     uint[] public arr;
8     mapping(uint => address) map;
9     struct MyStruct {
10         uint foo;
11     }
12     mapping(uint => MyStruct) myStructs;
13
14     function f() public {
15         // call _f with state variables
16         _f(arr, map, myStructs[1]);
17     }
18
19     // get a struct from a mapping
20     MyStruct storage myStruct = myStructs[1];
21     // create a struct in memory
22     MyStruct memory myNewStruct = MyStruct(0);
23
24     function _f(
25         uint[] storage _arr,
26         mapping(uint => address) storage _map,
27         MyStruct storage _myStruct
28     ) internal {
29         // do something with storage variables
30     }
31
32     // You can return memory variables
33
34     [vm] from: 0x503...ad04 to: Array (constructor) value: 0 wei data: 0x00...f003

```

Tutorial 17



Conclusion:

This experiment provided an in-depth exploration of Solidity programming through structured practical implementation using the Remix IDE. Fundamental concepts—including data types, variable classifications, function definitions, visibility specifiers, modifiers, constructors, control flow mechanisms, data structures, and transaction management—were systematically implemented and evaluated.

The process of designing, compiling, and deploying smart contracts on the Remix Virtual Machine (VM) enabled a comprehensive understanding of smart contract architecture and blockchain execution mechanisms. The practical exposure strengthened conceptual clarity and technical proficiency in Ethereum-based development.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment 05

Aim: Deploying a Voting/Ballot Smart Contract

Roll No.	04
Name	Mohit Patil
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1:Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO3:Implement smart contracts in Ethereum using different development frameworks.

Blockchain Lab Experiment 5

Mohit Patil
D20A 04

Aim: Deploying a Voting/Ballot Smart Contract

Theory

1. Relevance of `require` Statements in Solidity Programs

In Solidity, the `require` statement is used as a validation mechanism to enforce conditions before executing a function. It acts as a guard clause that ensures only valid inputs, authorized users, or correct states are allowed to proceed further in the program.

If the condition specified inside the `require` statement evaluates to **false**, the following actions occur:

- The function execution immediately stops.
- All state changes made during that transaction are reverted.
- The remaining gas is refunded (except the gas already consumed).
- An optional error message is returned.

This rollback mechanism is crucial in blockchain applications because it prevents invalid or malicious transactions from altering the contract's state permanently. Since blockchain data is immutable, such protective mechanisms are essential for maintaining integrity and security.

Example: Voting Smart Contract

In a Voting (Ballot) Smart Contract, `require` can be used in the following

scenarios: To check whether a voter has the right to vote:

```
require(voters[msg.sender].weight > 0, "Has no right to vote");
```

- To ensure that a voter has not already voted.
- To verify that only the chairperson can grant voting rights.

Therefore, `require` statements enhance:

- **Security** – Prevent unauthorized access.
- **Correctness** – Ensure logical conditions are satisfied.
- **Reliability** – Maintain consistency of blockchain data.
- **User Experience** – Provide meaningful error messages for debugging.

2. Important Keywords: **mapping, storage, and memory**

(a) **mapping**

A `mapping` in Solidity is a key-value data structure similar to a hash table or dictionary. Its syntax is:

```
mapping(keyType => valueType)
```

Example:

```
mapping(address => Voter) public voters;
```

In this case, each Ethereum address is associated with a `Voter` structure.

Characteristics of Mapping:

- Provides fast lookup of values using keys.
- Does not store keys explicitly.
- Does not have a length property.
- Cannot be iterated over directly.
- More gas-efficient for data retrieval compared to arrays.

Mappings are widely used in smart contracts such as voting systems, token balances, and ownership tracking.

(b) storage

`storage` refers to the permanent data area of a smart contract stored on the Ethereum blockchain. Variables declared at the contract level are stored in storage by default.

Key Features:

- Data is persistent across transactions.
- Changes remain recorded on the blockchain.
- Writing to storage consumes higher gas fees.
- Suitable for maintaining contract state.

For example, voter details stored inside a mapping remain available throughout the contract's lifecycle unless explicitly modified.

(c) memory

`memory` is a temporary data location used during function execution. Variables declared in memory exist only for the duration of the function call.

Key Features:

- Data is discarded after function execution.
- Less expensive than storage.
- Used for temporary variables, parameters, and intermediate computations.

Example use cases include:

- Temporary string manipulation.
- Handling function arguments.
- Processing proposal names before storing them permanently.

A smart contract developer must carefully decide whether to use `storage` or `memory` to optimize gas costs and ensure efficiency.

3. Why Use `bytes32` Instead of `String`?

In earlier implementations of Ballot smart contracts, `bytes32` was commonly used for storing proposal names instead of `string`. The primary reason for this choice was gas efficiency and performance optimization.

`bytes32`

- Fixed-size data type (exactly 32 bytes).
- Requires less storage management.
- Faster comparison operations.
- Lower gas consumption.
- Limited to 32 characters.

Due to its fixed size, `bytes32` is efficient but not flexible for longer or user-friendly names.

`string`

- Dynamically sized data type.
- Can store variable-length text.
- More user-friendly and readable.
- Requires complex memory handling in the Ethereum Virtual Machine (EVM).
- Higher gas consumption.

While `string` improves usability and flexibility, it increases computational and storage costs.

CODE:-

```
// SPDX-License-Identifier: GPL-3.0

pragma solidity >=0.7.0 <0.9.0;

contract Ballot {

    struct Voter {
        uint weight;
        bool voted;
        address delegate;
        uint vote;
    }

    //  Changed bytes32 → string
    struct Proposal {
        string name;
        uint voteCount;
    }

    address public chairperson;

    mapping(address => Voter) public voters;

    Proposal[] public proposals;

    //  Changed constructor parameter
    constructor(string[] memory proposalNames) {
        chairperson = msg.sender;
        voters[chairperson].weight = 1;

        for (uint i = 0; i < proposalNames.length; i++) {
            proposals.push(Proposal({
                name: proposalNames[i],
                voteCount: 0
            }));
        }
    }
}
```

```

function giveRightToVote(address voter) external {
    require(msg.sender == chairperson, "Only chairperson can give right to vote.");
    require(!voters[voter].voted, "The voter already voted.");
    require(voters[voter].weight == 0, "Voter already has the right to vote.");

    voters[voter].weight = 1;
}

```

```

function delegate(address to) external {
    Voter storage sender = voters[msg.sender];

    require(sender.weight != 0, "You have no right to vote");
    require(!sender.voted, "You already voted.");
    require(to != msg.sender, "Self-delegation is disallowed.");

    while (voters[to].delegate != address(0)) {
        to = voters[to].delegate;
        require(to != msg.sender, "Found loop in delegation.");
    }

    Voter storage delegate_ = voters[to];
    require(delegate_.weight >= 1);

    sender.voted = true;
    sender.delegate = to;

    if (delegate_.voted) {
        proposals[delegate_.vote].voteCount += sender.weight;
    } else {
        delegate_.weight += sender.weight;
    }
}

```

```

function vote(uint proposal) external {
    Voter storage sender = voters[msg.sender];

    require(sender.weight != 0, "Has no right to vote");
    require(!sender.voted, "Already voted.");

```

```

sender.voted = true;
sender.vote = proposal;

```

```

proposals[proposal].voteCount += sender.weight;
}

```

```

function winningProposal() public view returns (uint winningProposal_) {
    uint winningVoteCount = 0;

```

```

    for (uint p = 0; p < proposals.length; p++) {
        if (proposals[p].voteCount > winningVoteCount) {
            winningVoteCount = proposals[p].voteCount;
            winningProposal_ = p;
        }
    }
}

```

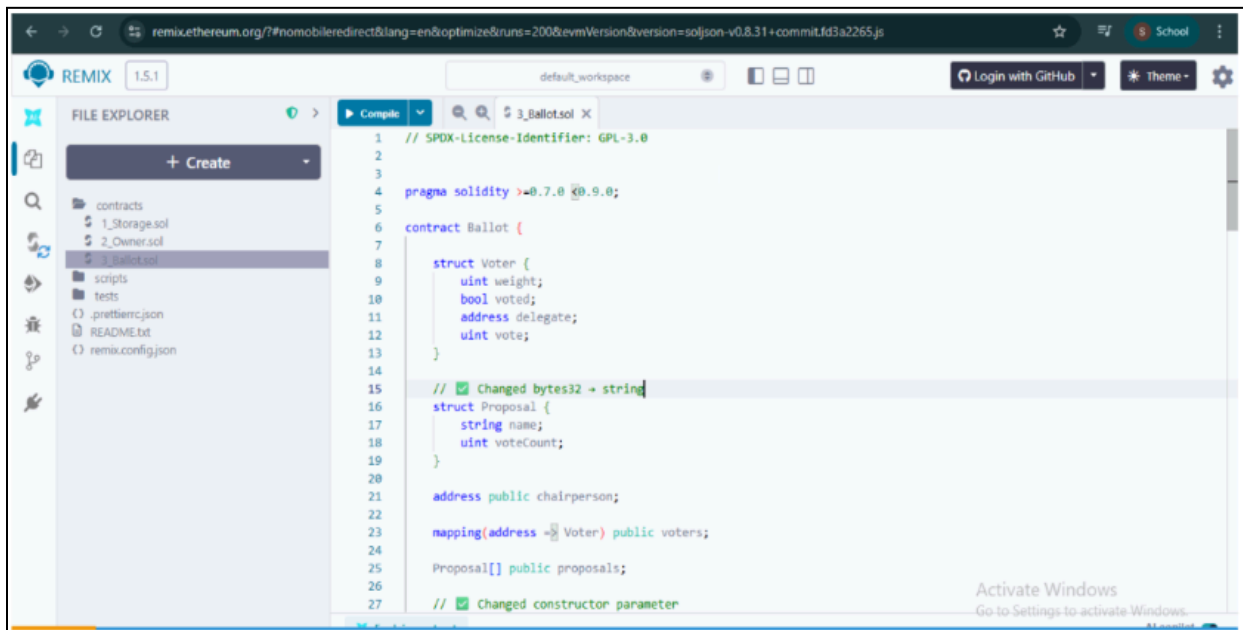
//  Changed return type

```

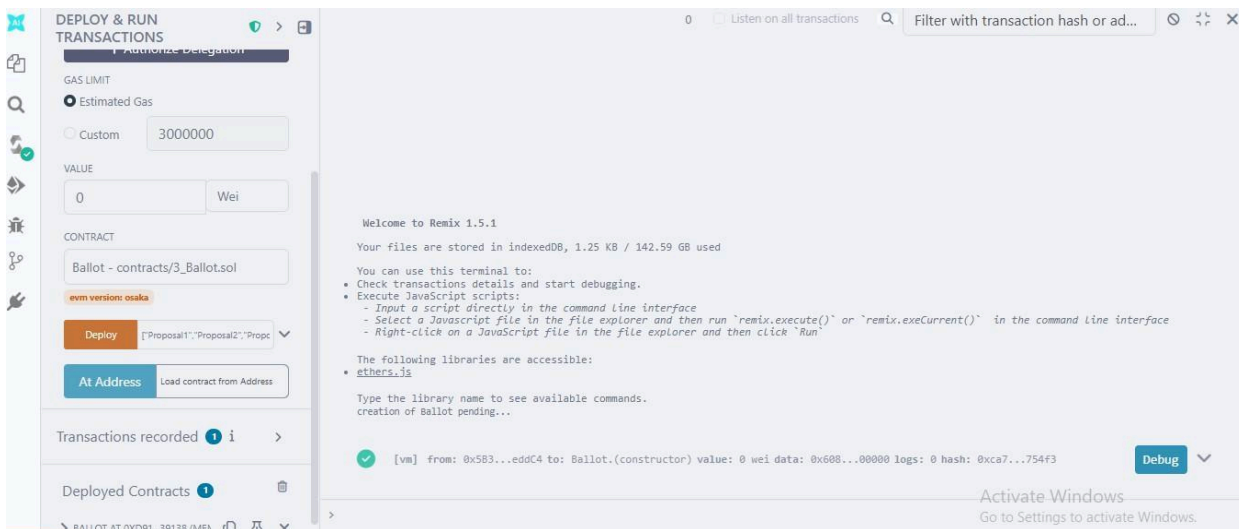
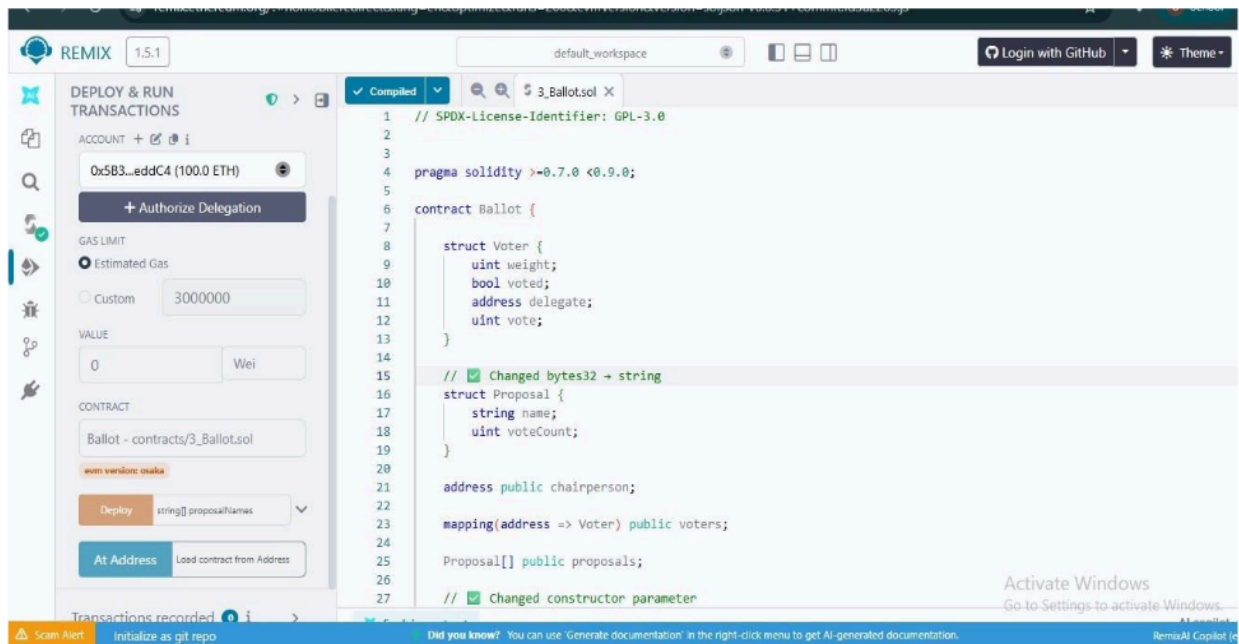
function winnerName() external view returns (string memory winnerName_) {
    winnerName_ = proposals[winningProposal()].name;
}
}

```

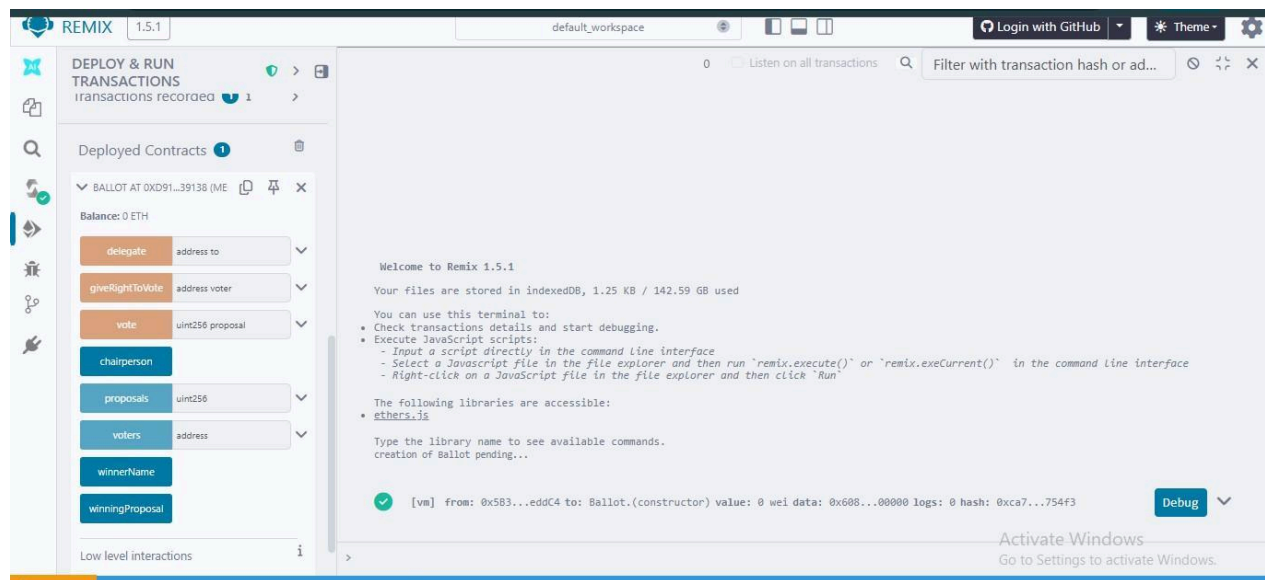
OUTPUT:-



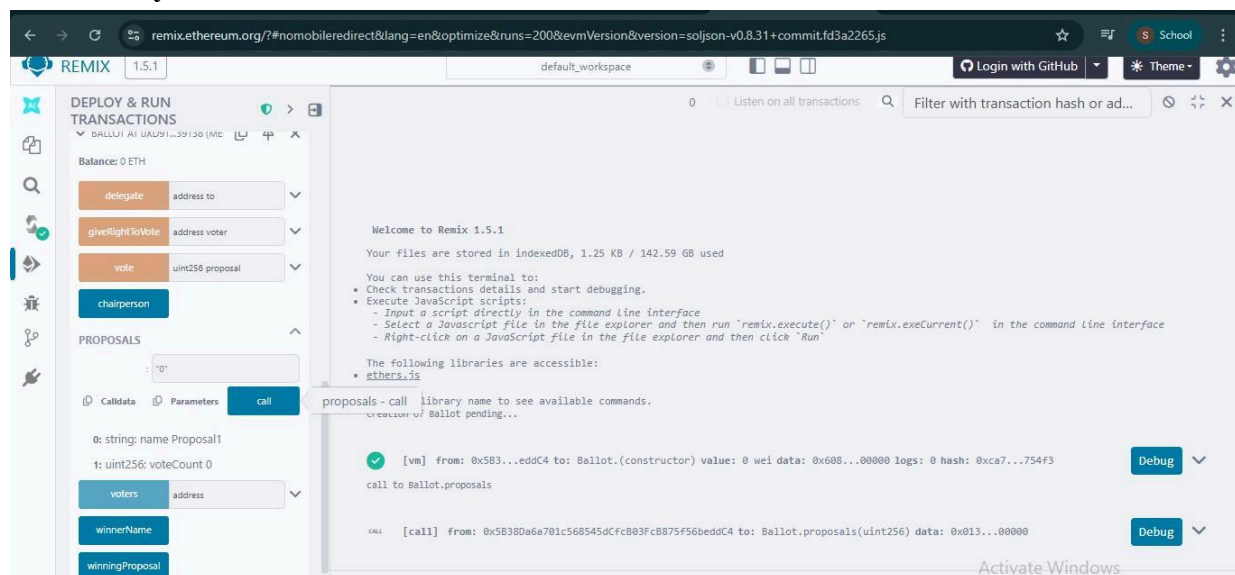
Deploying and running of the contract



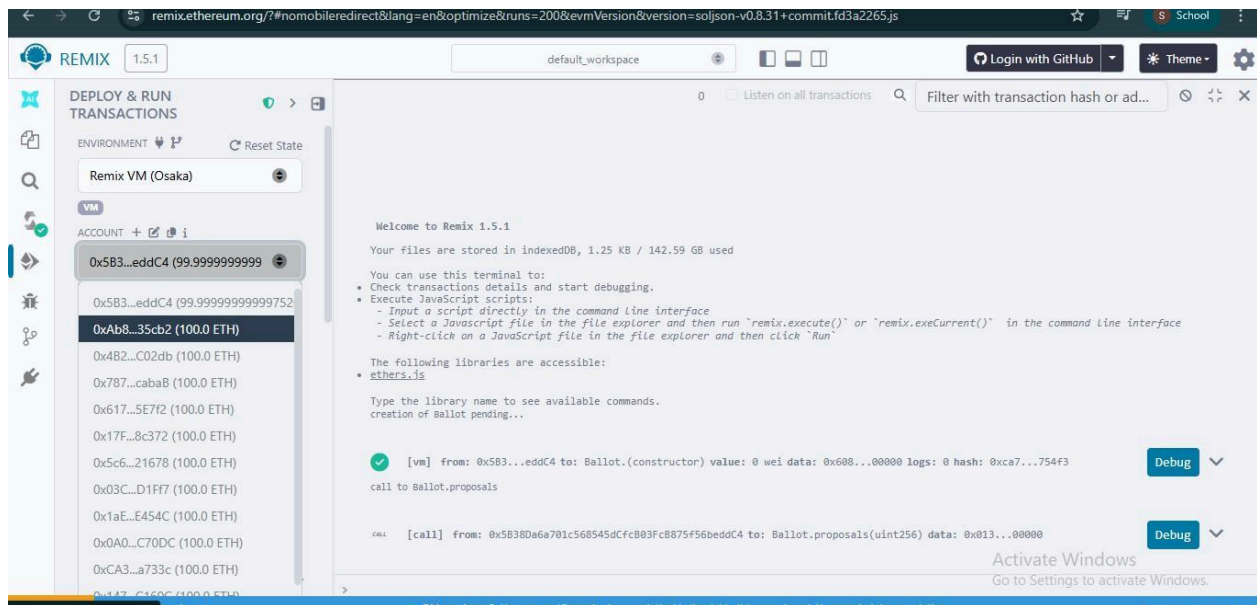
Loading the Proposal Candidate's Names (string)



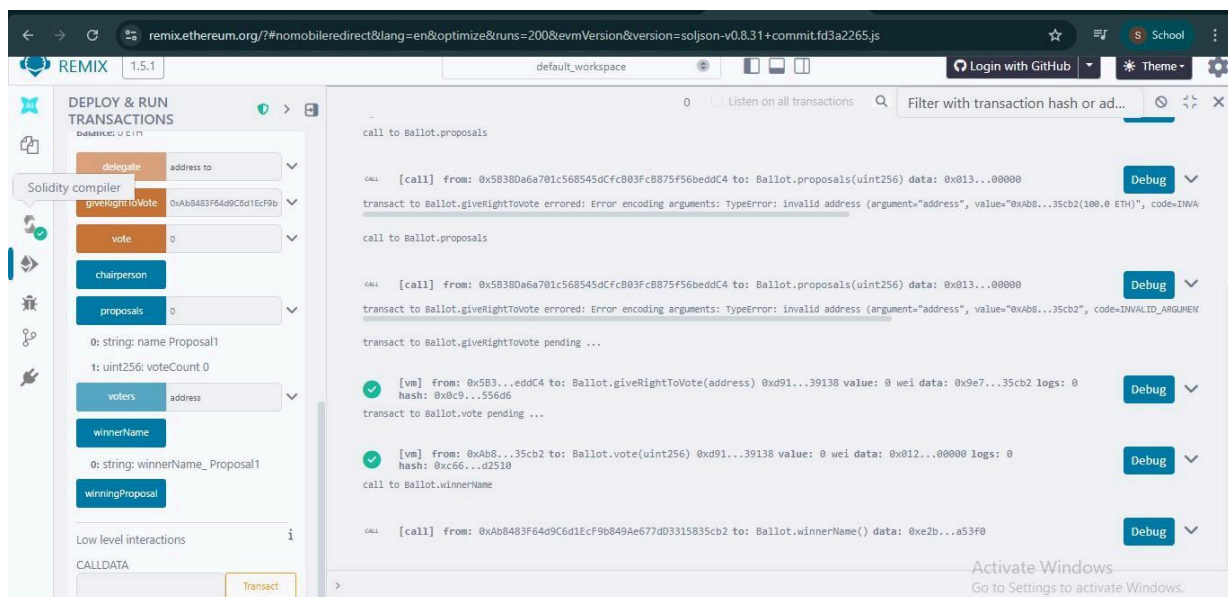
Viewing the details of the Proposal Candidate and Giving the right to an account other than and by the chairman



Selecting the account which was given the right to vote and then writing the proposal candidate's index to vote.



In this final screenshot we can see that after the delegation's vote the weight of one vote of the one voter who was delegated is the number of people who delegated the voter as their voter (Default weight of any voter is 1).



The screenshot displays a web-based interface for a smart contract, likely a voting system. It features several interactive elements:

- Functions (orange buttons):** `delegate`, `giveRightToVote`, and `vote`.
- State Variables (blue buttons):** `chairperson`, `proposals`, `voters`, `winnerName`, and `winningProposal`.
- Input Fields:** A dropdown menu for `address to` (under `delegate`), a text input for `0xAb8483F64d9C6d1EcF9b` (under `giveRightToVote`), a numeric input for `0` (under `vote`), and a dropdown menu for `0` (under `proposals`).
- Output/Status:** A dropdown menu for `address` (under `voters`), and a text input for `0: string: winnerName_ Proposal1` (under `winnerName`).
- Additional Information:** A text input for `0: string: name Proposal1` and a text input for `1: uint256: voteCount 0`.

CONCLUSION:-

In this practical assignment, a Voting (Ballot) Smart Contract was implemented and deployed using Solidity through the Remix IDE environment. The experiment enabled a deeper understanding of how decentralized applications function on the Ethereum blockchain. Essential features such as `require` statements, mapping, and data location specifiers (`storage` and `memory`) were utilized to maintain validation, proper data management, and reliable execution of the contract.

Additionally, the comparison between `bytes32` and `string` data types demonstrated the balance that developers must maintain between minimizing gas consumption and improving readability.

Overall, this exercise enhanced conceptual clarity as well as practical knowledge of smart contract architecture, highlighting the importance of efficiency, security, and thoughtful design in blockchain-based voting systems.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY**
Department of Information Technology
Academic Year: 2025-26

ITC801 : Blockchain & DLT Lab
Experiment 06

Aim: Creating Smart Contracts and performing transactions using Solidity and Remix IDE

Roll No.	04
Name	Mohit Patil
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1:Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO3:Implement smart contracts in Ethereum using different development frameworks. CO5:Interpret different Crypto assets and Crypto currencies

Blockchain Experiment No 6

Aim: Creating Smart Contracts and performing transactions using Solidity and Remix IDE

Theory:

A Smart Contract is a self-executing program stored on the blockchain that automatically enforces the terms of an agreement when predefined conditions are met. It eliminates the need for intermediaries, ensuring transparency, security, and trust between parties. Smart contracts are written in Solidity (for Ethereum-based systems) and executed on the Ethereum Virtual Machine (EVM).

Significance of Smart Contracts in the Ethereum Blockchain:

- Automation: Executes actions automatically when conditions are satisfied.
 - Transparency: The contract code and transactions are visible on the blockchain.
 - Security: Once deployed, contracts are immutable and tamper-proof.
 - Cost-effective: Removes intermediaries and reduces transaction costs.
 - Trustless System: Participants can interact without needing to trust each other directly.
- Smart contracts form the backbone of decentralized applications (DApps) and decentralized finance (DeFi) systems on Ethereum.

Code:

Contract 1: VotingSystem

Voting System :

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;
contract VotingSystem {

    struct Candidate {

        uint id;
        string name;
        uint voteCount;
    }

    mapping(uint => Candidate) public candidates;
    mapping(address => bool) public voters;

    uint public candidatesCount;
```

```

function addCandidate(string memory _name) public {
    candidatesCount++;
    candidates[candidatesCount] = Candidate(candidatesCount, _name, 0);
}

function vote(uint _candidateId) public {
    require(!voters[msg.sender], "Already voted!");
    require(_candidateId > 0 && _candidateId <= candidatesCount, "Invalid candidate");

    voters[msg.sender] = true;
    candidates[_candidateId].voteCount++;
}

function getVotes(uint _candidateId) public view returns (uint) {
    return candidates[_candidateId].voteCount;
}
}

```

Contract 2:

```

Voting
Admin;
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract VotingAdmin {

    address public admin;
    bool public votingActive;

    constructor() {
        admin = msg.sender;
        votingActive = false;
    }

    modifier onlyAdmin() {
        require(msg.sender == admin, "Not admin");
        _;
    }

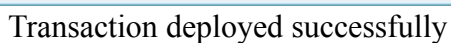
    function startVoting() public onlyAdmin {
        votingActive = true;
    }

    function endVoting() public onlyAdmin {
        votingActive = false;
    }

    function getStatus() public view returns (bool) {
        return votingActive;
    }
}

```


ScreenShots :



Low level interaction

non-payable

addCandidate

string

"Bob"

Value

0

wei

Gas limit

auto

0

Transact

Low level interaction

non-payable

addCandidate

string

"Alice"

Value

0

wei

Gas limit

auto

0

Transact

Both Candidates Added

call

candidatesCount

0:uint256: 2

Call

Balance

0 ETH

Candidate Count

call candidates uint256

1

0:uint256: id 1

1:string: name Alice

2:uint256: voteCount 2

Call

call candidates uint256

2

0:uint256: id 2

1:string: name Bob

2:uint256: voteCount 0

Call

Candidates

call getVotes uint256

Low level interaction

non-payable vote uint256

1

Value 0 wei

Gas limit auto 0

Transact

✓ [vm] from: 0x5B3...eddC4 to: VotingSystem.vote(uint256) 0xd91...39138 value: 0 wei data: 0x012...00001 logs: 0 hash: 0x8f7...c9c5d

status 1 Transaction mined and execution completed

transaction hash 0x8f7ef33e303b9e55f8de9c8106ffaad0c0a25f5ae3cc3826979fd5ad47d9c5d

block hash 0xb5764a0a768a4ddc85d5623eae0013740f6f1c4fc0eb5e90a8bcf443ba87683f

block number 6

from 0x5B38Da6a701c568545dcFc003Fc8875F56beddC4

to VotingSystem.vote(uint256) 0xd9145CCCE52D386f254917e481e844e9943f39138

transaction cost 68570 gas

execution cost 47366 gas

output 0x

decoded input { "uint256_candidateId": "1"

Vote

call getVotes uint256

1

0:uint256: 2

Call

Balance

0 ETH

call getVotes uint256

2

0:uint256: 0

Call

Balance

0 ETH

Vote Count

Conclusion:

The Blockchain-based Voting System was successfully implemented using Solidity smart contracts and deployed on the Remix IDE. Two smart contracts were designed to handle voting operations and administrative control, ensuring modularity and better system design.

The system allowed adding candidates, casting votes, and retrieving vote counts while preventing double voting through the use of mappings and conditions. All transactions were executed on the blockchain, ensuring transparency, security, and immutability of data.

Thus, the experiment helped in understanding the working of smart contracts, their deployment, and execution in a decentralized environment. The objectives of designing and implementing smart contracts were successfully achieved.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab

Experiment 07

Aim: Implement the embedding wallet (Metamask) and transaction using Solidity

Roll No.	04
Name	Mohit Patil
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1:Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO3:Implement smart contracts in Ethereum using different development frameworks. CO5:Interpret different Crypto assets and Crypto currencies

Blockchain Experiment No: 7

AIM: Implement the embedding wallet (Metamask) and transaction using Solidity

Theory:

Blockchain is a decentralized and distributed digital ledger that records transactions securely across multiple systems. It ensures transparency, security, and immutability, meaning once data is recorded, it cannot be changed.

Ethereum is a popular blockchain platform that allows the execution of Smart Contracts. Smart contracts are self-executing programs written in Solidity that automatically perform actions when predefined conditions are met, eliminating the need for intermediaries.

MetaMask is a cryptocurrency wallet and browser extension used to interact with blockchain networks. It stores user accounts and private keys securely and allows users to send, receive, and sign transactions. It also acts as a bridge between Remix IDE and the blockchain network.

Testnets such as Sepolia and RSK Testnet are used for testing purposes. They provide free tokens, allowing developers to deploy and test smart contracts without using real cryptocurrency, reducing financial risk.

Remix IDE is a web-based development environment used to write, compile, deploy, and test smart contracts. It provides a user-friendly interface and supports direct integration with MetaMask for deploying contracts on blockchain networks.

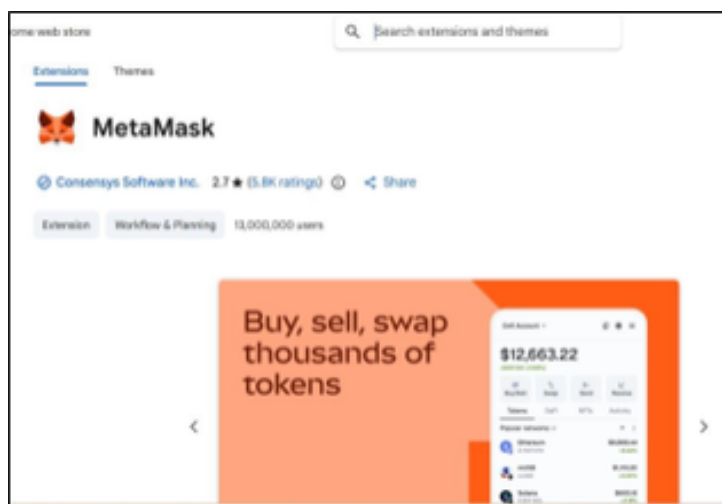
The smart contract deployment process includes writing the contract in Solidity, compiling it to remove errors, connecting MetaMask to Remix, and deploying the contract on a testnet by paying gas fees using test tokens.

A blockchain transaction refers to any interaction with the network, such as deploying a contract or calling its functions. Each transaction requires authentication through a digital signature and is permanently recorded on the blockchain. These transactions can be verified using blockchain explorers.

Implementation

Step 1: Install MetaMask

Install MetaMask extension and create a wallet





<

MetaMask password

Losing this password means losing wallet access on all devices, **MetaMask can't reset it.**

Create new password

••••••••

👁

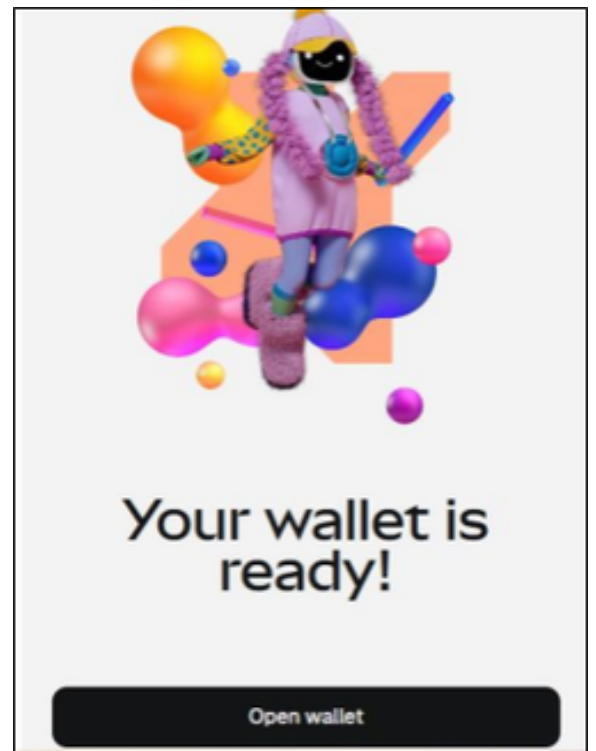
Must be at least 8 characters

Confirm password

••••••••

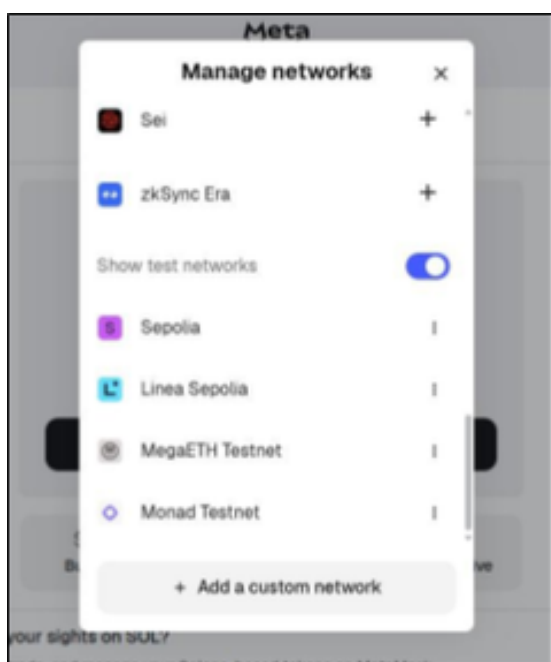
👁

☐ Get product updates, tips, and news including by email. We may use your interactions to improve what we share.



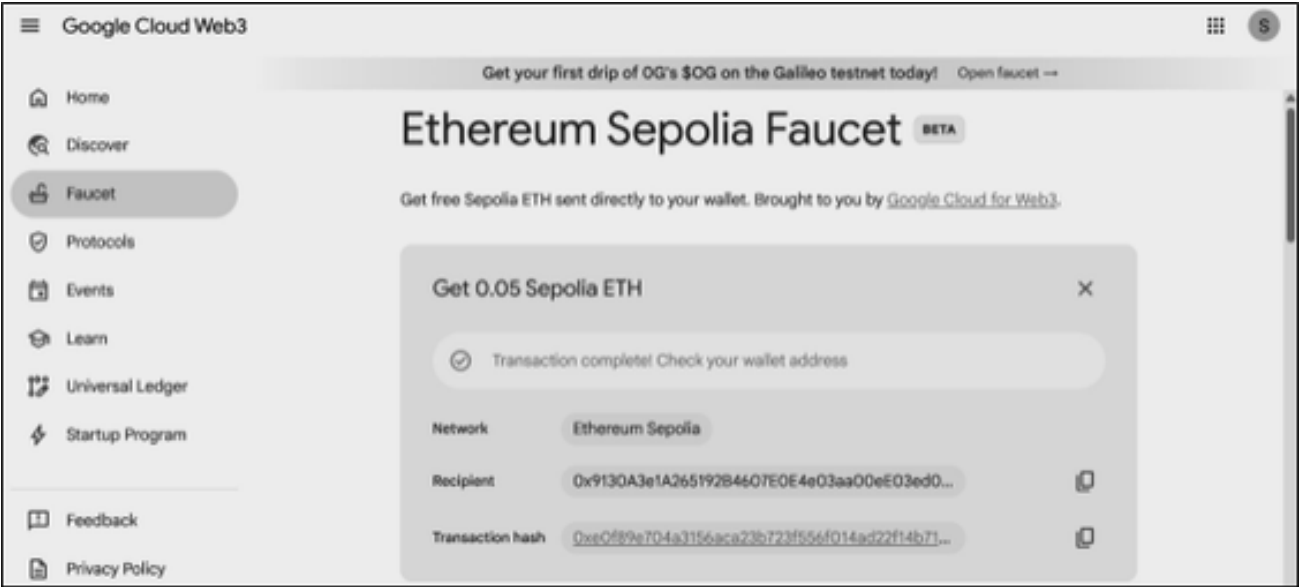
Step 2: Add Sepolia Testnet

Configure MetaMask to connect to Sepolia network.



Step 3: Fund Wallet

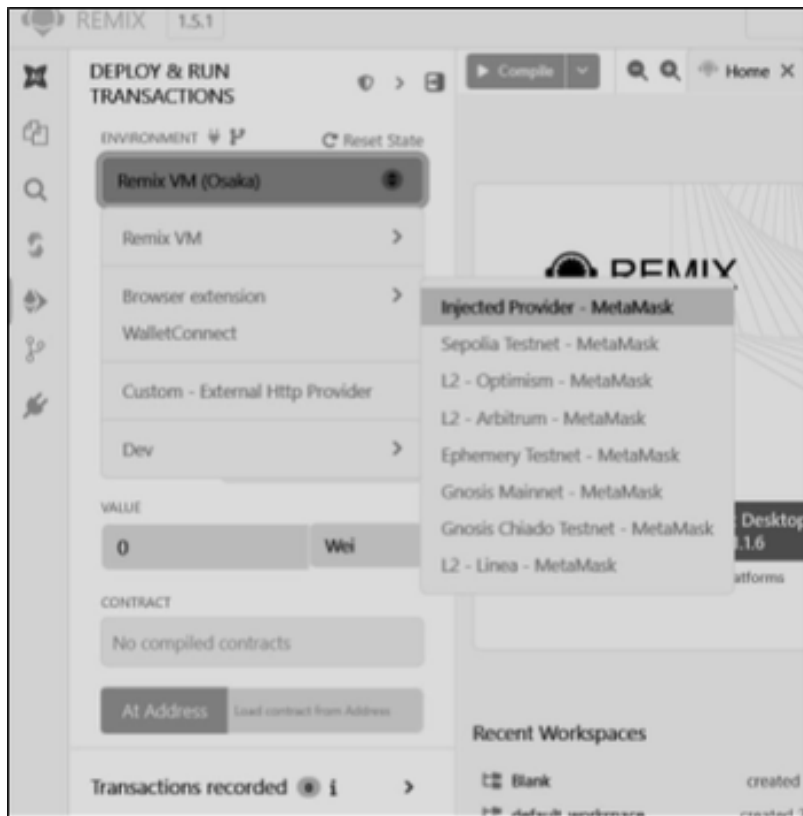
Use faucet to get test ETH

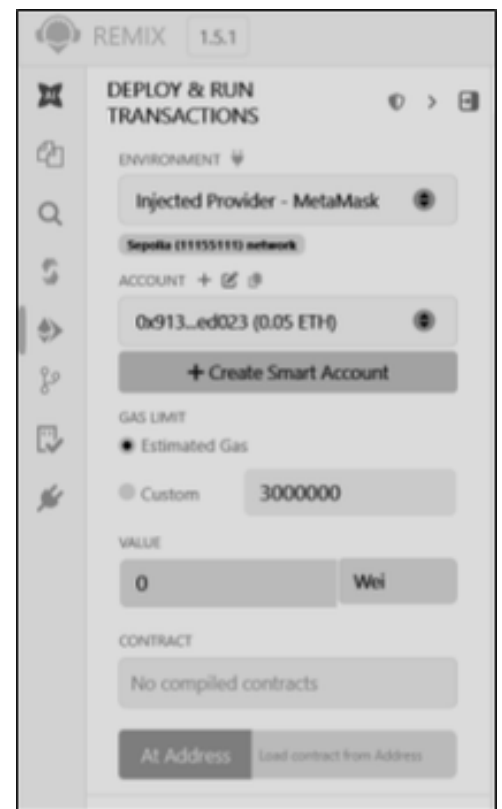


Step4: Connect the Sepolia Testnet / RSK Testnet to Remix IDE

1. In the Remix IDE, select the Environment as Injected Web3.

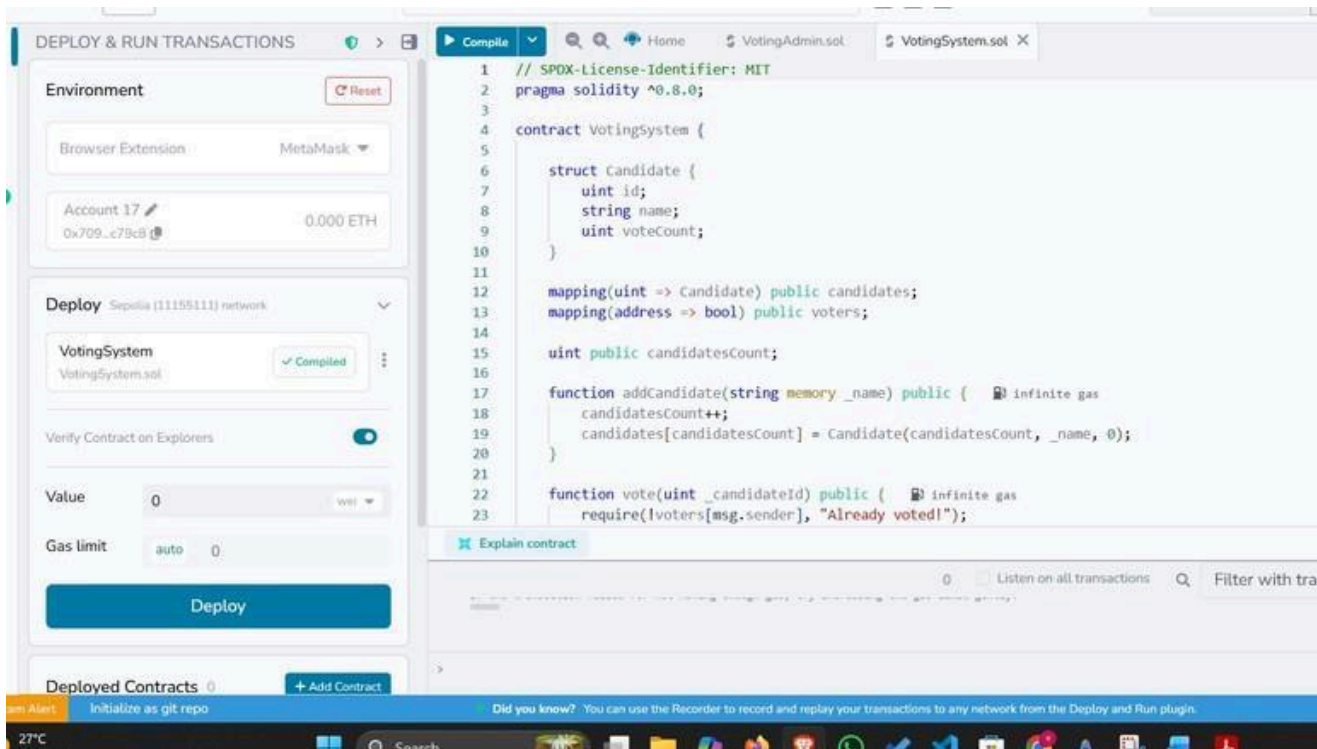
Note : Injected Web3 connects Remix with the active account in Metamask.





Compile the Contract

1. Open Remix
2. Paste your contract code
3. Go to Solidity Compiler (left panel)
4. Select version 0.8.x (match `^0.8.0`)
5. Click Compile `Votingsystem.sol`



Connect MetaMask (Injected Provider)

1. Go to Deploy & Run Transactions tab
2. In Environment → select Injected Provider - MetaMask
3. MetaMask popup will open → click Connect Make sure:
 - Network = Sepolia Test Network
 - You have 0.05 Sepolia ETH

Environment

Reset

Browser Extensi... MetaMask ▼

Account 19 
0x862...c1199  0.000 ETH

Deploy Sepolia (11155111) network ▼

VotingSystem
VotingSystem.sol

✓ Compiled ⋮

Verify Contract on Explorers ☒

Value 0 wei ▼

Gas limit custom 3000000

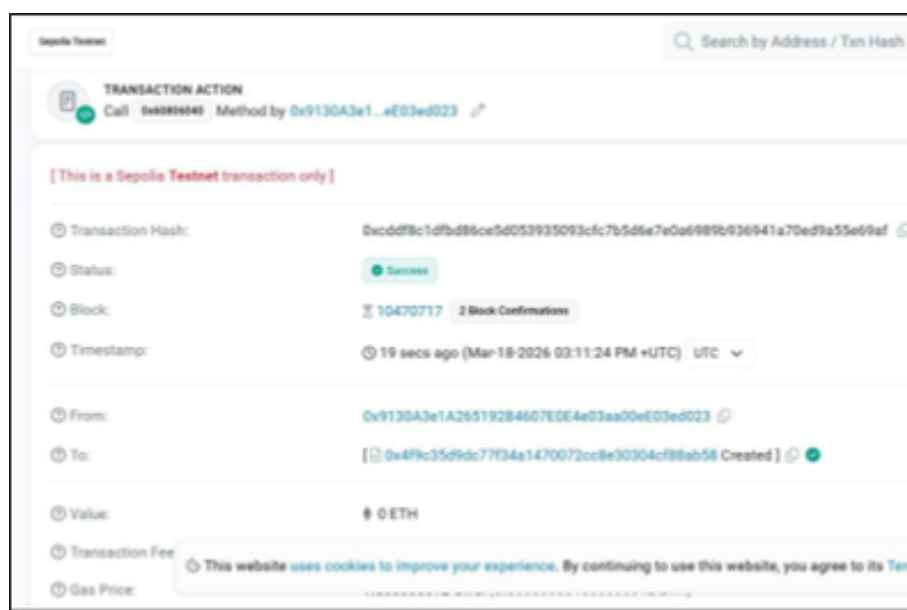
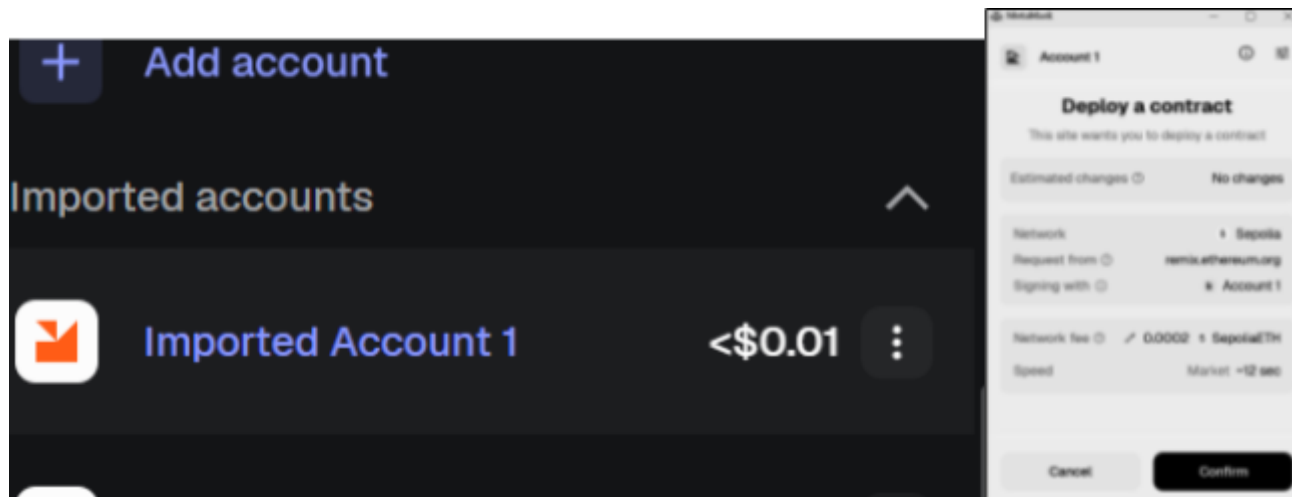
Deploy

Deploy Contract

Your constructor needs a Voting address Steps:

1. Copy your second MetaMask account address (voting)
 - MetaMask → Account 2 → Copy address
2. Paste that address in constructor field
3. Click Deploy
4. MetaMask will pop up → Confirm transaction Wait a few seconds...

After success → your contract appears under Deployed Contracts



Contract deployment

Status

Confirmed

View on block explorer

Copy transaction ID

From

Ox9f30A...ed023

New contract

To

Transaction

Nonce

1

Amount

-0 SepoliaETH

Gas Limit (Units)

826833

Gas Used (Units)

82149

Base fee (GWEI)

0.000000012

Priority fee (GWEI)

1.5

Total gas fee

0.005232 SepoliaETH

Max fee per gas

0.000000002 SepoliaETH

Total

0.00123172 SepoliaETH

DEPLOY & RUN TRANSACTIONS

0x9f30A...ed023 0.04753655298

Create Smart Account

GAS LIMIT

Estimated Gas

Custom 3000000

VALUE

0 Wei

CONTRACT

PharmaSupply - PharmaSupply.sol

view source code

Verify Contract on Explorers

Deploy & verify address: pharmacy

At Address Load contract from Address

Transactions recorded 1

Deployed Contracts

PHARMASUPPLY AT 0x812...801

Welcome to Remix 1.5.1

Your files are stored in IndexedDB, 398.24 KB /

You can use this terminal to:

- Check transaction details and start debugging.
- Execute JavaScript scripts:
 - Input a script directly in the command line
 - Select a Javascript file in the file explorer
 - Right-click on a Javascript file in the file explorer

The following libraries are accessible:

- ethers.js

Type the library name to see available commands or create a new contract...

view on Etherscan view on Blockscout

[block:18478717 txIndex:46] from: 0x9f30A...ed023

[verification] Contract deployed. Checking expl

Etherscan verification skipped: API key not pro

Please input the API key in Remix Settings - in

[Blockscout] Verification submitted. Awaiting c

[Morescan] Verification submitted. Awaiting c

[Sourceify] Verification Error: Contract notDef

[Sourceify] Please open the "Contract Verificat

[Blockscout] Verification Successful! View Co

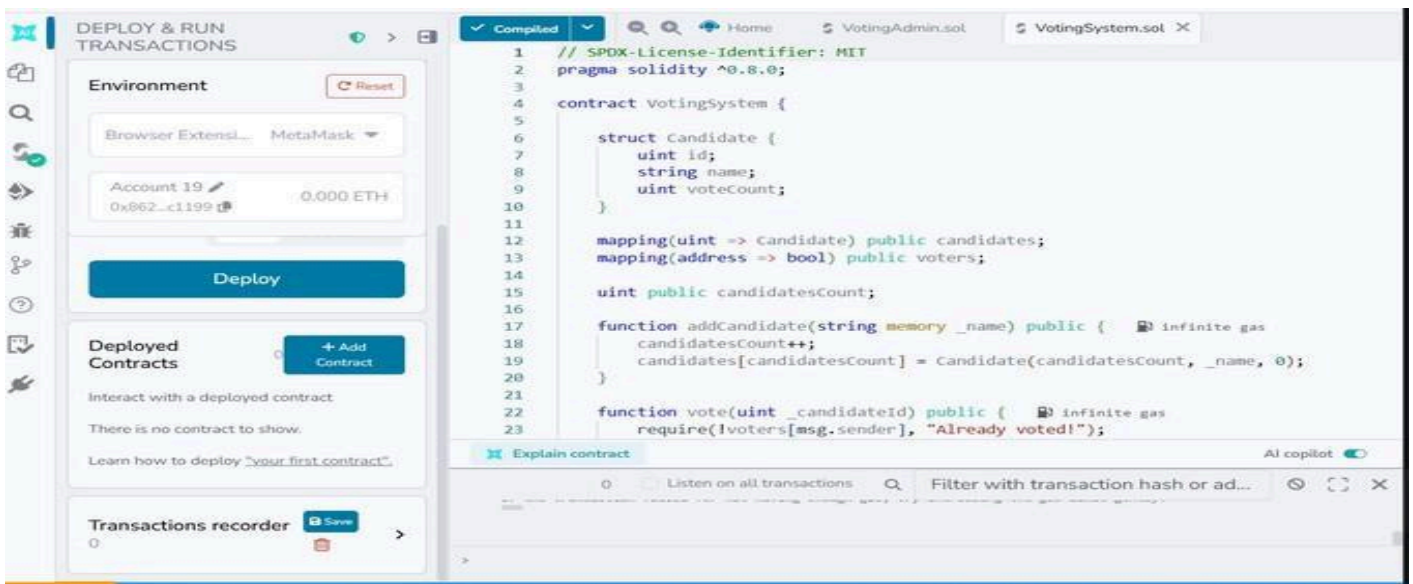
[Morescan] Verification Successful! View Co



Interact with Contract

Step 1: Add Medicine (Manufacturer only)

- Make sure MetaMask is on Account 1 (manufacturer)
- In Remix:
 - Enter:
 - `_candidate`: "Alice"
- Click addCandidate Confirm in MetaMask



Step 2: Calculate Vote

- Still on Account 1
- Click calculateVote
- Confirm transaction

[view on etherscan](#)[view on blocksout](#)

[block:10470765 txIndex:24] from: 0x913...e0023 to: PharmaSupply.shipMedicine() 0xbdc2...b6cf8 val: data: 0x39f...32075 logs: 0 hash: 0x9cd...6ffe2

status

1 Transaction mined and execution succeed

transaction hash

0x2c41b01fa5f9c8e66c4a8a4f55b3d2d9340a5f8b9f2ec8c1caf3337e880

block hash

0x9c092762208d4c4297e5f1674d25516e04d0f0030341892096c5d699ca16ffe2

block number

10470765

from

0x9130A3e1A20519204687E0E4e03ae06c01ed823

to

PharmaSupply.shipMedicine() 0xbdc2ef31a07E3BcA204f70f091a0C8D2a5efb1cF8

transaction cost

45570 gas

decoded input

()

decoded output

-

logs

[]

raw logs

[]

Account 1

Transaction request

Estimated changes

No changes

Network

Sepolia

Request from

remix.ethereum.org

Interacting with

0xbdc2ef_b6cf8

Network fee

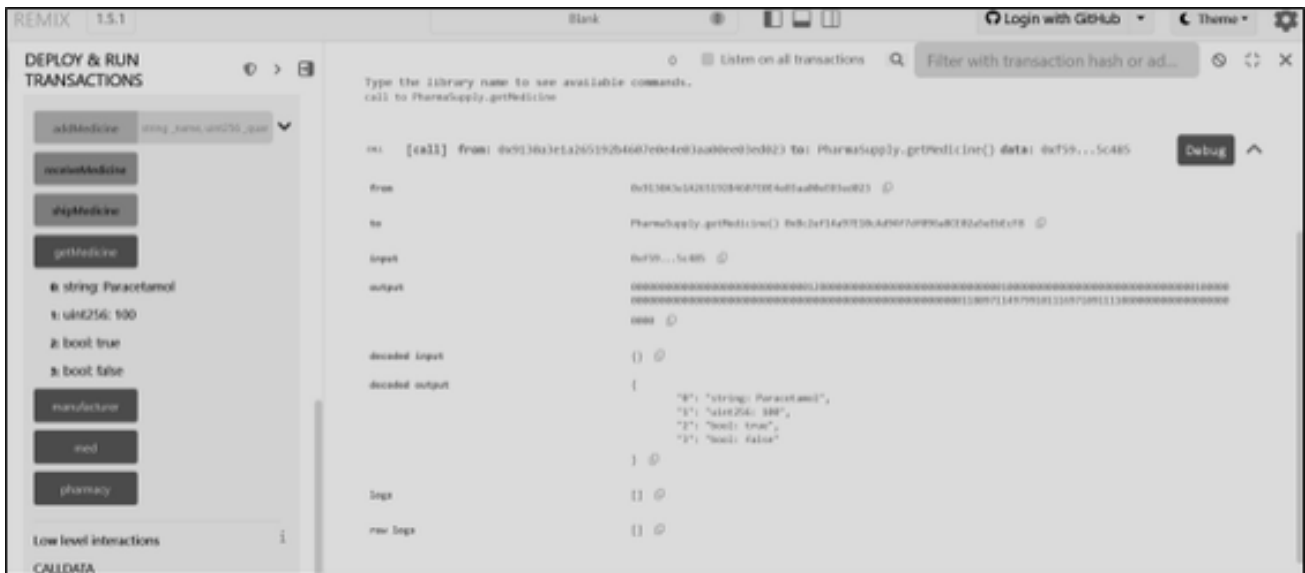
0.0001 SepoliaETH

Speed

Market ~12 sec

Cancel

Confirm



Conclusion

The experiment was successfully carried out by integrating the MetaMask wallet with Remix IDE and performing transactions on the Sepolia test network. The smart contract was compiled, deployed, and interacted with using MetaMask, which enabled secure transaction signing and real-time confirmation on the blockchain.

The use of a test network allowed safe execution without real financial risk, while also providing practical exposure to gas fees, transaction validation, and blockchain explorers. This experiment helped in understanding wallet integration, smart contract deployment, and the overall working of decentralized applications.

Thus, the objective of implementing MetaMask and performing blockchain transactions was successfully achieved.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment 08

Aim: To implement the Blockchain platform Ganache and deploy a simple Solidity smart contract using Metamask and Remix IDE.

Roll No.	04
Name	Mohit Patil
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO3:Implement smart contracts in Ethereum using different development frameworks. CO5:Interpret different Crypto assets and Crypto currencies

Blockchain Experiment 8

AIM: Implementing a Blockchain Platform using Ganache

Theory:

1. Blockchain Technology

A blockchain is a distributed, decentralized, and immutable ledger that records transactions across a network of computers. Each record (called a block) contains a cryptographic hash of the previous block, a timestamp, and transaction data, forming a chain. Because every participant holds a copy of the ledger and any alteration to a block invalidates all subsequent blocks, the data stored on a blockchain is highly tamper-resistant and transparent.

Key properties of blockchain:

- Decentralization: No single entity controls the network; authority is distributed across nodes.
- Immutability: Once data is written to the blockchain, it cannot be altered or deleted.
- Transparency: All transactions are visible to network participants.
- Consensus: Nodes agree on the state of the ledger through consensus mechanisms (e.g., Proof of Work, Proof of Stake).
- Security: Cryptographic hashing (SHA-256) ensures data integrity.

2. Ethereum & Smart Contracts

Ethereum is an open-source, decentralized blockchain platform that supports programmable transactions through Smart Contracts. A smart contract is a self-executing program stored on the blockchain whose terms are written directly into code. Once deployed, a smart contract runs automatically when predefined conditions are met, without the need for intermediaries.

Smart contracts in Ethereum are written in Solidity, a statically-typed, contract-oriented programming language. They are compiled into bytecode and deployed to the Ethereum Virtual Machine (EVM), which executes the contract logic on every node in the network.

Key concepts:

- Gas: A unit measuring the computational effort required to execute operations. Users pay gas fees in ETH to incentivize miners/validators.
- ABI (Application Binary Interface): Defines how to interact with the deployed contract — the function signatures and data encodings.
- Address: Every deployed contract gets a unique Ethereum address on the blockchain.
- msg.sender: A global variable in Solidity that holds the address of the account that called the current function.

3. Ganache

Ganache (by Truffle Suite / Consensys) is a personal blockchain for Ethereum development. It simulates

a full Ethereum node locally on your machine, providing:

- 10 pre-funded test accounts, each loaded with 100 ETH.
- Instant mining — transactions are confirmed immediately (no wait time).
- A local RPC server (default: <http://127.0.0.1:7545>) that tools like MetaMask and Remix can connect to.
- A GUI showing real-time accounts, blocks, transactions, and logs for easy inspection.

Ganache is used exclusively for development and testing. It allows developers to deploy and test contracts without spending real ETH or waiting for mainnet/testnet confirmations.

4. MetaMask

MetaMask is a cryptocurrency wallet and browser extension that acts as a bridge between a web browser and the Ethereum blockchain. It allows users to manage Ethereum accounts, sign transactions, and interact with decentralized applications (dApps) directly from the browser. In development, MetaMask is configured to point to a local Ganache network instead of the Ethereum mainnet, enabling safe testing with dummy ETH.

5. Remix IDE

Remix IDE is a web-based integrated development environment for Solidity smart contract development, available at remix.ethereum.org. It provides a code editor, Solidity compiler, contract deployer, and debugger in a single interface. Remix can connect to various environments including the built-in JavaScript VM, Hardhat, or an Injected Provider (MetaMask), the last of which routes deployment through MetaMask to whichever network (Ganache, testnet, or mainnet) MetaMask is currently connected to.

6. Supply Chain Use Case

A supply chain smart contract automates the tracking of goods as they move between parties — in this experiment, a pharmaceutical product (medicine) passing through four stages: Manufacturer, Distributor, Retailer, and Customer. Each stage updates the contract state on-chain, creating a transparent and tamper-proof audit trail. This eliminates the need for a trusted central authority to verify ownership transfers and ensures all parties see the same data.

Implementation:

Step 1: Install Ganache

Ganache is a personal blockchain for Ethereum development used to deploy contracts, develop applications, and run tests.

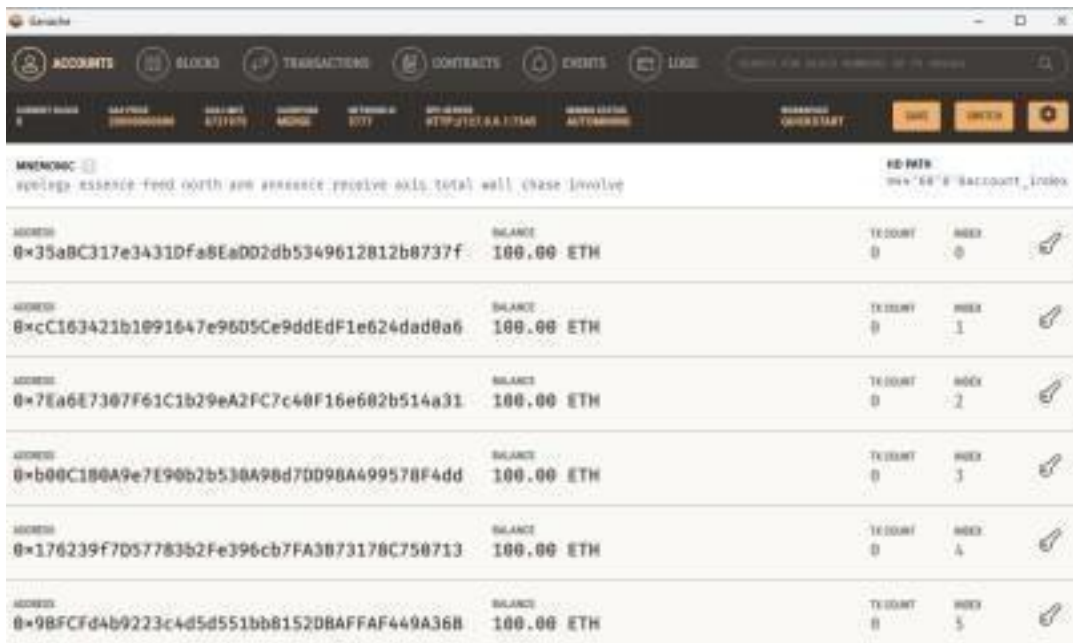
1. Download Ganache v2.7.1 from the official Truffle Suite website ([release.assets.githubusercontent.com](https://github.com/trufflesuite/ganache/releases)).

2. Run the installer. The installation prompt shows publisher as Consensys Software Inc.
3. Check 'Launch when ready' and click Install.



4. Once installed, open Ganache and click Quickstart Ethereum to start a local blockchain.





ACCOUNTS	BLOCKS	TRANSACTIONS	CONTRACTS	EXPORTS	LOGS
ACCOUNTS ACCOUNT ID ADDRESS BALANCE TX COUNT INDEX KEY					
ACCOUNT ID ADDRESS BALANCE TX COUNT INDEX KEY					
ACCOUNT ID ADDRESS BALANCE TX COUNT INDEX KEY					
ACCOUNT ID ADDRESS BALANCE TX COUNT INDEX KEY					
ACCOUNT ID ADDRESS BALANCE TX COUNT INDEX KEY					
ACCOUNT ID ADDRESS BALANCE TX COUNT INDEX KEY					

Note the following details from Ganache for later use:

- RPC Server URL: <http://127.0.0.1:7545>
- Network ID: 5777
- Private keys of accounts (click the key icon next to each address)

Step 2: Install & Setup MetaMask

MetaMask is a browser extension that serves as an Ethereum wallet and allows interaction with decentralized applications.

2.1 Install MetaMask

5. Open Google Chrome and go to the Chrome Web Store.
6. Search for MetaMask (by Consensys Software Inc.) and click Add to Chrome.



7. After installation, click 'Create a new wallet' on the MetaMask welcome screen.

8. Set a password and save the Secret Recovery Phrase securely.



2.2 Add Ganache Network to MetaMask

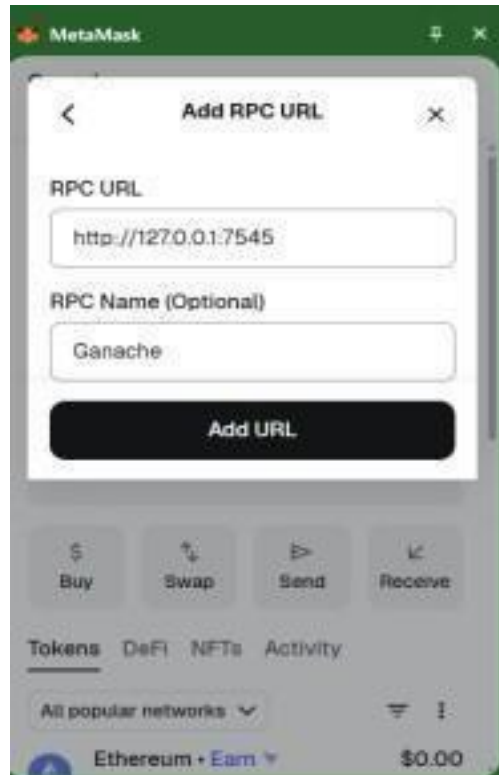
9. In MetaMask, click the network dropdown and select 'Add a custom network'.

10. Enter the following details for the RPC

URL: RPC URL: `http://127.0.0.1:7545`

RPC Name: Ganache

11. Click 'Add URL'.



12. Fill in the remaining custom network details:

- Network Name: Localhost 8545

- Default RPC URL: Ganache (127.0.0.1:7545)
- Chain ID: 1337
- Currency Symbol: ETH

13. Click Save.



Add a custom network

Network name
Localhost 8545

Default RPC URL
Ganache 127.0.0.1:7545

Chain ID
1337

Currency symbol
ETH

Block explorer URL

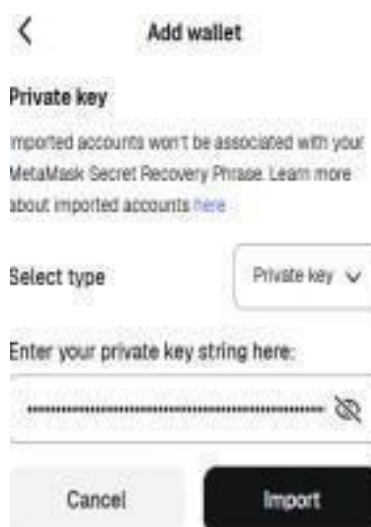
Save

2.3 Import Ganache Accounts into MetaMask

14. In Ganache, click the key icon next to an account address to reveal its private key.

15. In MetaMask, go to Accounts > Add wallet > Import using Private Key.

16. Paste the private key and click Import.



Add wallet

Private key

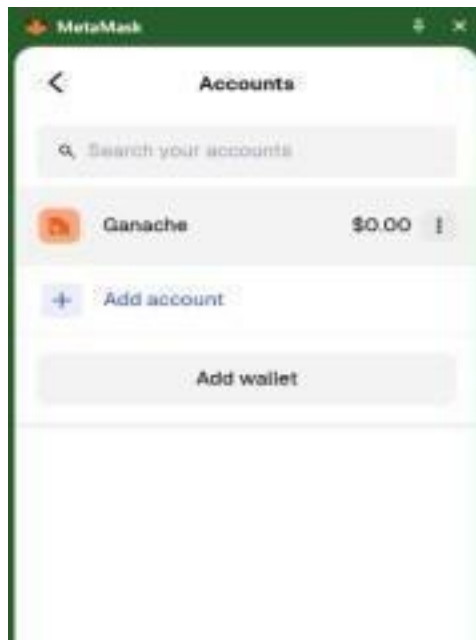
Imported accounts won't be associated with your MetaMask Secret Recovery Phrase. [Learn more about imported accounts here](#)

Select type
Private key

Enter your private key string here:

Cancel Import

17. The imported account (labeled 'Ganache') now appears in MetaMask under the Localhost 8545 network.



Step 3: Create the Smart Contract in Remix IDE

Remix IDE is a web-based Solidity development environment at remix.ethereum.org.

18. Open remix.ethereum.org in the browser.
19. In the File Explorer, create a new file named VotingSystem.sol.
20. Write the following smart contract for a Voting System chain:

5777

Step 4: Compile the Smart Contract

21. In Remix, click the Solidity Compiler icon on the left sidebar.
22. Select compiler version 0.8.x.
23. Click Compile SupplyChain.sol.
24. A green checkmark confirms successful compilation.



Step 5: Connect Remix IDE with MetaMask

25. In Remix, navigate to the Deploy & Run Transactions tab.
26. In the Environment dropdown, select Injected Provider - MetaMask.
27. MetaMask will prompt a connection request — click Connect.
28. Ensure MetaMask is switched to the Localhost 8545 (Ganache) network.
29. The imported Ganache account will now appear as the active account in Remix.

Step 6: Deploy the Smart Contract

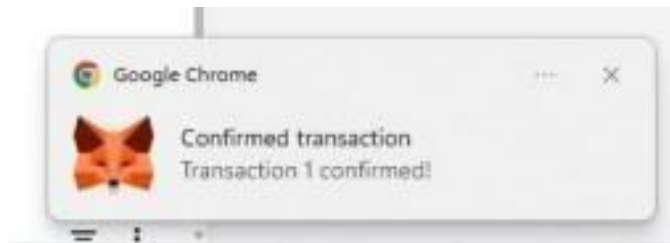
30. In the Deploy & Run panel, confirm VoteSystem is selected under Contract.
31. Click Deploy. MetaMask will open a confirmation popup.



32. Review the Network fee and click Confirm in MetaMask.



33. A 'Confirmed transaction' notification from Chrome confirms the deployment was successful.



Step 7: Verify Transaction in Ganache

After deployment, Ganache reflects the transaction. The deploying account's balance decreases due to gas fees paid.

ACCOUNT ID: 1

SAL: 00

The screenshot shows the MetaMask interface with the 'ACCOUNTS' tab selected. The account list shows a balance of 99.92 ETH. Below the account list, a table displays transaction history with columns for ADDRESS, BALANCE, TX COUNT, and INDEX. The transactions listed are:

ADDRESS	BALANCE	TX COUNT	INDEX
0x35aBC317e3431Dfa8Ea002db5349612812b0737f	99.92 ETH	2	1
0xcC163421b1091647e96D5Ce9ddEdF1e624dad8a6	100.00 ETH	0	1
0x7Ea6E7307F61C1b29eA2FC7c49F16e602b514a31	100.00 ETH	0	2
0xb00C188A9e7E90b2b530A98d7D098A499578F4dd	100.00 ETH	0	3
0x176239f7D57783b2Fe396cb7FA3B73178C750713	100.00 ETH	0	4
0x9BFCfd4b9223c4d5d551bb815208AFFAF449A368	100.00 ETH	0	5

The contract deployment details in MetaMask show:

- Status: Confirmed
- From: 0x35aBC...0737f → To: New contract
- Gas Used: 849,377 units
- Total gas fee: 0.016988 ETH
- Account balance after: 99.92 ETH

The screenshot shows the 'Contract deployment' modal in MetaMask. It displays the status as 'Confirmed' and provides a link to 'View on block explorer'. The transaction details are as follows:

Contract deployment	
Status	Confirmed
From	0x35aBC...0737f
To	New contract
Transaction	
Nonce	1
Amount	-0 ETH
Gas Limit (units)	849377
Gas Used (units)	849377
Base fee (GWEI)	0.00252972
Priority fee (GWEI)	1013476728
Total gas fee	0.016988 ETH
	\$39.42
Max fee per gas	0.00000002 ETH
	\$0.00
Total	0.01698754 ETH
	\$39.42

Step 8: Interact with the Smart Contract

After deployment, the contract functions appear in the Remix Deploy & Run panel under Deployed Contracts.

Transaction Flow — Pharma Supply Chain

Account 1 (Manufacturer) — Call addMedicine:

```
addMedicine(1, "Paracetamol", 100)
```

Account 2 (Distributor) — Switch account in MetaMask, then call:

```
sendToDistributor(1)
```

Account 3 (Retailer) — Switch account in MetaMask, then call:

```
sendToRetailer(1)
```

Account 4 (Customer) — Switch account in MetaMask, then call:

```
purchase(1)
```

Call getMedicine to verify the full supply chain:

```
getMedicine(1)
```

Expected output from getMedicine(1):

- id: 1
- name: Paracetamol
- quantity: 100
- manufacturer: 0x35aBC...0737f (Account 1 address)
- distributor: 0xcC163...d0a6 (Account 2 address)
- retailer: 0x7Ea6E...a31 (Account 3 address)
- customer: 0xb00C1...4dd (Account 4 address)

Output

- Medicine batch created successfully on the Ganache local blockchain.
- Complete supply chain flow executed: Manufacturer → Distributor → Retailer → Customer.
- All transactions confirmed and visible in Ganache with updated block counts and reduced ETH balances.
- Smart contract deployed with status Confirmed; gas fees deducted correctly.

Conclusion

The experiment was successfully carried out by implementing the Ganache platform and integrating it with MetaMask and Remix IDE. The smart contract was compiled, deployed, and executed using Ganache accounts, which provided a local blockchain environment with pre-funded accounts.

The transactions were processed instantly, and all details such as gas usage, block creation, and account balances were observed directly in Ganache. This provided a clear understanding of how blockchain transactions work in a controlled environment without requiring real cryptocurrency.

Thus, the experiment helped in understanding the practical implementation of a local blockchain, wallet integration, and smart contract deployment successfully.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY**
Department of Information Technology
Academic Year: 2025-26

ITC801 : Blockchain & DLT Lab
Experiment 09

Aim: To implement a Private Ethereum Blockchain using Geth.

Roll No.	04
Name	Mohit Patil
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO3:Implement smart contracts in Ethereum using different development frameworks.

Blockchain Lab

EXP 9

Aim - To implement a Private Ethereum Blockchain using Geth.

Theory -

Ethereum is an open-source, decentralized blockchain platform that enables developers to build and deploy **smart contracts** and **decentralized applications (DApps)**. Unlike public blockchains such as the Ethereum mainnet, a **Private Ethereum Blockchain** is restricted to specific participants, making it ideal for **development, testing, and enterprise applications** where confidentiality, control, and performance are essential.

Geth (Go-Ethereum) is one of the most widely used Ethereum clients, written in the Go programming language. It allows users to run an Ethereum node, mine Ether, and interact with the blockchain through **JSON-RPC, command-line, or JavaScript consoles**.

Setting up a **Private Ethereum Network** involves creating a customized blockchain environment isolated from the public Ethereum network. It enables developers to test smart contracts, consensus mechanisms, and network behavior without incurring gas costs or depending on public validators.

Steps Involved:

1. **Choosing a Network ID:**
2. Each private blockchain must have a unique network ID to differentiate it from public or other private networks.
3. **Choosing a Consensus Algorithm:**
4. Ethereum supports different consensus mechanisms such as **Proof of Work (PoW)** and **Proof of Authority (PoA)**. In private networks, **Clique (PoA)** is often used for faster block confirmation.
5. **Creating a Genesis Block:**
6. The genesis block is the first block of the blockchain, defining network parameters such as difficulty, gas limit, initial account balances, and consensus settings.
7. **Initializing the Geth Database:**
8. The `geth init` command initializes the blockchain using the `genesis.json` file, preparing the data directory for node operation.
9. **Setting up Networking:**
10. Nodes communicate using **enodes** (Ethereum Node URLs) that contain IP addresses and public keys for peer discovery and connection.
11. **Running the Member Nodes:**

12. Each participant runs a Geth node that connects to the private network and participates in block validation and transaction processing.
13. **Running a Signer (in Clique):**
14. In the **Clique consensus**, designated **signers** validate and sign blocks. These accounts are pre-configured in the genesis file to ensure trusted block production.

Outputs -

1. First install all the necessary dependencies like curl, git, build-essential.

```
sudo apt install wget curl git build-essential -y
```

```
deilsy@deilsy:~$ sudo apt install wget curl git build-essential -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
wget is already the newest version (1.21.4-1ubuntu4.1).
wget set to manually installed.
git is already the newest version (1:2.43.0-1ubuntu7.3).
build-essential is already the newest version (12.10ubuntu1).
The following additional packages will be installed:
  libcurl3t64-gnutls libcurl4t64
The following packages will be upgraded:
  curl libcurl3t64-gnutls libcurl4t64
3 upgraded, 0 newly installed, 0 to remove and 57 not upgraded.
Need to get 902 kB of archives.
After this operation, 0 B of additional disk space will be used.
Get:1 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 curl amd64 8
.5.0-2ubuntu10.8 [227 kB]
Get:2 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libcurl4t64
amd64 8.5.0-2ubuntu10.8 [342 kB]
Get:3 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libcurl3t64-
gnutls amd64 8.5.0-2ubuntu10.8 [334 kB]
Fetched 902 kB in 1s (650 kB/s)
(Reading database ... 46694 files and directories currently installed.)
Preparing to unpack .../curl_8.5.0-2ubuntu10.8_amd64.deb ...
Unpacking curl (8.5.0-2ubuntu10.8) over (8.5.0-2ubuntu10.6) ...
Preparing to unpack .../libcurl4t64_8.5.0-2ubuntu10.8_amd64.deb ...
Unpacking libcurl4t64:amd64 (8.5.0-2ubuntu10.8) over (8.5.0-2ubuntu10.6) ...
Preparing to unpack .../libcurl3t64-gnutls_8.5.0-2ubuntu10.8_amd64.deb ...
Unpacking libcurl3t64-gnutls:amd64 (8.5.0-2ubuntu10.8) over (8.5.0-2ubuntu10
.6) ...
Setting up libcurl4t64:amd64 (8.5.0-2ubuntu10.8) ...
Setting up libcurl3t64-gnutls:amd64 (8.5.0-2ubuntu10.8) ...
Setting up curl (8.5.0-2ubuntu10.8) ...
Processing triggers for man-db (2.12.0-4build2) ...
Processing triggers for libc-bin (2.39-0ubuntu8.7) ...
```

2. Install geth using wget

```
cd /usr/local/bin && sudo wget
```

```
https://gethstore.blob.core.windows.net/builds/geth-linux-amd64-1.12.0-e501b3b0.tar.gz
```

```

deilsy@deilsy:~$ cd /usr/local/bin && sudo wget https://gethstore.blob.core.w
indows.net/builds/geth-linux-amd64-1.12.0-e501b3b0.tar.gz
--2026-03-26 03:54:03-- https://gethstore.blob.core.windows.net/builds/geth
-linux-amd64-1.12.0-e501b3b0.tar.gz
Resolving gethstore.blob.core.windows.net (gethstore.blob.core.windows.net).
.. 20.60.40.164
Connecting to gethstore.blob.core.windows.net (gethstore.blob.core.windows.n
et)|20.60.40.164|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 26856444 (26M) [application/octet-stream]
Saving to: 'geth-linux-amd64-1.12.0-e501b3b0.tar.gz'

geth-linux-amd64-1 100%[=====>] 25.61M 734KB/s in 27s

2026-03-26 03:54:32 (968 KB/s) - 'geth-linux-amd64-1.12.0-e501b3b0.tar.gz' s
aved [26856444/26856444]

deilsy@deilsy:/usr/local/bin$ ls
geth-linux-amd64-1.12.0-e501b3b0.tar.gz
deilsy@deilsy:/usr/local/bin$ █

```

3. Extract the tar file and move the geth into /usr/local/bin/geth path & check the geth version

```

sudo tar -xvf geth-linux-amd64-1.12.0-e501b3b0.tar.gz
sudo mv geth-linux-amd64-1.12.0-e501b3b0/geth /usr/local/bin/geth
geth version

```

```

deilsy@deilsy:/usr/local/bin$ sudo mv geth-linux-amd64-1.12.0-e501b3b0/geth /
usr/local/bin/geth
deilsy@deilsy:/usr/local/bin$ geth version
Geth
Version: 1.12.0-stable
Git Commit: e501b3b05db8e169f67dc78b7b59bc352b3c638d
Git Commit Date: 20230525
Architecture: amd64
Go Version: go1.20.3
Operating System: linux
GOPATH=
GOROOT=
deilsy@deilsy:/usr/local/bin$ █

```

4. Create a folder named private_ethereum_setup and create another 2 directories naming node1 and node2 individually inside private_ethereum_setup directory.

```

mkdir private_ethereum_setup
cd private_ethereum_setup && mkdir node1 node2

```

```

deilsy@deilsy:~$ ls private_ethereum_setup/
node1  node2
deilsy@deilsy:~$ █

```

5. Check the files

6. Create account for node1

```
geth --datadir "node1" account new
```

```
deilsy@deilsy:~/private_ethereum_setup$ geth --datadir "node1" account new
INFO [03-26|04:01:54.815] Maximum peer count          ETH=50 LE
S=0 total=50
INFO [03-26|04:01:54.816] Smartcard socket not found, disabling err="stat
/run/pcscd/pcscd.comm: no such file or directory"
Your new account is locked with a password. Please give a password. Do not f
orget this password.
Password:
Repeat password:

Your new key was generated

Public address of the key: 0x563A5a670260417B163AD1696fCBfA6849906d06
Path of the secret key file: node1/keystore/UTC--2026-03-26T04-02-01.7653580
44Z--563a5a670260417b163ad1696fcbfa6849906d06

- You can share your public address with anyone. Others need it to interact
with you.
- You must NEVER share the secret key with anyone! The key controls access t
o your funds!
- You must BACKUP your key file! Without the key, it's impossible to access
account funds!
- You must REMEMBER your password! Without the password, it's impossible to
decrypt the key!

deilsy@deilsy:~/private_ethereum_setup$ █
```

7. Do the same for node2

```
geth --datadir "node2" account new
```

```
deilsy@deilsy:~/private_ethereum_setup$ geth --datadir "node2" account new
INFO [03-26|04:02:43.813] Maximum peer count          ETH=50 LE
S=0 total=50
INFO [03-26|04:02:43.813] Smartcard socket not found, disabling err="stat
/run/pcscd/pcscd.comm: no such file or directory"
Your new account is locked with a password. Please give a password. Do not f
orget this password.
Password:
Repeat password:

Your new key was generated

Public address of the key: 0x6d04eA66725A40F5669E2d6f4f77eb4Ecf756a3f
Path of the secret key file: node2/keystore/UTC--2026-03-26T04-02-49.2597870
89Z--6d04ea66725a40f5669e2d6f4f77eb4ecf756a3f

- You can share your public address with anyone. Others need it to interact
with you.
- You must NEVER share the secret key with anyone! The key controls access t
o your funds!
- You must BACKUP your key file! Without the key, it's impossible to access
account funds!
- You must REMEMBER your password! Without the password, it's impossible to
decrypt the key!
```

8. Create a file called genesis.json in the same directory and copy paste these contents

```
{
```

```

"config": {
  "chainId": 12345,
  "homesteadBlock": 0,
  "eip150Block": 0,
  "eip155Block": 0,
  "eip158Block": 0,
  "byzantiumBlock": 0,
  "constantinopleBlock": 0,
  "petersburgBlock": 0,
  "istanbulBlock": 0,
  "muirGlacierBlock": 0,
  "berlinBlock": 0,
  "londonBlock": 0,
  "arrowGlacierBlock": 0,
  "grayGlacierBlock": 0,
  "clique": {
    "period": 5,
    "epoch": 30000
  }
},
"difficulty": "1",
"gasLimit": "8000000000",
"extradata":
"0x0000000000000000000000000000000000000000000000000000000000000000
0000000098608ADf9c785d54f40cDcf6700E990771b1922600000000000000
0000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000
0",
"alloc": {
  "0x98608ADf9c785d54f40cDcf6700E990771b19226": {
"balance": "1000000000000000000000000" },
  "0x7B25e791D24A3F5c453A9E5468cF6cEa2243092C": {
"balance": "12000000000000000000000000" }
}
}

```

Make changes in the alloc part replace the public keys of the account that you have generated in earlier steps.

```

{
  "config": {
    "chainId": 12345,
    "homesteadBlock": 0,
    "eip150Block": 0,
    "eip155Block": 0,
    "eip158Block": 0,
    "byzantiumBlock": 0,
    "constantinopleBlock": 0,
    "petersburgBlock": 0,
    "istanbulBlock": 0,
    "muirGlacierBlock": 0,
    "berlinBlock": 0,
    "londonBlock": 0,
    "arrowGlacierBlock": 0,
    "grayGlacierBlock": 0,
    "clique": {
      "period": 5,
      "epoch": 30000
    }
  },
  "difficulty": "1",
  "gasLimit": "8000000000",
  "extradata": "0x0000000000000000000000000000000000000000000000000000000000000000",
  "alloc": {
    "0x563A5a670260417B163AD1696fCBfA6849906d06": { "balance": "10000000000",
    "0x6d04eA66725A40F5669E2d6f4f77eb4Ec756a3f": { "balance": "12000000000"
  }
}

```

9. Initializing geth for node1 using the genesis.json file
 geth init --datadir node1 genesis.json

```

deilsy@deilsy:~/private_ethereum_setup$ geth init --datadir node1 genesis.js
n
INFO [03-26|04:06:53.763] Maximum peer count                      ETH=50 LE
S=0 total=50
INFO [03-26|04:06:53.764] Smartcard socket not found, disabling   err="stat
/run/pcscd/pcscd.comm: no such file or directory"
INFO [03-26|04:06:53.767] Set global gas cap                      cap=50,00
0,000
INFO [03-26|04:06:53.768] Initializing the KZG library            backend=g
okzg
INFO [03-26|04:06:53.810] Defaulting to pebble as the backing database
INFO [03-26|04:06:53.810] Allocated cache and file handles       database=
/home/deilsy/private_ethereum_setup/node1/geth/chaindata cache=16.00MiB hand
les=16
INFO [03-26|04:06:53.957] Opened ancient database                 database=
/home/deilsy/private_ethereum_setup/node1/geth/chaindata/ancient/chain reado
nly=false
INFO [03-26|04:06:53.957] Writing custom genesis block
INFO [03-26|04:06:53.958] Persisted trie from memory database    nodes=3 s
ize=413.00B time="100.853µs" gcnodes=0 gcsiz=0.00B gctime=0s livenodes=0 li
vesize=0.00B
INFO [03-26|04:06:53.976] Successfully wrote genesis state        database=
chaindata hash=732ee4..dabc1f
INFO [03-26|04:06:53.976] Defaulting to pebble as the backing database
INFO [03-26|04:06:53.976] Allocated cache and file handles       database=
/home/deilsy/private_ethereum_setup/node1/geth/lightchaindata cache=16.00MiB
handles=16
INFO [03-26|04:06:54.074] Opened ancient database                 database=
/home/deilsy/private_ethereum_setup/node1/geth/lightchaindata/ancient/chain
readonly=false
INFO [03-26|04:06:54.074] Writing custom genesis block
INFO [03-26|04:06:54.074] Persisted trie from memory database    nodes=3 s
ize=413.00B time="18.276µs" gcnodes=0 gcsiz=0.00B gctime=0s livenodes=0 li
vesize=0.00B
INFO [03-26|04:06:54.086] Successfully wrote genesis state        database=
lightchaindata hash=732ee4..dabc1f
deilsy@deilsy:~/private_ethereum_setup$ █

```

10. Do the same for node2

```
geth init --datadir node2 genesis.json
```



```

deilsy@deilsy:~/private_ethereum_setup$ geth init --datadir node2 genesis.js
n
INFO [03-26|04:07:16.531] Maximum peer count                      ETH=50 LE
S=0 total=50
INFO [03-26|04:07:16.531] Smartcard socket not found, disabling  err="stat
/run/pcscd/pcscd.comm: no such file or directory"
INFO [03-26|04:07:16.534] Set global gas cap                      cap=50,00
0,000
INFO [03-26|04:07:16.534] Initializing the KZG library            backend=g
okzg
INFO [03-26|04:07:16.580] Defaulting to pebble as the backing database
INFO [03-26|04:07:16.580] Allocated cache and file handles       database=
/home/deilsy/private_ethereum_setup/node2/geth/chaindata cache=16.00MiB hand
les=16
INFO [03-26|04:07:16.713] Opened ancient database                 database=
/home/deilsy/private_ethereum_setup/node2/geth/chaindata/ancient/chain reado
nly=false
INFO [03-26|04:07:16.713] Writing custom genesis block
INFO [03-26|04:07:16.713] Persisted trie from memory database    nodes=3 s
ize=413.00B time="23.604µs" gcnodes=0 gcsiz=0.00B gctime=0s livenodes=0 liv
esize=0.00B
INFO [03-26|04:07:16.724] Successfully wrote genesis state        database=
chaindata hash=732ee4..dabc1f
INFO [03-26|04:07:16.724] Defaulting to pebble as the backing database
INFO [03-26|04:07:16.724] Allocated cache and file handles       database=
/home/deilsy/private_ethereum_setup/node2/geth/lightchaindata cache=16.00MiB
handles=16
INFO [03-26|04:07:16.799] Opened ancient database                 database=
/home/deilsy/private_ethereum_setup/node2/geth/lightchaindata/ancient/chain
readonly=false
INFO [03-26|04:07:16.800] Writing custom genesis block
INFO [03-26|04:07:16.800] Persisted trie from memory database    nodes=3 s
ize=413.00B time="29.651µs" gcnodes=0 gcsiz=0.00B gctime=0s livenodes=0 liv
esize=0.00B
INFO [03-26|04:07:16.814] Successfully wrote genesis state        database=
lightchaindata hash=732ee4..dabc1f

```

11. Now we will install bootnode for configuring both the nodes.

```

sudo add-apt-repository -y ppa:ethereum/ethereum
sudo apt-get update
sudo apt-get install bootnode

```

12. Create a key for the bootnode and save it to boot.key in the folder

```
bootnode -genkey boot.key
```

```

deilsy@deilsy:~/private_ethereum_setup$ bootnode -genkey boot.key
deilsy@deilsy:~/private_ethereum_setup$ ls
boot.key  genesis.json  node1  node2
deilsy@deilsy:~/private_ethereum_setup$

```

13. Copy this command and copy the enode://.... This will be required later

```
bootnode -nodekey boot.key -verbosity 9 -addr :30305
```

```
deilsy@delsy:~/private_ethereum_setup$ bootnode -nodekey boot.key -verbosity
9 -addr :30305
enode://3f690f88c00ce563d9dd609b0674ad97df9fc6b77673bd86917f51333118e81f54e7
5919746a331d55af7f6edba870a5ef5ed12b60af554ccb922bc77ab41f27@127.0.0.1:0?dis
cport=30305
Note: you're using cmd/bootnode, a developer tool.
We recommend using a regular node as bootstrap node for production deploymen
ts.
INFO [03-26|04:20:52.969] New local node record                      seq=1,774
,498,852,968 id=7212962da899bfc0 ip="invalid IP" udp=0 tcp=0
```

14. Now on another terminal we will start mining node1

```
geth --datadir node1 --port 30306 --bootnodes
enode://3f690f88c00ce563d9dd609b0674ad97df9fc6b77673bd86917f5133
3118e81f54e75919746a331d55af7f6edba870a5ef5ed12b60af554ccb922bc7
7ab41f27@127.0.0.1:0?discport=30305 --networkid 123454321
--unlock 0x563A5a670260417B163AD1696fCBfA6849906d06 --password
node1/password.txt --authrpc.port 8551 --miner.etherbase
0x563A5a670260417B163AD1696fCBfA6849906d06 --mine
```

Replace the address with your own created

```

deilsy@delsy:~/private_ethereum_setup$ geth --datadir node1 --port 30306 --bootnodes enode://3f690f88c00ce563d9dd609b0674ad97df9fc6b77673bd86917f51333118e81f54e75919746a331d55af7f6edba870a5ef5ed12b60af554ccb922bc77ab41f27@127.0.0.1:0?discport=30305 --networkid 123454321 --unlock 0x563A5a670260417B163AD1696fCBfA6849906d06 --password node1/password.txt --authrpc.port 8551 --miner .etherbase 0x563A5a670260417B163AD1696fCBfA6849906d06 --mine
INFO [03-26|04:28:57.471] Maximum peer count                      ETH=50 LE
S=0 total=50
INFO [03-26|04:28:57.472] Smartcard socket not found, disabling   err="stat
/run/pcscd/pcscd.comm: no such file or directory"
INFO [03-26|04:28:57.474] Set global gas cap                      cap=50,00
0,000
INFO [03-26|04:28:57.474] Initializing the KZG library            backend=g
okzg
INFO [03-26|04:28:57.529] Allocated trie memory caches           clean=154
.00MiB dirty=256.00MiB
INFO [03-26|04:28:57.529] Using pebble as the backing database
INFO [03-26|04:28:57.530] Allocated cache and file handles       database=
/home/deilsy/private_ethereum_setup/node1/geth/chaindata cache=512.00MiB han
dles=524,288
INFO [03-26|04:28:57.629] Opened ancient database                database=
/home/deilsy/private_ethereum_setup/node1/geth/chaindata/ancient/chain reado
nly=false
INFO [03-26|04:28:57.641] Initialising Ethereum protocol         network=1
23,454,321 dbversion=8
INFO [03-26|04:28:57.641]
INFO [03-26|04:28:57.641] -----
-----
INFO [03-26|04:28:57.641] Chain ID: 12345 (unknown)
INFO [03-26|04:28:57.641] Consensus: Clique (proof-of-authority)

```

15. Now do same for node2

```

geth --datadir node2 --port 30307 --bootnodes
enode://3f690f88c00ce563d9dd609b0674ad97df9fc6b77673bd86917
f51333118e81f54e75919746a331d55af7f6edba870a5ef5ed12b60af55
4ccb922bc77ab41f27@127.0.0.1:0?discport=30305 --networkid
123454321 --unlock
0x6d04eA66725A40F5669E2d6f4f77eb4Ecf756a3f --password
node2/password.txt --authrpc.port 8552

```

```

deilsy@deilsy:~/private_ethereum_setup$ geth --datadir node2 --port 30307 --b
ootnodes enode://3f690f88c00ce563d9dd609b0674ad97df9fc6b77673bd86917f5133311
8e81f54e75919746a331d55af7f6edba870a5ef5ed12b60af554ccb922bc77ab41f27@127.0.
0.1:0?discport=30305 --networkid 123454321 --unlock 0x6d04eA66725A40F5669E2d
6f4f77eb4Ecf756a3f --password node2/password.txt --authrpc.port 8552
INFO [03-26|04:35:17.744] Maximum peer count          ETH=50 LE
S=0 total=50
INFO [03-26|04:35:17.745] Smartcard socket not found, disabling err="stat
/run/pcscd/pcscd.comm: no such file or directory"
INFO [03-26|04:35:17.748] Set global gas cap          cap=50,00
0,000
INFO [03-26|04:35:17.748] Initializing the KZG library backend=g
okzg
INFO [03-26|04:35:17.795] Allocated trie memory caches clean=154
.00MiB dirty=256.00MiB
INFO [03-26|04:35:17.795] Using pebble as the backing database
INFO [03-26|04:35:17.795] Allocated cache and file handles database=
/home/deilsy/private_ethereum_setup/node2/geth/chaindata cache=512.00MiB han
dles=524,288
INFO [03-26|04:35:17.928] Opened ancient database      database=
/home/deilsy/private_ethereum_setup/node2/geth/chaindata/ancient/chain reado
nly=false
INFO [03-26|04:35:17.929] Initialising Ethereum protocol network=1
23,454,321 dbversion=<nil>
INFO [03-26|04:35:17.930]
INFO [03-26|04:35:17.930] -----
-----
INFO [03-26|04:35:17.930] Chain ID: 12345 (unknown)
INFO [03-26|04:35:17.930] Consensus: Clique (proof-of-authority)
INFO [03-26|04:35:17.930]
INFO [03-26|04:35:17.930] Pre-Merge hard forks (block based):
INFO [03-26|04:35:17.930] - Homestead: #0 (https:/
/github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-up
grades/homestead.md)

```

16. Now we will interact with the blockchain using javascript console

```
geth attach node1/geth.ipc
```

```

deilsy@deilsy:~/private_ethereum_setup$ geth attach node1/geth.ipc
Welcome to the Geth JavaScript console!

instance: Geth/v1.12.0-stable-e501b3b0/linux-amd64/go1.20.3
coinbase: 0x563a5a670260417b163ad1696fcbfa6849906d06
at block: 105 (Thu Mar 26 2026 04:37:38 GMT+0000 (UTC))
  datadir: /home/deilsy/private_ethereum_setup/node1
  modules: admin:1.0 clique:1.0 debug:1.0 engine:1.0 eth:1.0 miner:1.0 net:1.
0 rpc:1.0 txpool:1.0 web3:1.0

To exit, press ctrl-d or type exit
> █

```

17. Interacting with the blockchain

```
> net.peerCount
1
> eth.accounts
["0x563a5a670260417b163ad1696fcbfa6849906d06"]
> eth.accounts[0]
"0x563a5a670260417b163ad1696fcbfa6849906d06"
> eth.blockNumber
302
> eth.getBalance(eth.accounts[0])
1e+23
> web3.fromWei(eth.getBalance("0x563A5a670260417B163AD1696fCBfA6849906d06"),
"ether")
100000
```



```
> web3.fromWei(eth.getBalance("0xd04eA66725A40F5669E2d6f4f77eb4Ec756a3f"),
"ether")
120010
> web3.eth.getTransaction("0x3cb6cc9cd65c927b0a0af8351e757ef8fdf76ea00d66b88
63b3c9fdcf3657508")
{
  accessList: [],
  blockHash: "0xe772e9613d2f39454f41076dd9e740948bac2e0ecfa254702b4f474fbecc
a438",
  blockNumber: 338,
  chainId: "0x3039",
  from: "0x563a5a670260417b163ad1696fcbfa6849906d06",
  gas: 30000,
  gasPrice: 1000000007,
  hash: "0x3cb6cc9cd65c927b0a0af8351e757ef8fdf76ea00d66b8863b3c9fdcf3657508"
,
  input: "0x",
  maxFeePerGas: 1000000014,
  maxPriorityFeePerGas: 1000000000,
  nonce: 0,
  r: "0x84be672e734bdae2f6d262abda2a08687028a86e035d8c73c8c80662fd6c64fe",
  s: "0x196df54d8f571bde60289c94926793a29f05947203c25418295a1cd80c7f9c6a",
  to: "0xd04eA66725A40F5669E2d6f4f77eb4Ec756a3f",
  transactionIndex: 0,
  type: "0x2",
  v: "0x0",
  value: 100000000000000000000
}
```

```
> web3.fromWei(eth.getBalance("0xd04eA66725A40F5669E2d6f4f77eb4EcF756a3f"),
"ether")
120010
> web3.eth.getTransaction("0x3cb6cc9cd65c927b0a0af8351e757ef8fdf76ea00d66b88
63b3c9fdcf3657508")
{
  accessList: [],
  blockHash: "0xe772e9613d2f39454f41076dd9e740948bac2e0ecfa254702b4f474fbecc
a438",
  blockNumber: 338,
  chainId: "0x3039",
  from: "0x563a5a670260417b163ad1696fcbfa6849906d06",
  gas: 30000,
  gasPrice: 1000000007,
  hash: "0x3cb6cc9cd65c927b0a0af8351e757ef8fdf76ea00d66b8863b3c9fdcf3657508"
,
  input: "0x",
  maxFeePerGas: 1000000014,
  maxPriorityFeePerGas: 1000000000,
  nonce: 0,
  r: "0x84be672e734bdae2f6d262abda2a08687028a86e035d8c73c8c80662fd6c64fe",
  s: "0x196df54d8f571bde60289c94926793a29f05947203c25418295a1cd80c7f9c6a",
  to: "0xd04eA66725A40F5669E2d6f4f77eb4EcF756a3f",
  transactionIndex: 0,
  type: "0x2",
  v: "0x0",
  value: 100000000000000000000
}
```



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment 10

Aim: Explore the Cryptocurrency Landscape

Roll No.	04
Name	Mohit Patil
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1:Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO5:Interpret different Crypto assets and Crypto currencies CO6:Analyze the use of Blockchain with AI, IoT and Cyber Security using case studies.

EXPERIMENT 10

Aim: Explore the Cryptocurrency Landscape

Theory:

Cryptocurrency Landscape – Theory

Cryptocurrencies are decentralized digital assets that operate on blockchain technology, enabling secure, transparent, and immutable transactions without the need for a central authority. These systems rely on distributed ledgers where transactions are validated through consensus mechanisms and recorded across a network of nodes.

Data Sources

Reliable and diverse data sources are essential for understanding cryptocurrency markets:

- **Financial Data Platforms** – Services like CoinGecko and CoinMarketCap provide structured data such as market capitalization, circulating supply, trading volume, and historical price trends.
- **News Aggregators** – Platforms such as CryptoPanic collect real-time news from multiple sources, helping track major events impacting the market.
- **Social Sentiment Analytics** – Tools like LunarCrush analyze social media activity to measure public sentiment and engagement.
- **Regulatory Sources** – Official portals like the U.S. Securities and Exchange Commission and frameworks such as MiCA provide insights into legal policies and compliance requirements.

Types of Cryptocurrencies

The cryptocurrency ecosystem consists of various categories based on functionality:

- **Store of Value (SoV)** – Bitcoin (BTC), known for limited supply and long-term value preservation.
- **Utility Tokens** – Ether (ETH) and Chainlink (LINK), used for network operations and smart contract execution.
- **Stablecoins** – Tether (USDT) and USD Coin (USDC), designed to maintain price stability by being pegged to fiat currencies.
- **Governance Tokens** – Uniswap (UNI) and Aave (AAVE), enabling users to participate in decentralized decision-making.

Key Characteristics

Cryptocurrencies exhibit the following features:

- **Decentralization** – No central authority controls the system.

- **Transparency** – Transactions are publicly recorded on the blockchain.
- **Immutability** – Data cannot be altered once confirmed.
- **Security** – Cryptographic algorithms ensure data integrity and safety.

Market Dynamics

The cryptocurrency market is influenced by multiple factors:

- **Volatility** – Prices fluctuate significantly due to demand, sentiment, and external events.
- **Liquidity** – Depends on trading volume and market participation.
- **Market Sentiment** – News, social media, and investor behavior impact price trends.
- **Regulation** – Government policies and frameworks influence adoption and stability.

Blockchain Technology Advancements

Modern blockchain systems are evolving to overcome limitations:

- **Consensus Mechanisms** – Shift from Proof of Work (PoW) to Proof of Stake (PoS) for efficiency.
- **Scalability Solutions** – Layer-2 technologies and sharding improve transaction speed.
- **Interoperability** – Enables communication between different blockchain networks.
- **Privacy Enhancements** – Zero-Knowledge Proofs (ZKP) allow secure verification without revealing sensitive data.

Adoption and Emerging Trends

Cryptocurrency adoption continues to grow across sectors:

- **Digital Payments** – Companies like Stripe and Shopify support crypto transactions.
- **Decentralized Finance (DeFi)** – Financial services operate without intermediaries.
- **Non-Custodial Wallets** – MetaMask and Phantom give users full asset control.
- **Asset Tokenization** – Real-world assets are increasingly represented on blockchain networks.

Utilizing Api for analysis and visualization :

Code:

```
import requests
import matplotlib.pyplot as plt
import pandas as pd
import datetime

# CONFIGURATION
# Replace 'YOUR_API_KEY' with the key you got from CryptoCompare

API_KEY = 'YOUR_API_KEY'
SYMBOL = 'LINK'
COMPARISON_SYMBOL = 'USD'
LIMIT = 180 # Number of days

# ----- Fetch Data Function -----
def fetch_cryptocompare_data(symbol, comparison_symbol, limit, api_key):
    url = f"https://min-api.cryptocompare.com/data/v2/histoday"
    params = {
        "fsym": symbol,
        "tsym": comparison_symbol,
        "limit": limit,
        "api_key": api_key
    }
    response = requests.get(url, params=params)
    data = response.json()
    if data['Response'] == 'Success':
        # CryptoCompare returns OHLCV data in 'Data' -> 'Data' return data['Data']['Data']
    else:
        print(f"Error fetching data: {data.get('Message')}")
        return None

# ----- Dashboard Function -----
def cryptocompare_dashboard(symbol, comparison_symbol, limit, api_key):
    raw_data = fetch_cryptocompare_data(symbol, comparison_symbol, limit, api_key)
    if not raw_data:
        return

    # Create DataFrame
    df = pd.DataFrame(raw_data)

    # Convert 'time' (unix timestamp) to datetime
    df['timestamp'] = pd.to_datetime(df['time'], unit='s')
    df.set_index('timestamp', inplace=True)
```

```
# Standardize column names to match your previous logic # 'close' is used as the daily price
```

```
df['price'] = df['close']
```

```
df['volume'] = df['volumeto'] # Total volume in USD (volumeto)
```

```
# Calculate metrics
```

```
df['returns'] = df['price'].pct_change() df['MA7'] = df['price'].rolling(window=7).mean()
```

```
df['MA30'] = df['price'].rolling(window=30).mean() df['cumulative'] = (1 +  
df['returns']).cumprod() df['volatility'] = df['returns'].rolling(window=7).std() * 100
```

```
# Plotting
```

```
fig, axes = plt.subplots(3, 2, figsize=(16, 12)) fig.suptitle(f'CryptoCompare: {symbol} Analysis  
Dashboard',
```

```
fontsize=16, fontweight='bold')
```

```
# 1. Price Trend
```

```
axes[0, 0].plot(df.index, df['price'], color='tab:blue', linewidth=2) axes[0, 0].set_title('Historical  
Price Trend (USD)')
```

```
axes[0, 0].grid(True, linestyle='--', alpha=0.7)
```

```
# 2. Price vs Volume
```

```
axes[0, 1].plot(df.index, df['price'], color='tab:blue', label='Price')
```

```
axes[0, 1].bar(df.index, df['volume'], color='gray', alpha=0.3, label='Volume (USD)')
```

```
axes[0, 1].set_title('Price vs Trading Volume') axes[0, 1].legend()
```

```
# 3. Daily Returns Distribution
```

```
axes[1, 0].hist(df['returns'].dropna(), bins=40, color='tab:purple', edgecolor='white')
```

```
axes[1, 0].set_title('Daily Returns Distribution')
```

```
# 4. Moving Averages
```

```
axes[1, 1].plot(df.index, df['price'], color='lightgray', alpha=0.5, label='Price')
```

```
axes[1, 1].plot(df.index, df['MA7'], color='tab:orange', label='7-Day MA')
```

```
axes[1, 1].plot(df.index, df['MA30'], color='tab:red', label='30-Day MA')
```

```
axes[1, 1].set_title('Trend Analysis (Moving Averages)') axes[1, 1].legend()
```

```
# 5. Cumulative Returns
```

```
axes[2, 0].fill_between(df.index, df['cumulative'], color='tab:green', alpha=0.2)
```

```
axes[2, 0].plot(df.index, df['cumulative'], color='tab:green') axes[2, 0].set_title('Cumulative  
Growth')
```

6. Volatility

```
axes[2, 1].plot(df.index, df['volatility'], color='tab:red') axes[2, 1].set_title('7-Day Rolling  
Volatility (%)')
```

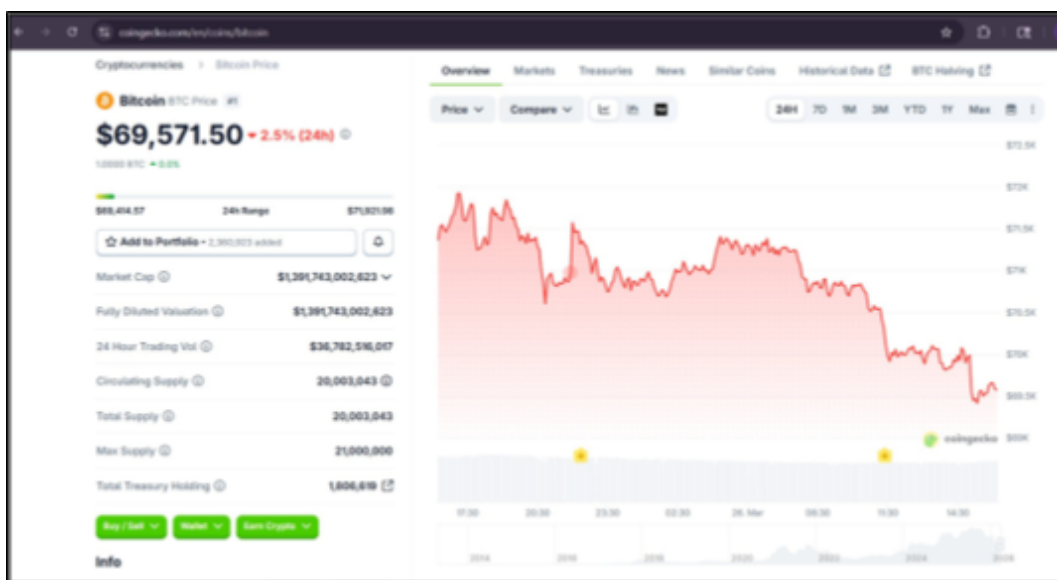
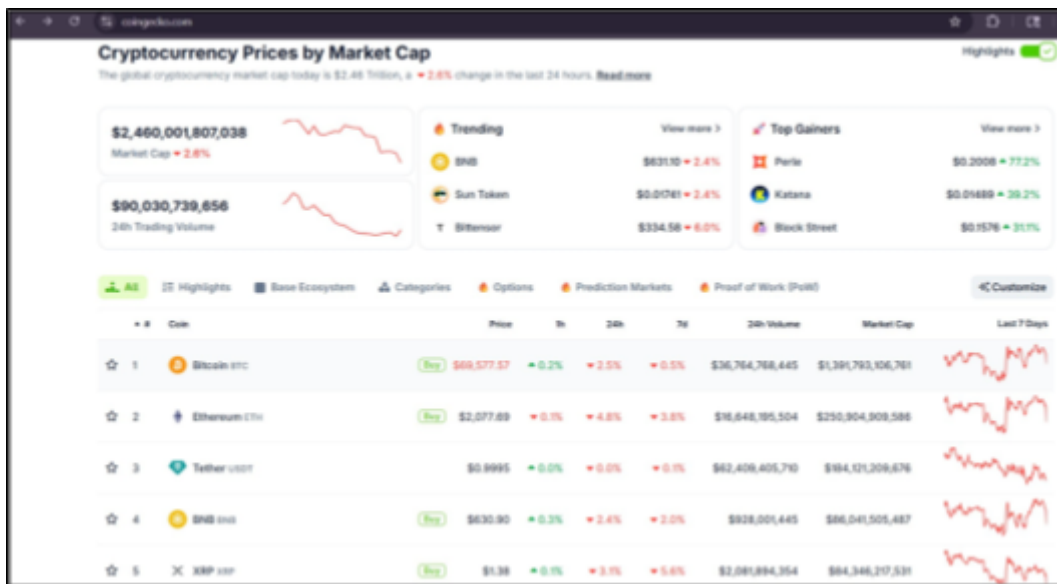
```
plt.tight_layout(rect=[0, 0.03, 1, 0.95]) plt.show()
```

Run

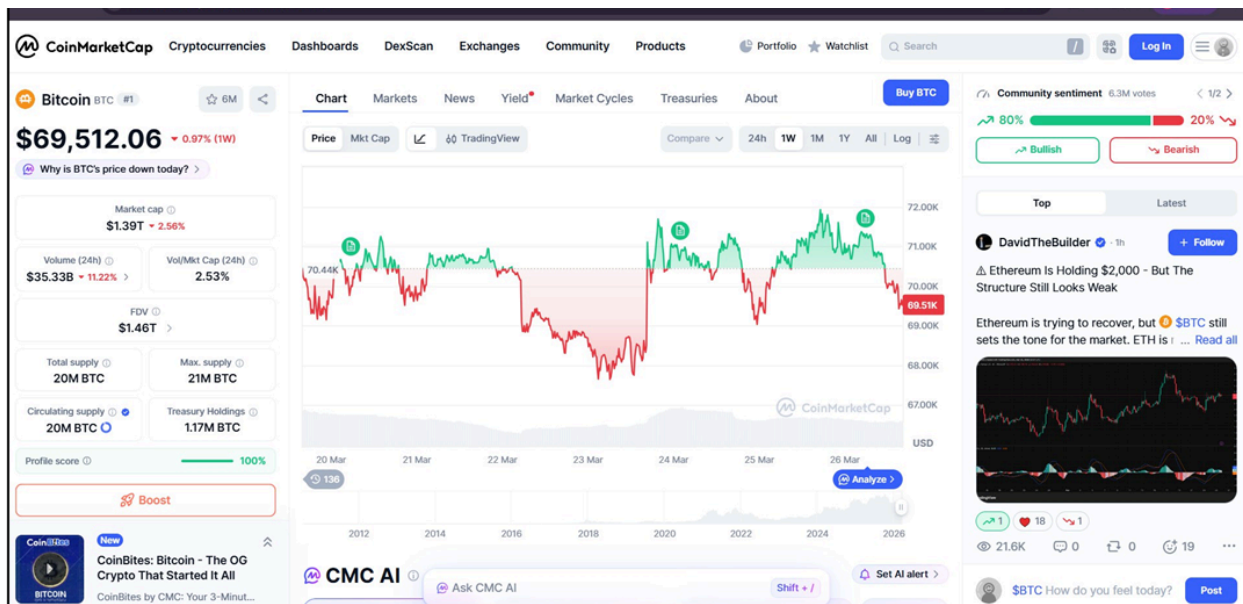
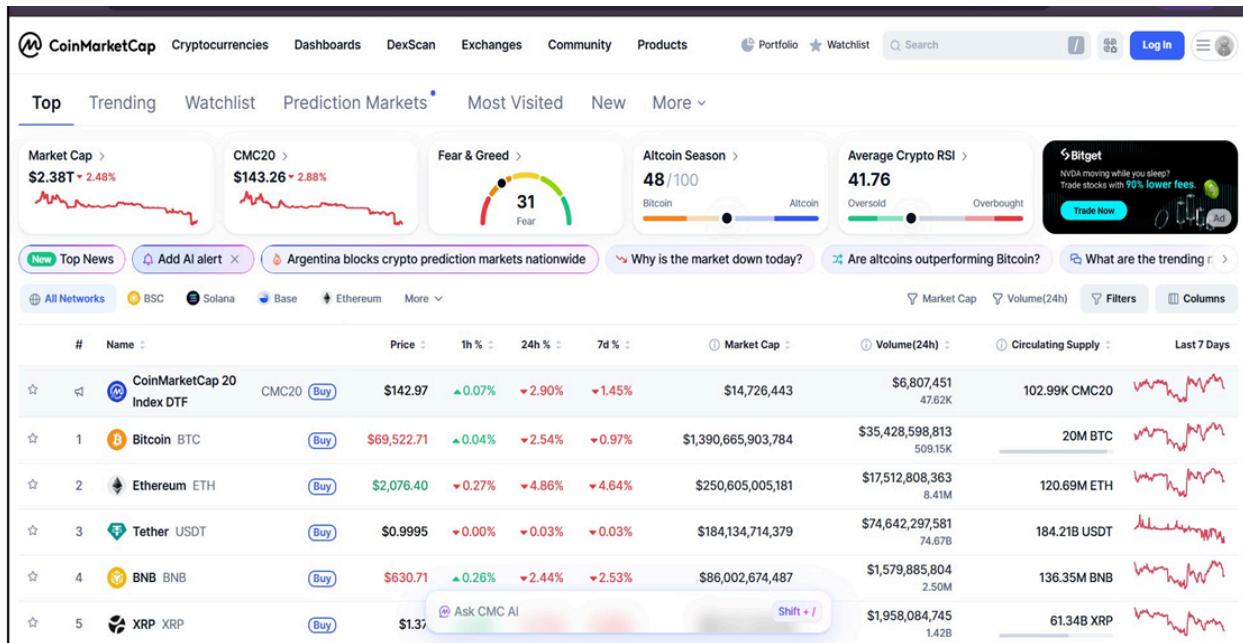
cryptocompare_dashboard(SYMBOL, COMPARISON_SYMBOL, LIMIT, API_KEY)

Output:-

Coingecko Website

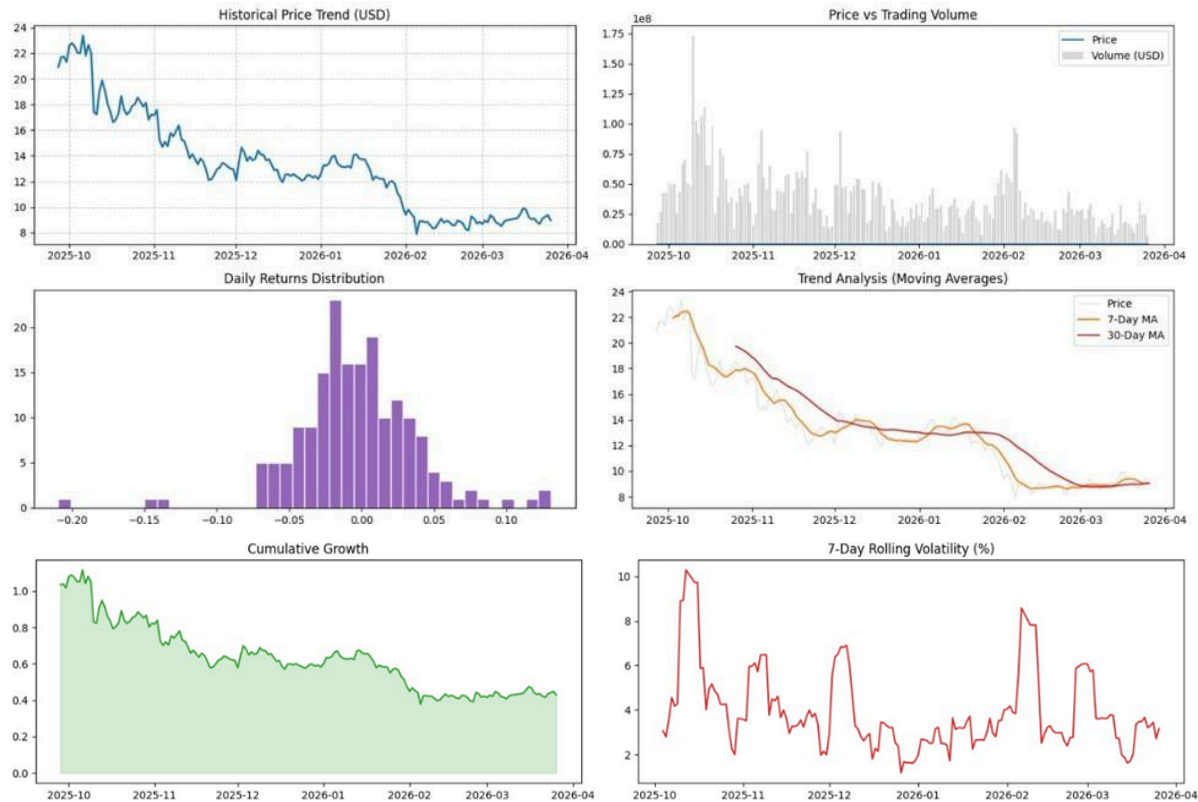


CoinMarketCap website



Dashboard

CryptoCompare: LINK Analysis Dashboard



Conclusion:-

By integrating automated market data processing, visual trend analysis, blockchain technology evaluation, and adoption metrics, a comprehensive view of the cryptocurrency ecosystem can be achieved. This approach highlights the relationship between blockchain's underlying technical foundations and real-world market behavior, offering valuable insights into current trends as well as potential future developments in the digital asset industry.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment 11

Aim: Implementation of Hyperledger Fabric Network and Asset Management using Chaincode

Roll No.	04
Name	Mohit Patil
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO4:Develop applications in permissioned Hyperledger Fabric network.

EXPERIMENT 11

Aim - To implement a Hyperledger Fabric network, deploy chaincode, and perform asset creation, querying, and transfer operations.

Theory -

Hyperledger Fabric Network Setup

- Consortium & Membership Service Provider (MSP): Fabric is permissioned, meaning only authorized organizations can join. MSPs define identities and roles.
- Peers: Nodes that maintain the ledger and endorse transactions. Each organization typically runs peers.
- Orderer: The ordering service packages endorsed transactions into blocks and delivers them to peers.
- Ledger: Composed of a blockchain (immutable transaction log) and a state database (current values).
- Channels: Private sub-networks where a subset of organizations transact confidentially.

Chaincode (Smart Contracts)

- Chaincode defines the business logic for asset management.
- Written in Go, Java, or Node.js.
- Deployed to peers, then instantiated on a channel.
- Functions typically include createAsset, queryAsset, transferAsset, and deleteAsset.

Transaction Flow

1. Proposal Creation: A client application submits a transaction proposal (e.g., create asset).
2. Endorsement: Peers simulate the transaction using chaincode and return signed results.
3. Ordering: The orderer sequences endorsed transactions into blocks.
4. Validation & Commit: Peers validate endorsements, check for conflicts, and commit the block to the ledger.

Asset Operations

- Asset Creation:
 - Chaincode function createAsset stores asset attributes (ID, owner, value) in the ledger's state database.
- Asset Querying:
 - queryAsset retrieves asset details by key. Rich queries are possible if CouchDB is used as the state database.
- Asset Transfer:
 - transferAsset updates the owner field of an asset. The transaction is endorsed, ordered, and committed, ensuring immutability and consensus.

Key Principles

- Immutability: Once committed, transactions cannot be altered.

- Consensus: Endorsement policies ensure multiple organizations agree before a transaction is valid.
- Privacy: Channels and private data collections allow selective visibility.
- Trust: Identities are cryptographically verified via MSPs.

In essence, Hyperledger Fabric provides a modular, permissioned blockchain framework where chaincode governs asset lifecycles. The theory emphasizes identity-driven trust, modular architecture, and consensus-based validation — ensuring secure creation, querying, and transfer of assets across organizations.

Prerequisite: Check if WSL is Installed

Open **PowerShell (Run as Administrator)** and run:

wsl --list --verbose

If it shows **no distributions**, you must install Ubuntu.

Install Ubuntu for WSL

Run this command in **PowerShell (Admin)**:

wsl --install -d Ubuntu

This installs:

- Ubuntu
- WSL2 kernel

The installation may take a few minutes.

Restart Your Computer

After installation finishes, **restart your PC**.

Open Ubuntu

Open:

Start Menu → Ubuntu

It will ask you to
create:

Username

Password

Now Ubuntu is ready.

Implementation

Step 1 — Update the system

code :

sudo apt update

sudo apt upgrade -y

Explain: `apt update` refreshes the package index — it tells your system what versions of software are available. `apt upgrade` installs the newer versions. Fabric's dependencies (Docker, curl, git) can fail or behave unexpectedly on outdated system libraries. Always do this first on a fresh machine.

```
pranav@DESKTOP-OSHSLJ1:~$ sudo apt update
sudo apt upgrade -y
[sudo] password for pranav:
Hit:1 http://archive.ubuntu.com/ubuntu noble InRelease
Get:2 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:3 http://archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:4 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages [1540 kB]
Get:5 http://archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:6 http://archive.ubuntu.com/ubuntu noble/universe amd64 Packages [15.0 MB]
Get:7 http://security.ubuntu.com/ubuntu noble-security/main Translation-en [246 kB]
Get:8 http://security.ubuntu.com/ubuntu noble-security/main amd64 Components [21.6 kB]
Get:9 http://security.ubuntu.com/ubuntu noble-security/main amd64 c-n-f Metadata [18.6 kB]
Get:10 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Packages [976 kB]
Get:11 http://security.ubuntu.com/ubuntu noble-security/universe Translation-en [219 kB]
Get:12 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Components [74.2 kB]
Get:13 http://security.ubuntu.com/ubuntu noble-security/universe amd64 c-n-f Metadata [20.8 kB]
Get:14 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Packages [2683 kB]
Get:15 http://security.ubuntu.com/ubuntu noble-security/restricted Translation-en [623 kB]
Get:16 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Components [208 B]
Get:17 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 c-n-f Metadata [544 B]
Get:18 http://security.ubuntu.com/ubuntu noble-security/multiverse amd64 Packages [28.8 kB]
Get:19 http://security.ubuntu.com/ubuntu noble-security/multiverse Translation-en [6732 B]
Get:20 http://security.ubuntu.com/ubuntu noble-security/multiverse amd64 Components [212 B]
Get:21 http://security.ubuntu.com/ubuntu noble-security/multiverse amd64 c-n-f Metadata [396 B]
Get:22 http://archive.ubuntu.com/ubuntu noble/universe Translation-en [5982 kB]
Get:23 http://archive.ubuntu.com/ubuntu noble/universe amd64 Components [3871 kB]
Get:24 http://archive.ubuntu.com/ubuntu noble/universe amd64 c-n-f Metadata [301 kB]
Get:25 http://archive.ubuntu.com/ubuntu noble/multiverse amd64 Packages [269 kB]
Get:26 http://archive.ubuntu.com/ubuntu noble/multiverse Translation-en [118 kB]
Get:27 http://archive.ubuntu.com/ubuntu noble/multiverse amd64 Components [35.0 kB]
Get:28 http://archive.ubuntu.com/ubuntu noble/multiverse amd64 c-n-f Metadata [8328 B]
Get:29 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [1837 kB]
Get:30 http://archive.ubuntu.com/ubuntu noble-updates/main Translation-en [336 kB]
Get:31 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Components [177 kB]
Get:32 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 c-n-f Metadata [16.7 kB]
```

Step 2 — Verify prerequisites

code :

```
sudo apt install jq curl git nodejs npm -y
```

```
node -v
```

```
npm -v
```

```
git --version
```

```
docker --version
```

```
docker ps
```

```
pranav@DESKTOP-OSHSLJ1:~$ node -v
v18.19.1
pranav@DESKTOP-OSHSLJ1:~$ npm -v
11.8.0
pranav@DESKTOP-OSHSLJ1:~$ git --version
git version 2.43.0
pranav@DESKTOP-OSHSLJ1:~$ docker --version
docker ps
Docker version 29.2.0, build 0b9d198
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS        NAMES
```

Explain: Fabric requires Docker (to run peer/orderer containers), Git (to clone the repo), and Node.js + npm (to build JavaScript chaincode). Running `docker ps` with no output confirms the Docker daemon is running and your user can access it. If `docker ps` requires sudo, you need to add your user to the docker group.

Step 3 — Clone fabric-samples

code :

```
git clone https://github.com/hyperledger/fabric-samples
```

```
cd fabric-samples
```

```

pranav@DESKTOP-OSHSLJ1:~$ git clone https://github.com/hyperledger/fabric-samples
Cloning into 'fabric-samples'...
remote: Enumerating objects: 15527, done.
remote: Counting objects: 100% (98/98), done.
remote: Compressing objects: 100% (60/60), done.
remote: Total 15527 (delta 57), reused 38 (delta 38), pack-reused 15429 (from 3)
Receiving objects: 100% (15527/15527), 24.16 MiB | 6.18 MiB/s, done.
Resolving deltas: 100% (8722/8722), done.
pranav@DESKTOP-OSHSLJ1:~$ cd fabric-samples

```

Explain: `fabric-samples` is the official repository containing the test-network scripts, ready-made chaincode examples (including `asset-transfer-basic`), and configuration files. Everything else in this guide runs from inside this directory.

Step 4 — Download the install script and run it
code :

```

curl -sLO https://raw.githubusercontent.com/hyperledger/fabric/main/scripts/install-fabric.sh
chmod +x install-fabric.sh
./install-fabric.sh docker samples binary
./install-fabric.sh docker

```

```

pranav@DESKTOP-OSHSLJ1~/fabric-samples$ curl -sLO https://raw.githubusercontent.com/hyperledger/fabric/main/scripts/install-fabric.sh
pranav@DESKTOP-OSHSLJ1~/fabric-samples$ chmod +x install-fabric.sh
pranav@DESKTOP-OSHSLJ1~/fabric-samples$ ./install-fabric.sh docker samples binary

Clone hyperledger/fabric-samples repo

==> Already in fabric-samples repo
fabric-samples v2.5.15 does not exist, defaulting to main. fabric-samples main branch is intended to work with recent versions of fabric.

Pull Hyperledger Fabric binaries

====> Downloading version 2.5.15 platform specific fabric binaries
====> Downloading: https://github.com/hyperledger/fabric/releases/download/v2.5.15/hyperledger-fabric-linux-amd64-2.5.15.tar.gz
====> Will unpack to: /home/pranav/fabric-samples

```

% Total	% Received	% Xferd	Average Speed	Time Dload	Time Upload	Time Total	Time Spent	Time Left	Current Speed
0	0	0	0	0	0	0	0	0	0
100	100	120M	0	0	6272k	0	0:00:19	0:00:19	6483k

```

==> Done.

```

```

====> List out hyperledger images
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
ghcr.io/hyperledger/fabric-baseos:2.5.15      654bd4554bf9      250MB      80.4MB
ghcr.io/hyperledger/fabric-ca:1.5.17          453a0a727e82      381MB      123MB
ghcr.io/hyperledger/fabric-ccenv:2.5.15       86dded7eda92      1GB        255MB
ghcr.io/hyperledger/fabric-orderer:2.5.15     9972c209be47      178MB      49.5MB
ghcr.io/hyperledger/fabric-peer:2.5.15        3e25adc31a75      230MB      66.2MB
hyperledger/fabric-baseos:2.5                 654bd4554bf9      250MB      80.4MB
hyperledger/fabric-baseos:2.5.15              654bd4554bf9      250MB      80.4MB
hyperledger/fabric-baseos:latest              654bd4554bf9      250MB      80.4MB
hyperledger/fabric-ca:1.5                     453a0a727e82      381MB      123MB
hyperledger/fabric-ca:1.5.17                  453a0a727e82      381MB      123MB
hyperledger/fabric-ca:latest                  453a0a727e82      381MB      123MB
hyperledger/fabric-ccenv:2.5                  86dded7eda92      1GB        255MB
hyperledger/fabric-ccenv:2.5.15               86dded7eda92      1GB        255MB
hyperledger/fabric-ccenv:latest               86dded7eda92      1GB        255MB
hyperledger/fabric-orderer:2.5                9972c209be47      178MB      49.5MB
hyperledger/fabric-orderer:2.5.15            9972c209be47      178MB      49.5MB
hyperledger/fabric-orderer:latest            9972c209be47      178MB      49.5MB
hyperledger/fabric-peer:2.5                   3e25adc31a75      230MB      66.2MB
hyperledger/fabric-peer:2.5.15                3e25adc31a75      230MB      66.2MB
hyperledger/fabric-peer:latest                3e25adc31a75      230MB      66.2MB

```

```

pranav@DESKTOP-OSHSLJI:~/fabric-samples$ ./install-fabric.sh docker

Pull Hyperledger Fabric docker images

FABRIC_IMAGES: peer orderer ccenv baseos
====> Pulling fabric Images
====> ghcr.io/hyperledger/fabric-peer:2.5.15
2.5.15: Pulling from hyperledger/fabric-peer
Digest: sha256:3e25adc31a757ee19dd8a24afdc9ac5aa46a86b3cb56fc93068b0e594706d6aa
Status: Image is up to date for ghcr.io/hyperledger/fabric-peer:2.5.15
ghcr.io/hyperledger/fabric-peer:2.5.15
====> ghcr.io/hyperledger/fabric-orderer:2.5.15
2.5.15: Pulling from hyperledger/fabric-orderer
Digest: sha256:9972c209be47f45e377a24c88f2e3d21fae4dc224751c68bdd33f2e7a603ffa8
Status: Image is up to date for ghcr.io/hyperledger/fabric-orderer:2.5.15
ghcr.io/hyperledger/fabric-orderer:2.5.15
====> ghcr.io/hyperledger/fabric-ccenv:2.5.15
2.5.15: Pulling from hyperledger/fabric-ccenv
Digest: sha256:86dded7eda9268e2cbf3691cb952edf545dc5faeaa1a79bfc9b47ec47d3ecfa9e
Status: Image is up to date for ghcr.io/hyperledger/fabric-ccenv:2.5.15
ghcr.io/hyperledger/fabric-ccenv:2.5.15

```

Explain: This one script does three things simultaneously. `docker` pulls all Fabric Docker images — `fabric-peer`, `fabric-orderer`, `fabric-ccenv`, `fabric-baseos`, `fabric-ca`, and critically `fabric-nodeenv` (needed to build Node.js chaincode). `binary` downloads the CLI tools — `peer`, `configtxgen`, `configtxlator` — into `~/fabric-samples/bin`. `samples` ensures the chaincode sample directories are present. Without `fabric-nodeenv`, chaincode deployment fails later.

Step 5 — Verify Docker images are present

code :

`docker images | grep fabric`

```

pranav@DESKTOP-OSHSLJI:~/fabric-samples$ docker images | grep fabric
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
ghcr.io/hyperledger/fabric-baseos:2.5.15      654bd4554bf9      250MB      80.4MB
ghcr.io/hyperledger/fabric-ca:1.5.17          453a0a727e82      381MB      123MB
ghcr.io/hyperledger/fabric-ccenv:2.5.15       86dded7eda92      1GB        255MB
ghcr.io/hyperledger/fabric-orderer:2.5.15     9972c209be47      178MB      49.5MB   U
ghcr.io/hyperledger/fabric-peer:2.5.15        3e25adc31a75      230MB      66.2MB   U
hyperledger/fabric-baseos:2.5                  654bd4554bf9      250MB      80.4MB
hyperledger/fabric-baseos:2.5.15              654bd4554bf9      250MB      80.4MB
hyperledger/fabric-baseos:latest              654bd4554bf9      250MB      80.4MB
hyperledger/fabric-ca:1.5                     453a0a727e82      381MB      123MB
hyperledger/fabric-ca:1.5.17                  453a0a727e82      381MB      123MB
hyperledger/fabric-ca:latest                  453a0a727e82      381MB      123MB
hyperledger/fabric-ccenv:2.5                  86dded7eda92      1GB        255MB
hyperledger/fabric-ccenv:2.5.15               86dded7eda92      1GB        255MB
hyperledger/fabric-ccenv:latest               86dded7eda92      1GB        255MB
hyperledger/fabric-nodeenv:2.5                17e2d447ca0d      570MB      146MB
hyperledger/fabric-orderer:2.5                9972c209be47      178MB      49.5MB   U
hyperledger/fabric-orderer:2.5.15             9972c209be47      178MB      49.5MB   U
hyperledger/fabric-orderer:latest             9972c209be47      178MB      49.5MB   U
hyperledger/fabric-peer:2.5                   3e25adc31a75      230MB      66.2MB   U
hyperledger/fabric-peer:2.5.15                3e25adc31a75      230MB      66.2MB   U
hyperledger/fabric-peer:latest                3e25adc31a75      230MB      66.2MB   U

```

Explain: Confirm `hyperledger/fabric-nodeenv:2.5` is in the list. This image is the Node.js build environment that Fabric uses to compile and run JavaScript chaincode inside a container. If it is missing, `deployCC` will fail with a "No such image" error.

Step 6 — Set PATH and config path

code :

`export PATH=$PATH:$HOME/fabric-samples/bin`

`export FABRIC_CFG_PATH=$HOME/fabric-samples/config`

```

pranav@DESKTOP-OSHSLSJI:~/fabric-samples$ export PATH=$PATH:$HOME/fabric-samples/bin
pranav@DESKTOP-OSHSLSJI:~/fabric-samples$ export FABRIC_CFG_PATH=$HOME/fabric-samples/config
pranav@DESKTOP-OSHSLSJI:~/fabric-samples$ peer version
peer:
  Version: v2.5.15
  Commit SHA: 83c7930
  Go version: go1.26.0
  OS/Arch: linux/amd64
  Chaincode:
    Base Docker Label: org.hyperledger.fabric
    Docker Namespace: hyperledger

```

Explain: The `peer` binary lives in `~/fabric-samples/bin`, which is not on your shell's PATH by default. Without this export, the shell cannot find the command and returns "peer not found". `FABRIC_CFG_PATH` tells the peer CLI where to find `core.yaml`, the main configuration file. Without it, the peer errors with "cannot init crypto, path does not exist".

code :

`peer version`

You should see `Version: v2.5.15`.

Step 7 — Start the network and create the channel

code :

`cd ~/fabric-samples/test-network`

`./network.sh up createChannel`

```

pranav@DESKTOP-OSHSLSJI:~/fabric-samples/test-network$ cd ~/fabric-samples/test-network
./network.sh down
Using docker and docker-compose
Stopping network
[+] down 7/7
  ✓Container peer0.org1.example.com      Removed
  ✓Container orderer.example.com         Removed
  ✓Container peer0.org2.example.com      Removed
  ✓Network fabric_test                  Removed
  ✓Volume compose_orderer.example.com   Removed
  ✓Volume compose_peer0.org1.example.com Removed
  ✓Volume compose_peer0.org2.example.com Removed
Error response from daemon: get docker_orderer.example.com: no such volume
Error response from daemon: get docker_peer0.org1.example.com: no such volume
Error response from daemon: get docker_peer0.org2.example.com: no such volume
Removing remaining containers
Removing generated chaincode docker images
Untagged: dev-peer0.org2.example.com-basic_1.0-1d4d445573112813889f87c307bc2b5fbe664c87b24c035bee15cbcc7f59f864cfaf1398b52d341cb:latest
Deleted: sha256:27056269ed9759828700d7bfc21149ac76c17d725ae6f9a12c07cc42a04f539f
Untagged: dev-peer0.org1.example.com-basic_1.0-1d4d445573112813889f87c307bc2b5fbe664c87b24c035bee15cbcc7f59f163835f604aa885d60dc:latest
Deleted: sha256:f7c74fe5fd5a6a73117e649e923a5de0a595771f793ee8e1b0047fc35d2d315c

```



```

pranav@DESKTOP-0SHSLJI:~/fabric-samples/test-network$ ./network.sh up createChannel
Using docker and docker-compose
Creating channel 'mychannel'.
If network is not up, starting nodes with CLI timeout of '5' tries and CLI delay of '3' seconds and using dat
Bringing up network
LOCAL_VERSION=v2.5.15
DOCKER_IMAGE_VERSION=v2.5.15
/home/pranav/fabric-samples/test-network/./bin/cryptogen
Generating certificates using cryptogen tool
Creating Org1 Identities
+ cryptogen generate --config=./organizations/cryptogen/crypto-config-org1.yaml --output=organizations
org1.example.com
+ res=0
Creating Org2 Identities
+ cryptogen generate --config=./organizations/cryptogen/crypto-config-org2.yaml --output=organizations
org2.example.com
+ res=0
Creating Orderer Org Identities
+ cryptogen generate --config=./organizations/cryptogen/crypto-config-orderer.yaml --output=organizations
+ res=0
Generating CCP files for Org1 and Org2
[+] up 7/7
✓Network fabric_test Created
✓Volume compose_orderer.example.com Created
✓Volume compose_peer0.org1.example.com Created
✓Volume compose_peer0.org2.example.com Created
✓Container peer0.org1.example.com Created
✓Container orderer.example.com Created
✓Container peer0.org2.example.com Created
CONTAINER ID IMAGE COMMAND CREATED STATUS

```

Explain: This single command does everything needed to have a running network with a channel. Internally it uses **cryptogen** to generate TLS certificates and identity material (MSP) for Org1, Org2, and the Orderer. It starts three Docker containers — **orderer.example.com** on port 7050, **peer0.org1.example.com** on port 7051, and **peer0.org2.example.com** on port 9051. It then runs **configtxgen** to create **mychannel.block** (the channel genesis block), submits it to the orderer, joins both peers to the channel, and sets anchor peers so the two organizations can discover each other via gossip. The default channel name is **mychannel**.

Step 8 — Deploy the chaincode

code :

```

./network.sh deployCC \
  -ccn basic \
  -ccp ../asset-transfer-basic/chaincode-javascript \
  -ccl javascript

```

```

pranav@DESKTOP-0SHSLJI:~/fabric-samples/test-network$ ./network.sh deployCC \
  -ccn basic \
  -ccp ../asset-transfer-basic/chaincode-javascript \
  -ccl javascript
Using docker and docker-compose
Deploying chaincode on channel 'mychannel'
executing with the following
- CHANNEL_NAME: mychannel
- CC_NAME: basic
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-javascript
- CC_SRC_LANGUAGE: javascript
- CC_VERSION: 1.0
- CC_SEQUENCE: auto
- CC_END_POLICY: NA
- CC_COLL_CONFIG: NA
- CC_INIT_FCN: NA
- DELAY: 3
- MAX_RETRY: 5
- VERBOSE: false
executing with the following
- CC_NAME: basic
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-javascript
- CC_SRC_LANGUAGE: javascript
- CC_VERSION: 1.0
+ '[' false = true ']'
+ peer lifecycle chaincode package basic.tar.gz --path ../asset-transfer-basic/chaincode-javascript --lang
+ res=0
Chaincode is packaged
Installation chaincode on peers0 done?

```

```

1826-03-17 16:28:58.033 UTC 0001 INFO [chaincodeCmd] ClientWait -> txid [b424caca912525c6978bb1da73f421e25a00030499eab767783fa7e
atus (VALID) at localhost:7051
1826-03-17 16:28:58.037 UTC 0002 INFO [chaincodeCmd] ClientWait -> txid [b424caca912525c6978bb1da73f421e25a00030499eab767783fa7e
atus (VALID) at localhost:9051
Chaincode definition committed on channel 'mychannel'
Using organization 1
Querying chaincode definition on peer0.org1 on channel 'mychannel'...
Attempting to Query committed status on peer0.org1. Retry after 3 seconds.
+ peer lifecycle chaincode querycommitted --channelID mychannel --name basic
+ res=0
Committed chaincode definition for chaincode 'basic' on channel 'mychannel':
Version: 1.0, Sequence: 1, Endorsement Plugin: escc, Validation Plugin: vsc, Approvals: [Org1MSP: true, Org2MSP: true]
Query chaincode definition successful on peer0.org1 on channel 'mychannel'
Using organization 2
Querying chaincode definition on peer0.org2 on channel 'mychannel'...
Attempting to Query committed status on peer0.org2. Retry after 3 seconds.
+ peer lifecycle chaincode querycommitted --channelID mychannel --name basic
+ res=0
Committed chaincode definition for chaincode 'basic' on channel 'mychannel':
Version: 1.0, Sequence: 1, Endorsement Plugin: escc, Validation Plugin: vsc, Approvals: [Org1MSP: true, Org2MSP: true]
Query chaincode definition successful on peer0.org2 on channel 'mychannel'
Chaincode initialization is not required

```

Explain: `-ccn basic` is the name your chaincode will be called by in all future commands. `-ccp` is the path to the chaincode source code. `-ccl javascript` tells Fabric to use the Node.js runtime. Internally this runs the full Fabric chaincode lifecycle — package the source into a `.tar.gz`, install it on both peers, have Org1 approve the definition, have Org2 approve the definition (both approvals are required since majority endorsement is the default policy), and finally commit the definition to the channel. Once committed, two new Docker containers appear — one chaincode container per peer (`dev-peer0.org1...` and `dev-peer0.org2...`).

Step 9 — Set peer identity environment variables code :

```

export FABRIC_CFG_PATH=$HOME/fabric-samples/config
export PATH=$PATH:$HOME/fabric-samples/bin
export CORE_PEER_TLS_ENABLED=true
export CORE_PEER_LOCALMSPID="Org1MSP"
export
CORE_PEER_TLS_ROOTCERT_FILE=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt
export
CORE_PEER_MSPCONFIGPATH=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/users/Admin@org1.example.com/msp
export CORE_PEER_ADDRESS=localhost:7051

```

```

pranav@DESKTOP-05HSLJI:~/fabric-samples/test-network$ export PATH=$PATH:$HOME/fabric-samples/bin
export FABRIC_CFG_PATH=$HOME/fabric-samples/config

export CORE_PEER_TLS_ENABLED=true
export CORE_PEER_LOCALMSPID="Org1MSP"
export CORE_PEER_TLS_ROOTCERT_FILE=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.o
t
export CORE_PEER_MSPCONFIGPATH=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/users/Admin@org1.
export CORE_PEER_ADDRESS=localhost:7051

```

Explain: The `peer` CLI has no built-in session — it does not know who you are. Every command needs to know which organization's identity to use, which peer to connect to, and how to verify TLS. These five variables provide exactly that. `CORE_PEER_LOCALMSPID` declares "I am Org1". `CORE_PEER_MSPCONFIGPATH` points to the Admin user's private signing key and certificate. `CORE_PEER_TLS_ROOTCERT_FILE` provides the CA certificate to verify the

peer's TLS identity. **CORE_PEER_ADDRESS** is which peer to send requests to. These must be re-exported in every new terminal session.

Step 10 — Initialize the ledger with sample data

code :

```
peer chaincode invoke \  
-o localhost:7050 \  
--ordererTLSHostnameOverride orderer.example.com \  
--tls \  
--cafile  
$HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/  
orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem \  
-C mychannel \  
-n basic \  
--peerAddresses localhost:7051 \  
--tlsRootCertFiles  
$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/  
peer0.org1.example.com/tls/ca.crt \  
--peerAddresses localhost:9051 \  
--tlsRootCertFiles  
$HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/  
peer0.org2.example.com/tls/ca.crt \  
--waitForEvent \  
-c '{"function": "InitLedger", "Args": []}'
```

```
pranav@DESKTOP-05HSLJ1:~/fabric-samples/test-network$ peer chaincode invoke \  
-o localhost:7050 \  
--ordererTLSHostnameOverride orderer.example.com \  
--tls \  
--cafile $HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscac  
erts/tlsca.example.com-cert.pem \  
-C mychannel \  
-n basic \  
--peerAddresses localhost:7051 \  
--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.co  
m/tls/ca.crt \  
--peerAddresses localhost:9051 \  
--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.co  
m/tls/ca.crt \  
--waitForEvent \  
-c '{"function": "InitLedger", "Args": []}'  
2026-03-17 16:37:35.757 UTC 0001 INFO [chaincodeCmd] ClientWait -> txid [f982da882feb2411792ddb2c62b47591512c0269f789efe8ab9ce462  
atus (VALID) at localhost:9051  
2026-03-17 16:37:35.757 UTC 0002 INFO [chaincodeCmd] ClientWait -> txid [f982da882feb2411792ddb2c62b47591512c0269f789efe8ab9ce462  
atus (VALID) at localhost:7051  
2026-03-17 16:37:35.758 UTC 0003 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200
```

Explain: **invoke** is used for any operation that writes to the ledger. **-o localhost:7050** is the orderer address. **--tls** and **--cafile** enable TLS and provide the CA cert to verify the orderer. **-C mychannel** and **-n basic** identify which channel and chaincode. The two **--peerAddresses** and **--tlsRootCertFiles** pairs are the critical part — the default endorsement policy requires a majority of organizations to endorse. With two orgs, both must sign. Sending to only one peer results in **ENDORSEMENT_POLICY_FAILURE**. **--waitForEvent** blocks until both peers confirm the

transaction is committed (status VALID) rather than just submitted. **InitLedger** seeds the ledger with six sample assets.

Step 11 — Query all assets

code :

```
peer chaincode query \  
-C mychannel \  
-n basic \  
-c '{"Args":["GetAllAssets"]}'
```

```
pranav@DESKTOP-OSHSLJI:~/fabric-samples/test-network$ peer chaincode query \  
-C mychannel \  
-n basic \  
-c '{"Args":["GetAllAssets"]}'  
[{"AppraisedValue":300,"Color":"blue","ID":"asset1","Owner":"Tomoko","Size":5,"docType":"asset"}, {"AppraisedValue":400,"Color":"red","ID":"asset2","Owner":"Brad","Size":5,"docType":"asset"}, {"AppraisedValue":500,"Color":"green","ID":"asset3","Owner":"Jin Soo","Size":10,"docType":"asset"}, {"AppraisedValue":600,"Color":"yellow","ID":"asset4","Owner":"Max","Size":10,"docType":"asset"}, {"AppraisedValue":700,"Color":"black","ID":"asset5","Owner":"Adriana","Size":15,"docType":"asset"}, {"AppraisedValue":800,"Color":"white","ID":"asset6","Owner":"Michel","Size":15,"docType":"asset"}]
```

Explain: **query** is for read-only operations. No transaction is created, no block is written, no endorsement from multiple peers is needed. It is fast and cheap. This returns all six assets seeded by **InitLedger**:

```
[ {"ID":"asset1","Color":"blue","Size":5,"Owner":"Tomoko","AppraisedValue":300},  
  {"ID":"asset2","Color":"red","Size":5,"Owner":"Brad","AppraisedValue":400},  
  {"ID":"asset3","Color":"green","Size":10,"Owner":"Jin Soo","AppraisedValue":500},  
  {"ID":"asset4","Color":"yellow","Size":10,"Owner":"Max","AppraisedValue":600},  
  {"ID":"asset5","Color":"black","Size":15,"Owner":"Adriana","AppraisedValue":700},  
  {"ID":"asset6","Color":"white","Size":15,"Owner":"Michel","AppraisedValue":800} ]
```

Step 12 — Read a specific asset

code :

```
peer chaincode query \  
-C mychannel \  
-n basic \  
-c '{"Args":["ReadAsset","asset5"]}'
```

```
pranav@DESKTOP-OSHSLJI:~/fabric-samples/test-network$ peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset5"]}'  
{"AppraisedValue":700,"Color":"black","ID":"asset5","Owner":"Adriana","Size":15,"docType":"asset"}
```

Result:

```
{"AppraisedValue":700,"Color":"black","ID":"asset5","Owner":"Adriana","Size":15}
```

Explain: **ReadAsset** takes an asset ID and returns that single asset's data from the world state. The world state is a key-value database that stores the current (latest) value for each asset. The full transaction history is stored in the blockchain, but queries read from this faster world state.

Step 13 — Create a new asset

code :

```
peer chaincode invoke \
```

```
-o localhost:7050 \
```

```
--ordererTLSHostnameOverride orderer.example.com \
```

```
--tls \
```

```
--cafile
```

```
$HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/  
orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem \
```

```
-C mychannel \
```

```
-n basic \
```

```
--peerAddresses localhost:7051 \
```

```
--tlsRootCertFiles
```

```
$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/  
peer0.org1.example.com/tls/ca.crt \
```

```
--peerAddresses localhost:9051 \
```

```
--tlsRootCertFiles
```

```
$HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/  
peer0.org2.example.com/tls/ca.crt \
```

```
--waitForEvent \
```

```
-c '{"function":"CreateAsset","Args":["asset10","blue","10","Pranav","1000"]}'
```

```
pranav@DESKTOP-OSHSLJI:~/fabric-samples/test-network$ peer chaincode invoke \  
-o localhost:7050 \  
--ordererTLSHostnameOverride orderer.example.com \  
--tls \  
--cafile $HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tls  
.pem \  
-C mychannel \  
-n basic \  
--peerAddresses localhost:7051 \  
--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example  
--peerAddresses localhost:9051 \  
--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example  
--waitForEvent \  
-c '{"function":"CreateAsset","Args":["asset10","blue","10","Pranav","1000"]}'  
2026-03-17 16:38:38.378 UTC 0001 INFO [chaincodeCmd] ClientWait -> txid [d88abbf8b7aaff8416fb4a8914a4d01c8064d053ebc5145787034]  
atus (VALID) at localhost:7051  
2026-03-17 16:38:38.371 UTC 0002 INFO [chaincodeCmd] ClientWait -> txid [d88abbf8b7aaff8416fb4a8914a4d01c8064d053ebc5145787034]  
atus (VALID) at localhost:9051  
2026-03-17 16:38:38.371 UTC 0003 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200  
{"Color":"blue","Size":10,"Owner":"Pranav","AppraisedValue":1000}
```

Explain: **CreateAsset** takes five arguments in order — [ID, Color, Size, Owner, AppraisedValue]. This is a write operation so both peers must endorse. The orderer packages the endorsed transaction into a block and delivers it to both peers. Both confirm VALID and the asset is permanently recorded on the ledger.

code :

```
peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset10"]}'
```

```
pranav@DESKTOP-OSHSLJI:~/fabric-samples/test-network$ peer chaincode query \  
-C mychannel \  
-n basic \  
-c '{"Args":["ReadAsset","asset10"]}'  
{"AppraisedValue":1000,"Color":"blue","ID":"asset10","Owner":"Pranav","Size":10}  
# {"AppraisedValue":1000,"Color":"blue","ID":"asset10","Owner":"Pranav","Size":10}
```

The transaction flow showing what happens inside Fabric every time you run **invoke** or **query** Shows what happens inside Fabric every time you run **peer chaincode invoke** — the proposal, endorsement, ordering, and commit flow — and why both peers are always required for any write operation.

Step 14 — Transfer asset ownership code :

```
peer chaincode invoke \  
-o localhost:7050 \  
--ordererTLSHostnameOverride orderer.example.com \  
--tls \  
--cafile  
$HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/  
orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem \  
-C mychannel \  
-n basic \  
--peerAddresses localhost:7051 \  
--tlsRootCertFiles  
$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/  
peer0.org1.example.com/tls/ca.crt \  
--peerAddresses localhost:9051 \  
--tlsRootCertFiles  
$HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/  
peer0.org2.example.com/tls/ca.crt \  
--waitForEvent \  
-c '{"function": "TransferAsset", "Args": ["asset10", "Rahul"]}'
```

```
pranav@DESKTOP-OSHSLJI:~/fabric-samples/test-network$ peer chaincode invoke \  
-o localhost:7050 \  
--ordererTLSHostnameOverride orderer.example.com \  
--tls \  
--cafile $HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.c  
.pem \  
-C mychannel \  
-n basic \  
--peerAddresses localhost:7051 \  
--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt \  
--peerAddresses localhost:9051 \  
--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt \  
--waitForEvent \  
-c '{"function": "TransferAsset", "Args": ["asset10", "Rahul"]}'  
2026-03-17 16:41:28.692 UTC 0001 INFO [chaincodeCmd] ClientWait -> txid [e834ab102eac67d6bbf5759291d3c3bd92083d1a340d1f5bd35104c2e387019] committed  
status (VALID) at localhost:7051  
2026-03-17 16:41:28.692 UTC 0002 INFO [chaincodeCmd] ClientWait -> txid [e834ab102eac67d6bbf5759291d3c3bd92083d1a340d1f5bd35104c2e387019] committed  
status (VALID) at localhost:7051  
2026-03-17 16:41:28.692 UTC 0003 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200 payload:"Pranav"
```

Explain: **TransferAsset** takes [assetID, newOwner]. It reads the current asset from the world state, updates the Owner field, and writes it back. The function returns the previous owner ("Pranav") as confirmation. Both peers endorse, the orderer commits the block, and the ledger is updated permanently.

Verify the ownership change:

code :

```
peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset10"]}'
```

```
# {"AppraisedValue":1000,"Color":"blue","ID":"asset10","Owner":"Rahul","Size":10}
```

```
2026-03-17 16:41:28.692 UTC 0003 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200 payload:"Pranav"
pranav@DESKTOP-OSHSLJI:~/fabric-samples/test-network$ peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset10"]}'
{"AppraisedValue":1000,"Color":"blue","ID":"asset10","Owner":"Rahul","Size":10}
```

Owner is now **Rahul**

Step 15 — To stop/bring down the Hyperledger Fabric network, run:

[./network.sh](#) down

```
pranav@DESKTOP-OSHSLJI:~/fabric-samples/test-network$ ./network.sh down
Using docker and docker-compose
Stopping network
[+] down 7/7
✓Container orderer.example.com          Removed
✓Container peer0.org2.example.com        Removed
✓Container peer0.org1.example.com        Removed
✓Network fabric_test                    Removed
✓Volume compose_peer0.org1.example.com  Removed
✓Volume compose_peer0.org2.example.com  Removed
✓Volume compose_orderer.example.com     Removed
Error response from daemon: get docker_orderer.example.com: no such volume
Error response from daemon: get docker_peer0.org1.example.com: no such volume
Error response from daemon: get docker_peer0.org2.example.com: no such volume
Removing remaining containers
Removing generated chaincode docker images
Untagged: dev-peer0.org2.example.com-basic_1.0-1d4d445573112813889f87c307bc2b5fbe664c87b24c035f864cfa1398b52d341cb:latest
Deleted: sha256:aa29d3b9fb04e6316131fbf334a23f28d8a1646fa9d40090916c02d54bc5b76d
Untagged: dev-peer0.org1.example.com-basic_1.0-1d4d445573112813889f87c307bc2b5fbe664c87b24c035163835f604aa885d60dc:latest
Deleted: sha256:08d49c295856f950367880744bcacfa56a7134bf208cacfab8a29ef9221703c
```

Conclusion

The Hyperledger Fabric network was successfully implemented, demonstrating asset creation, querying, and transfer using chaincode. The process highlights key concepts such as consensus, immutability, and secure transaction flow. Overall, it provides a clear understanding of permissioned blockchain systems and their real-world applications in asset management.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY**
Department of Information Technology
Academic Year: 2025-26

ITC801: Blockchain and DLT Lab
Experiment 12

Aim: Mini Project

Roll No.	04
Name	Mohit Patil
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1:Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO3:Implement smart contracts in Ethereum using different development frameworks. CO4:Develop applications in permissioned Hyperledger Fabric network. CO5:Interpret different Crypto assets and Crypto currencies CO6:Analyze the use of Blockchain with AI, IoT and Cyber Security using case studies.

Blockchain-Based Voting System

Secure, transparent, and tamper-proof electronic voting using distributed ledger technology

Group 2 :

Prathamesh Palve D20A_2

Bhagyesh Patil D20A_3

Mohit Patil D20A_4

Vedant Sanap D20A_50

Introduction to Digital Voting

What is Voting?

Voting is the fundamental process of democratic decision-making where eligible participants express their preferences to elect representatives or decide policies.

Problems in Traditional Systems

- Election fraud and ballot stuffing
- Lack of transparency in counting
- Manual errors in vote tallying
- Difficult audit and verification
- High operational costs

What is Blockchain?

A distributed digital ledger that records transactions across multiple computers in a way that makes them secure, transparent, and immutable.



Decentralization

Data stored across multiple nodes instead of single authority



Transparency

All transactions visible to authorized participants



Immutability

Once recorded, data cannot be altered or deleted



Problem Statement

Trust Deficit

Current systems lack mechanisms to verify vote integrity without manual recounts

Security Vulnerabilities

Centralized databases can be hacked, manipulated, or corrupted by malicious actors

Need for Innovation

Modern democracy requires secure, transparent, and accessible voting infrastructure

Proposed Solution

A blockchain-based voting system leverages distributed ledger technology to create an **immutable, transparent, and secure** voting platform that eliminates fraud while maintaining voter privacy.



System Architecture



Admin

Election administrator manages candidates and election parameters



Voter

Registered voters authenticate and cast encrypted votes



Smart Contract

Ethereum-based contract executes voting logic and stores votes



Results

Real-time, verifiable election results displayed to all participants

Smart Contracts Implementation

Voting Contract

Core smart contract written in Solidity that manages the entire voting process:

- Stores candidate list and vote counts
- Records voter addresses to prevent duplicate voting
- Tallies votes automatically
- Provides read-only access to results

Admin Control Contract

Separate contract for election management:

- Adds or removes candidates from ballot
- Starts and ends voting period
- Resolves disputes if necessary
- Maintains election integrity

Working Process



Admin Adds Candidates

Election administrator deploys contract and populates ballot with eligible candidates



Voters Register

Eligible voters register using unique credentials verified by the system



Votes Cast

Authenticated voters submit encrypted votes recorded on blockchain



Blockchain Storage

Each vote becomes immutable transaction visible to authorized auditors



Results Displayed

Real-time results update automatically as votes are counted

Technologies & Implementation



Solidity

Object-oriented programming language for writing Ethereum smart contracts



Remix IDE

Web-based integrated development environment for compiling and testing contracts



MetaMask / Ganache

Wallet interface connects frontend to blockchain; Ganache provides local test network



Ethereum Blockchain

Distributed ledger platform executing smart contracts and storing vote records

Advantages, Limitations & Future

Advantages

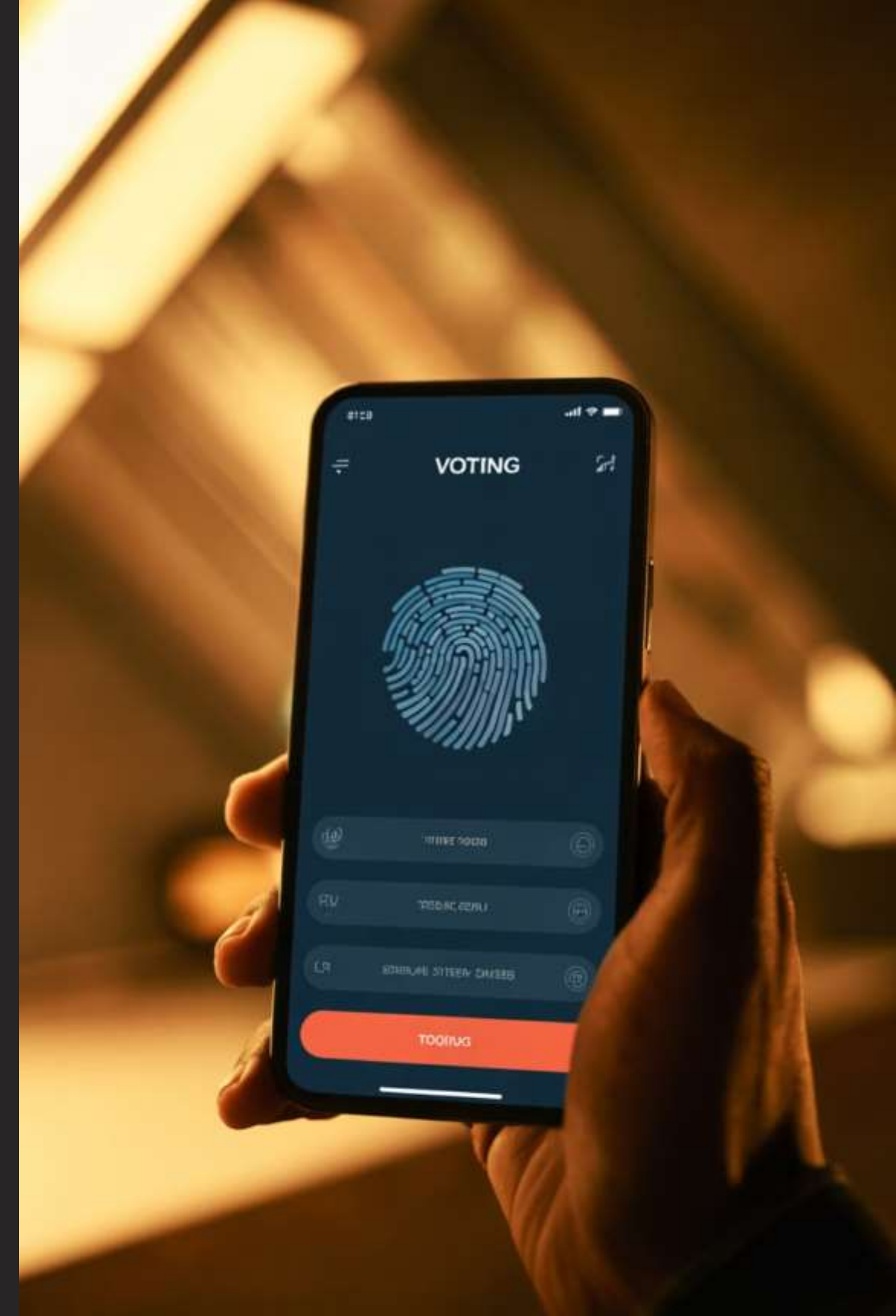
- **Transparency:** Anyone can verify vote counts
- **Security:** Cryptographic protection prevents tampering
- **Immutability:** Once recorded, votes cannot be altered
- **Decentralization:** No single point of failure

Limitations

- Requires internet connectivity
- Gas fees for blockchain transactions
- Technical complexity for non-technical users
- Scalability concerns with large elections

Future Enhancements

Mobile voting applications, biometric authentication, integration with government electoral systems, zero-knowledge proofs for enhanced privacy



Thankyou